

Coding, Deploying, Evaluating, and Live Database Communication Between Multi- Agent Systems with RAG in Python/VSCode

Written by Ami Gates, 2026

© 2026 Dr. Ami M. Gates|
All Rights Reserved | Free
Distribution Only

Notice of Copyright and Use

Copyright (c) 2026 Dr. Ami M. Gates. All rights reserved.

This material is provided for free educational use and distribution only.

You are permitted to download, read, and share this document in its original form.

Strictly Prohibited:

1. You may not modify, alter, or adapt any part of this work. If you use part of it – please reference the writer – Dr. Ami Gates,
2. You may not sell, rent, or commercialize this material or its contents.
3. You may not remove, alter, or obscure any copyright notices or watermarks.

Outline

This very large slide deck will progress learners through the following topics:

- 1) Understanding the goal of building a multiple AI Agentic Workflow (MAgent App) with RAG elements via Python in VSCode such that the App uses an LLM, enables Agents to communicate, and creates a useable output.
- 2) Once MAgent is successful locally, this guide will then extend into deploying MAgent live on the web.
- 3) Next, another Agentic App will be coded in Python/VSCode that will be used to evaluate the results generated by MAgent. Once this works locally, it too will be deployed live on the web again using appropriate tools, methods, and apps.
- 4) To enable live web deployment of multiple Agent-Based Apps (in this case MAgent and Eval) there will also be a need for App live communication. This will be done via Supabase (a cloud-based database platform)
- 5) Other libraries/app/elements used for these goals will include Stream lit Python libraries, CrewAI libraries, Render (for deployment) GitHub for file sharing, Supabase and its contained PostgreSQL database, Python in general, VSCode, and OpenAI for the LLM.
- 6) NOTICE: Some applications, especially those that make use of LLMs and live deployment are not free. We will use the least expensive versions but students will be required to use a credit card and to pay for OpenAI and Render usage.

Overview of Steps – Understanding the Goals

- 1) Understand, discuss, and practice with vibe coding and prompt engineering.
- 2) Create and set up the entire environment for local coding of the multiple agent with RAG workflows (also called systems, and apps).
 - 1) Review a range of options and methods.
 - 2) Understand the reasons for the given choices as well as alternatives.
- 3) Once the environment and all local libraries are set up, program in Python and VSCode a multiple Agent workflow (App) with RAG elements. This workflow/App is often referred to throughout these slides as MAgent for short.
- 4) Practice with coding a multi agent system, practice with various RAG elements such as the use of pdf files or specific websites to “tune” LLM responses or Agent goals, to use RAG for a database, as well as RAG for web searches (via Serper), etc.
- 5) CrewAI and Streamlit as well as other tools will be used.
- 6) The local MAgent will be locally tested and viewed locally as a webpage (not yet “live”).
- 7) Next, further tools will be discussed to live-deploy the MAgent (multiple agentic workflow with RAG) onto the web for anyone to use.
- 8) Output (in this case the first output is a Markdown file) will be analyzed and discussed.
- 9) Render, Github, and Streamlit together will be used to live-deploy the MAgent App. This process contains quite a bit of learning and several steps.
- 10) Once the above is complete and successful, an evaluation Agent will be coded in Python/VSCode that will evaluate (grade/judge) the output generated by the MAgent.
- 11) This will first be coded and tested locally. Once debugged and functional, the code will be updated, and we will learn about and employ a cloud-based data base (Supabase and PostgreSQL) so that we can also deploy the Eval App live on the web and it can live communicate with the output of the MAgent App (as well as other things).
- 12) We will complete several steps to accomplish this and will need to update the code of Eval and MAgent so that they communicate with a cloud database and each other live on the web.
- 13) Our final result will be two live web-based and deployed agentic applications that can live –communicate.

Vibe Coding and Prompt Engineering – Working in and with AI

Class Discussion

- 1) What is vibe coding?
- 2) What is Prompt Engineering?

Setting Up the AI-Coding Environment

Gates

Overall Goals and Related Challenges

- For building a modern, multi-agent AI system, **Visual Studio Code (VS Code)** combined with a lightweight Python environment manager like **venv** or **Conda** is the industry standard.
- To clarify a common point of confusion: **Anaconda and VS Code are not mutually exclusive**. VS Code is a code editor (where you type code), while Anaconda is an environment/package manager (how you install Python and libraries). However, full Anaconda is bloated for this task. Using **VS Code** with **Conda** (via the lightweight **Miniconda**) or Python's native **venv** provides the speed and control you need.
- We must use Python 3.12 (not a newer one) [as of 2026]

Install Visual Studio (VS) Code

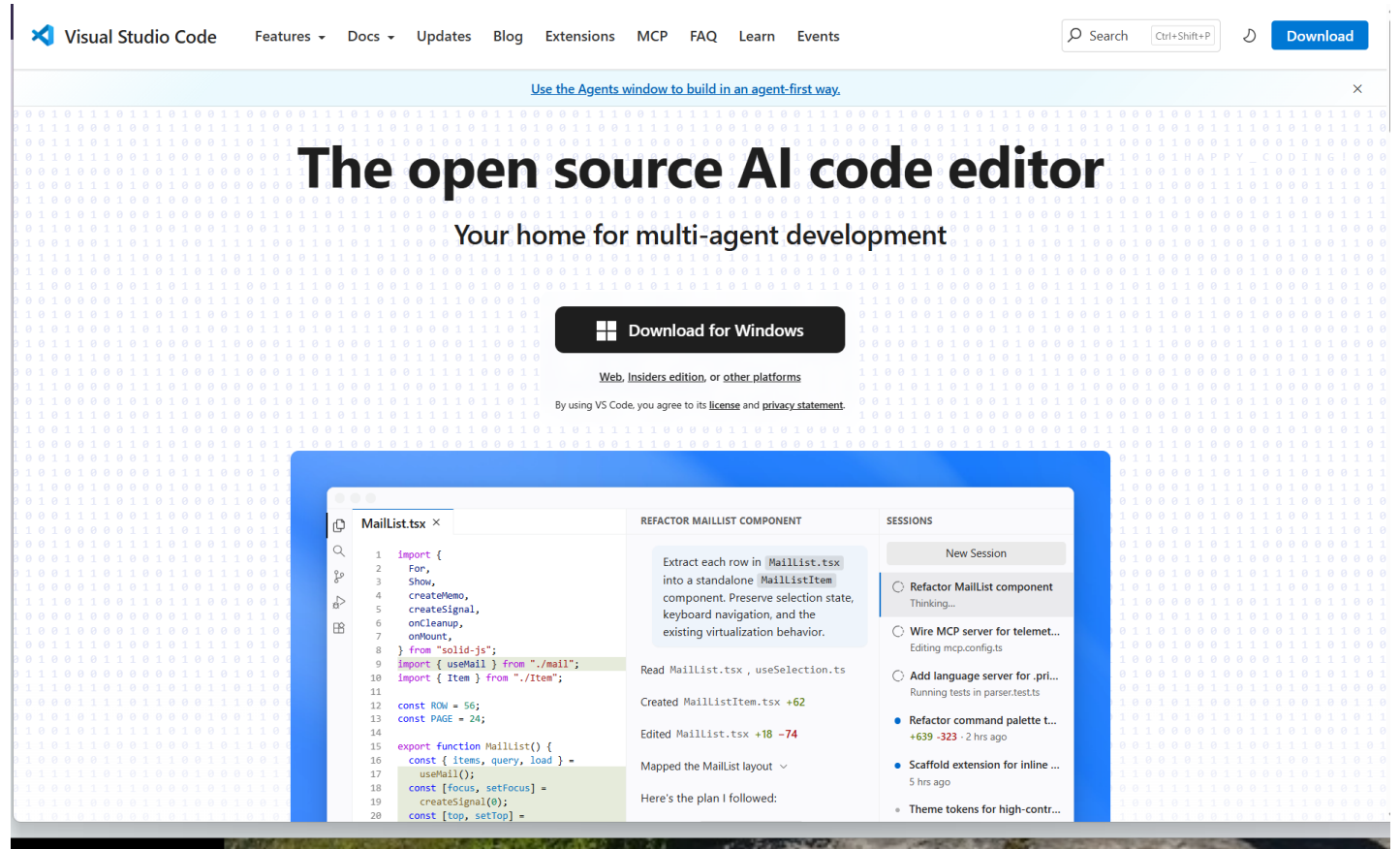
Step 1: Download and install VS Code.

- 1) Go specifically to: code.visualstudio.com (Use ONLY this exact link). This is dedicated a website just for the lightweight, free editor.
- 2) **Do not go to** <https://visualstudio.microsoft.com/> unless you want all of Visual Studio (which is very large and not necessary for our purposes).

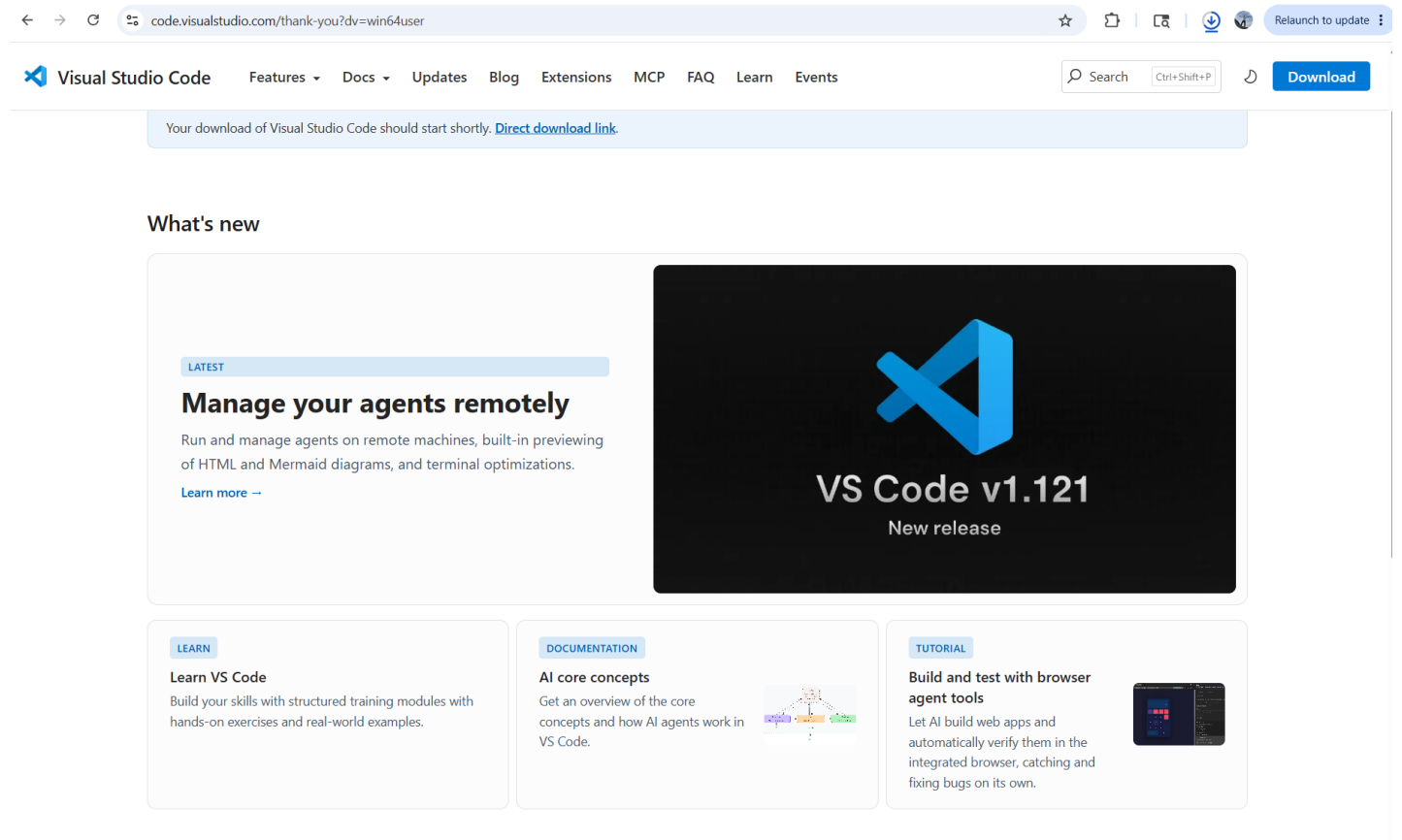
Step 2: Download the File

- Once that page loads, you should immediately see the **icon** on your screen.
- Click the button that says "**Download for Windows**" (or "Download for Mac" if you are on an Apple computer).
- The download will start automatically. (Actually, this does not always happen and you may need to double click the .exe file to start the install.

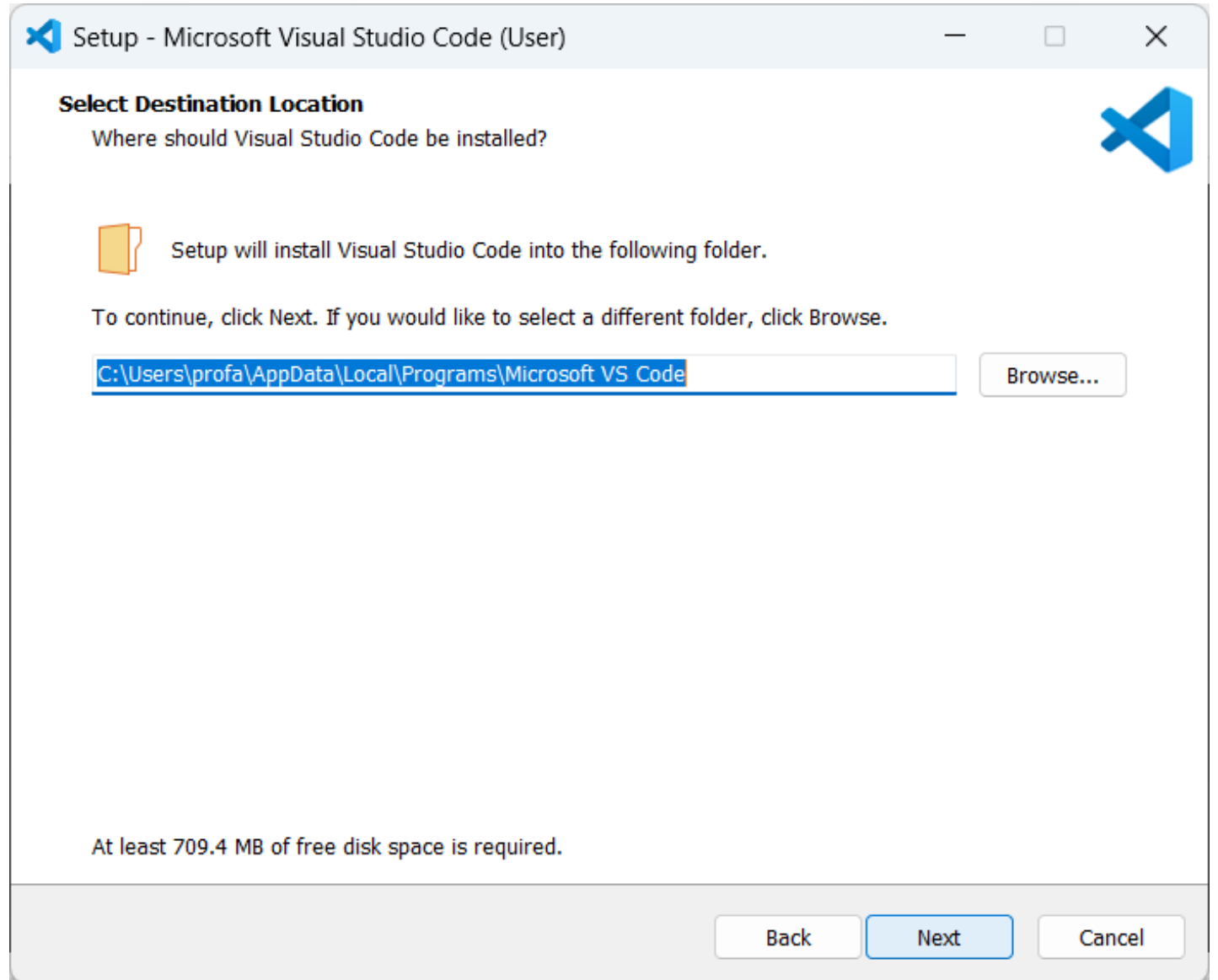
This is the
Site for the
VS Code
Download



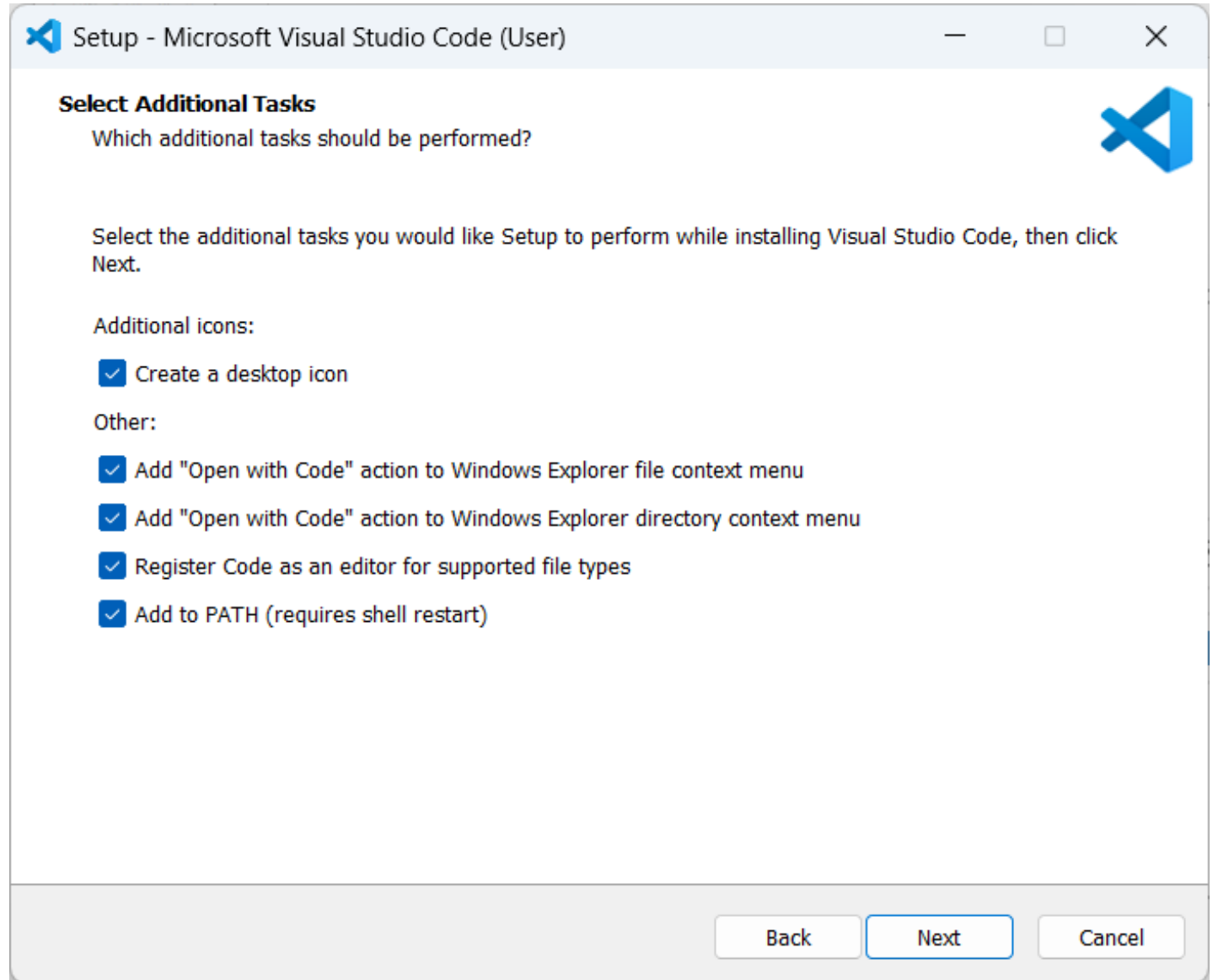
What you See As the File Downloads



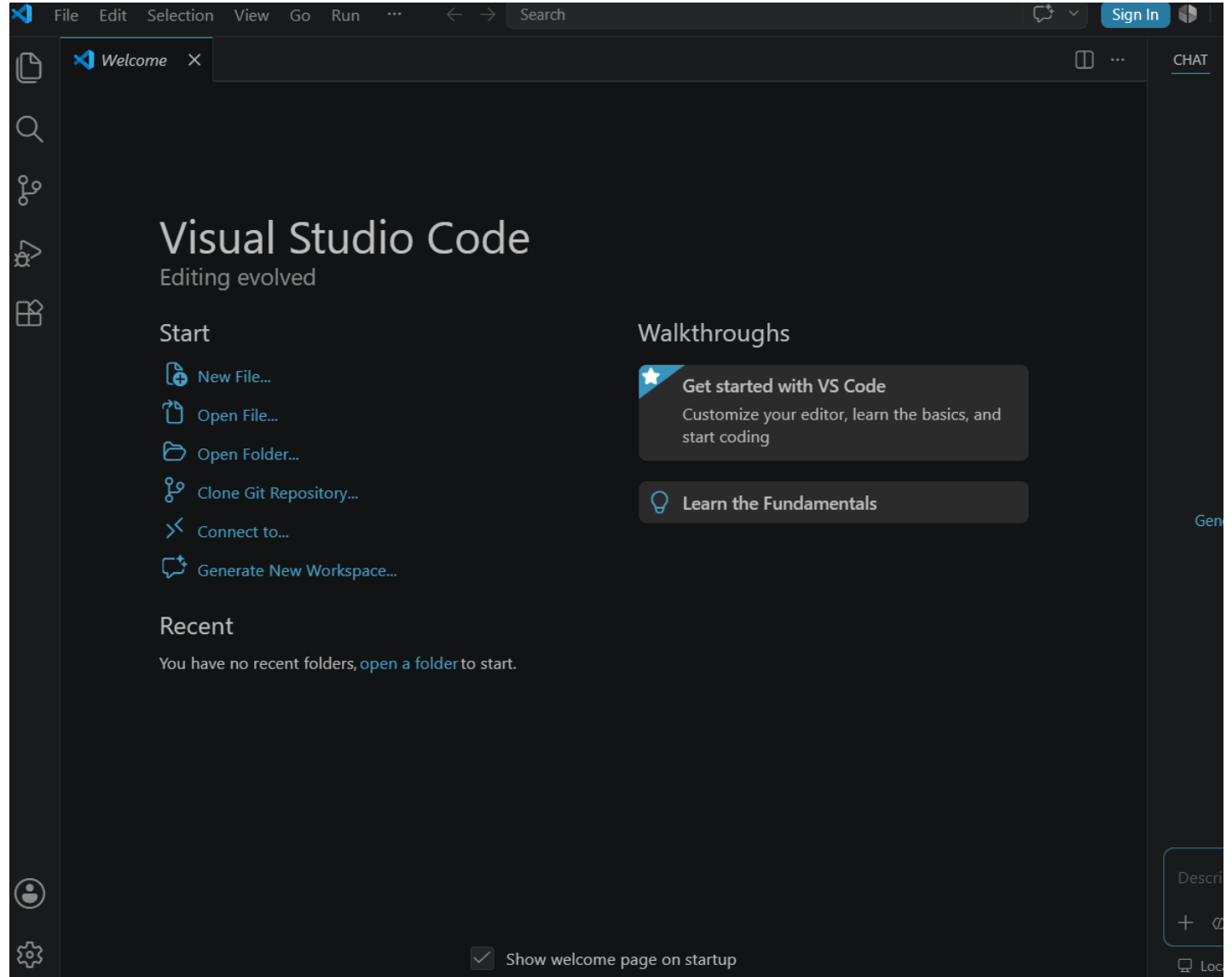
Make Sure
you Know
Where it Is



Make Selections



Launch VS Code – This is What you Will See



Create A Project Folder

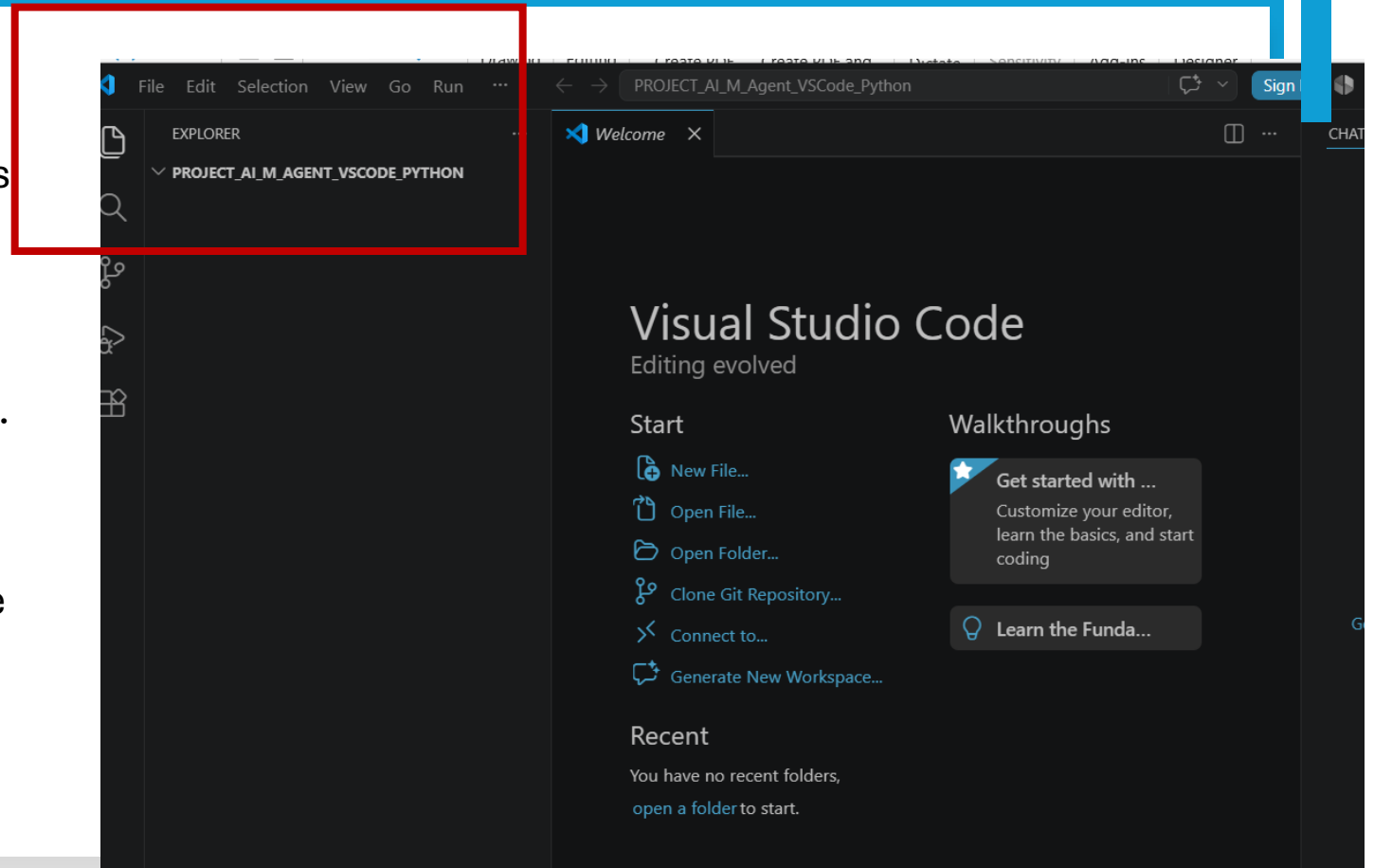
It is best if your project files need a dedicated home on your computer so Python knows where to find them.

1. Navigate to a place of your choosing like your **Desktop** or **Documents** folder or whatever.
2. Create a new Folder (for this Project) and name it something like `multi_agent_project`

Example: I have a path to where I do all my AI work and in that path I created a new folder called: `PROJECT_AI_M_Agent_VSCode_Python`

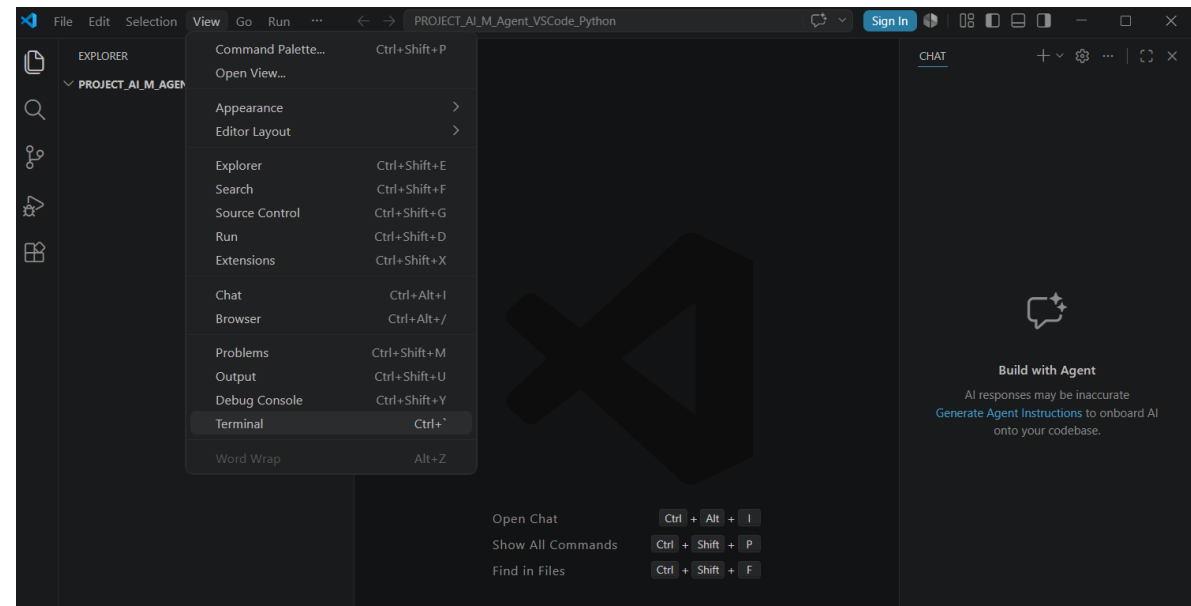
Connect Your Folder to VS Code

- Opening the folder directly in VS Code sets your working location automatically.
- Open the **VS Code** application.
- In the top menu bar, click **File > Open Folder...** (On Mac, it may just say **Open...**).
- Find and select your new AI Project folder, then click **Open**.
- You will see the folder name appear on the left-hand sidebar under the **Explorer** tab.



Open the Terminal inside VS Code

- In the top menu bar of VS Code, click **Terminal > New Terminal**.
- Or – you can choose “view” → “terminal”
- A panel will open at the bottom of your screen.
- You will see a command prompt ending in your folder name (e.g., ...\\multi_agent_project>).
- **This location is critical** because any files you create or run will happen right here.



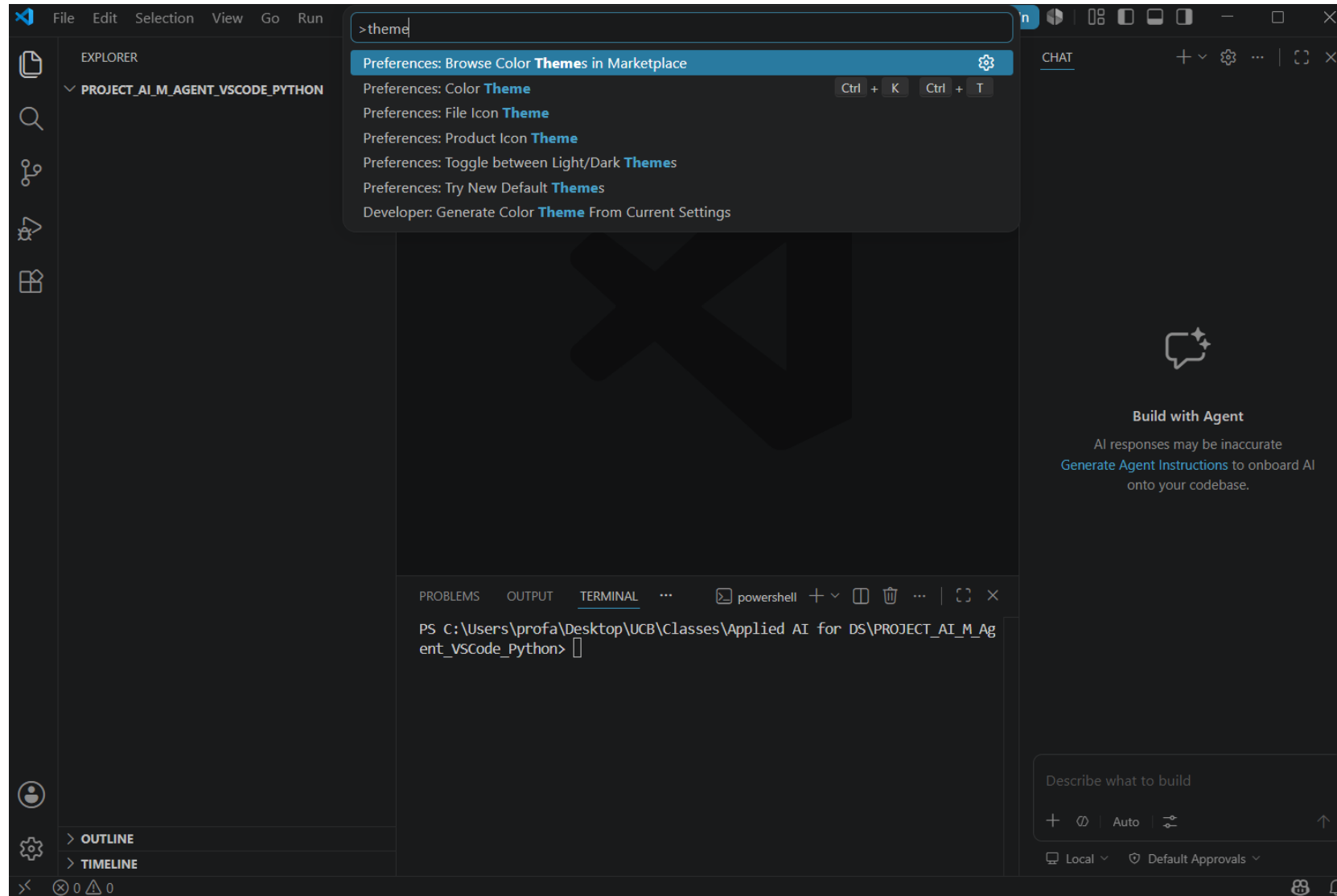
To Change From Black to White

Do *ctrl shift p*

Then, start to type “theme” and choose “color theme”.

There are many color options.

However, for this class (and for creating screen images) please use a white background so that everyone can see it.



FileEditSelectionViewGoRun

EXPLORER

PROJECT_AI_M_AGENT_VSCODE_PYTHON

Light 2026 Default Light
Light (Visual Studio) Visual Studio Light
Light Modern
Light+
Quiet Light
Solarized Light
Dark 2026 Default Dark
Abyss
Dark (Visual Studio) Visual Studio Dark
Dark Modern
Dark+
Kimbie Dark
Monokai
Monokai Dimmed

CHAT

Build with Agent

AI responses may be inaccurate
Generate Agent Instructions to onboard A
onto your codebase.

Describe what to build

Local Default Approvals

PROBLEMS OUTPUT TERMINAL powershell

PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python>

Next – Set Up the Rest of the Environment To AI -Dev

- Using **pure VS Code paired with native Python (3.12) virtual environments** is significantly better for production-grade AI agent development than Anaconda. (Anaconda is heavy, prone to massive dependency conflicts, and hides the underlying file structures.)
- The industry-standard method for using VS Code to build AI agents is to pair it with **Python's native venv module** and the **pip package manager**. This keeps your project lightweight, fast, and completely isolated.
- !!! You must use Python 3.12 (at this time) as many AI tools will not work with new Python versions.

Workflow (steps) to use venv and pip and to Bypass Anaconda.

You do not need to remove Anaconda. Just do the following:

- 1) **Create a Native Virtual Environment (venv)**
- 2) Activate the Environment in VS Code
- 3) Fast-Track Library Installation with ***requirements.txt***
- 4) Run the Environment Check

There are many steps needed to complete the above.

Let's look at each....

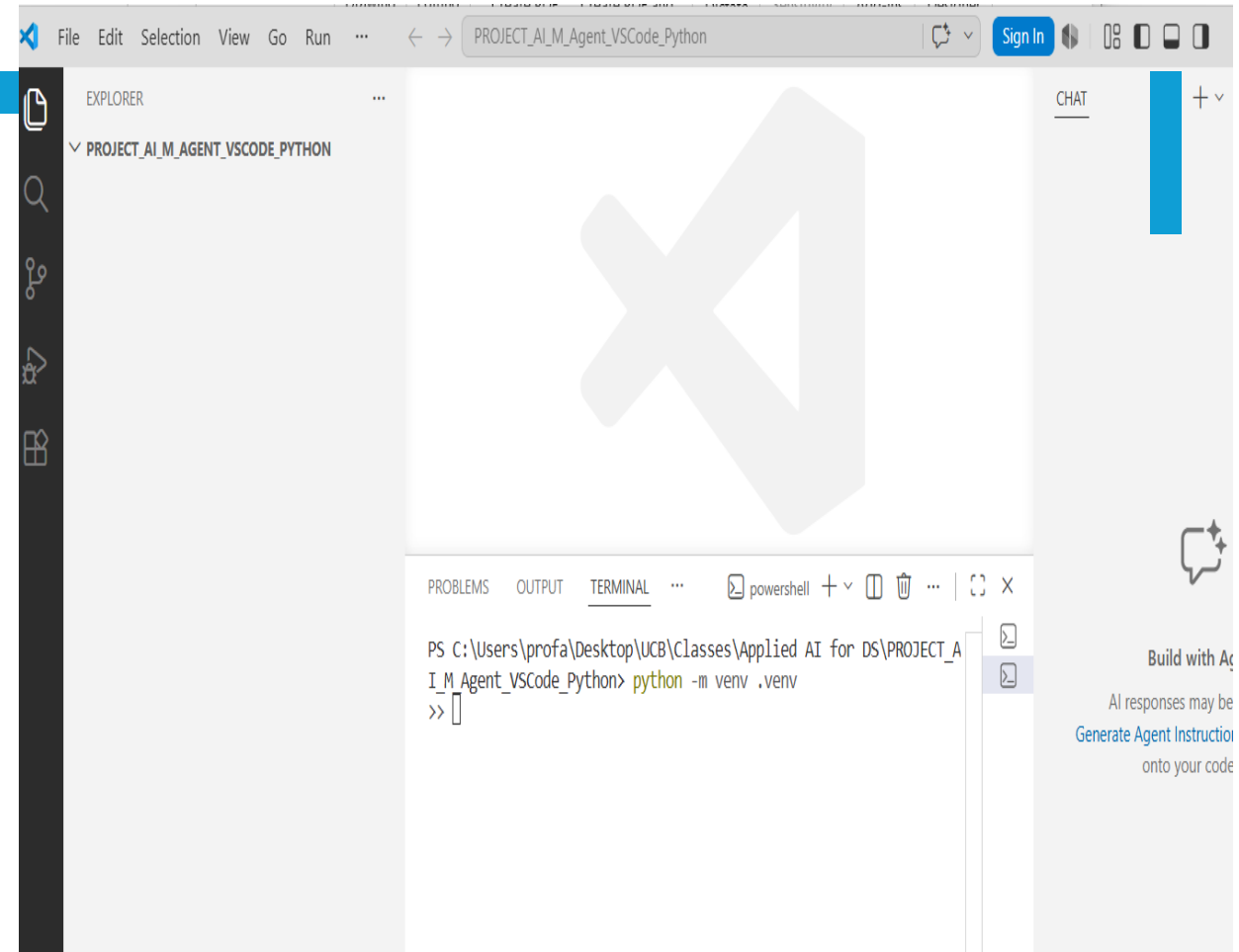
Step 1: Create a Native Virtual Environment (venv)

Create a dedicated, microscopic environment inside your project folder.

- 1) Open your project folder by name in VS Code.
- 2) Open a new terminal inside VS Code by going to Terminal > New Terminal (or press Ctrl+`).
- 3) Run the following command** to create a native Python environment named .venv
(You can do this for Windows/macOS/Linux)

python -m venv .venv

Note: What Does this Do? A hidden folder named .venv will appear in the project file explorer. This folder will house all the specific AI agent libraries, separate from the rest of the computer.



Error Message and Solution

If you try to create venv and get the following message (which would contain your path name and not mine 😊) then do what the next slide tells you.

Note that this error occurs because **Windows has a built-in "shortcut" that overrides your actual Python installation**, sending you to the Microsoft Store instead of running Python in your terminal.

Common Error (or similar)

Python was not found; run without arguments to install from the Microsoft Store, or disable this shortcut from Settings > Apps > Advanced app settings > App execution aliases.

PS

*C:\Users\profa\Desktop\...\PROJECT_AI_M_Agent_VSCo
de_Python>*

How to Fix the Error (Official Fix)

- To completely bypass Anaconda, the computer must have a standard version of Python installed globally.
- You MUST use Python 3.12 (not newer) at this time!
- Go to **python.org/downloads** and download **Python 3.12** for Windows (or your OS).
- Use: [Python 3.12.10 - April 8, 2025](#), Download [Windows installer \(64-bit\)](#) if using Windows (or similar for MAC). PAY ATTENTION. If you download the wrong one you will have a headache and lots of wasted time fixing it.
- Run the installer.
- **CRITICAL NOTE:** You must check the box at the bottom of the installer window that says "Add python.exe to PATH". If you miss this, you will get the exact same error again.
- Click **Install Now**.
- **Restart VS Code** completely so it registers the new system paths.
- Open a new terminal in VS Code and run the command again:

```
python -m venv .venv
```

Important Notes: Keep VS Code and your AI – Coding env Separate from Anaconda

- Note: Installing standalone Python (3.12 specifically!!) will **not** disrupt your existing Anaconda or Spyder installations, provided you manage your paths correctly.
- Anaconda operates in its own isolated environment directories. As long as you explicitly use the .venv virtual environment inside VS Code, the two ecosystems will remain separate.

1. Always Verify the Active Interpreter in VS Code

- Before running code or installing libraries, look at the **bottom-right corner** of the VS Code window.
- It should explicitly display ('.venv': venv).
- If it shows an Anaconda path (e.g., ('base': conda)), your terminal will default to Anaconda.
- Fix this by pressing Ctrl+Shift+P (or Cmd+Shift+P on Mac), typing **Python: Select Interpreter**, and choosing the .venv inside your project folder. (There are steps to this that follow...)

2. Force the Terminal to Use the Active Environment

- When installing libraries, never type a generic pip install command, as your computer might accidentally default to the Anaconda pip global path. Example:

On Windows `.\venv\Scripts\python -m pip install -r requirements.txt`

On Mac/Linux `./venv/bin/python -m pip install -r requirements.txt`

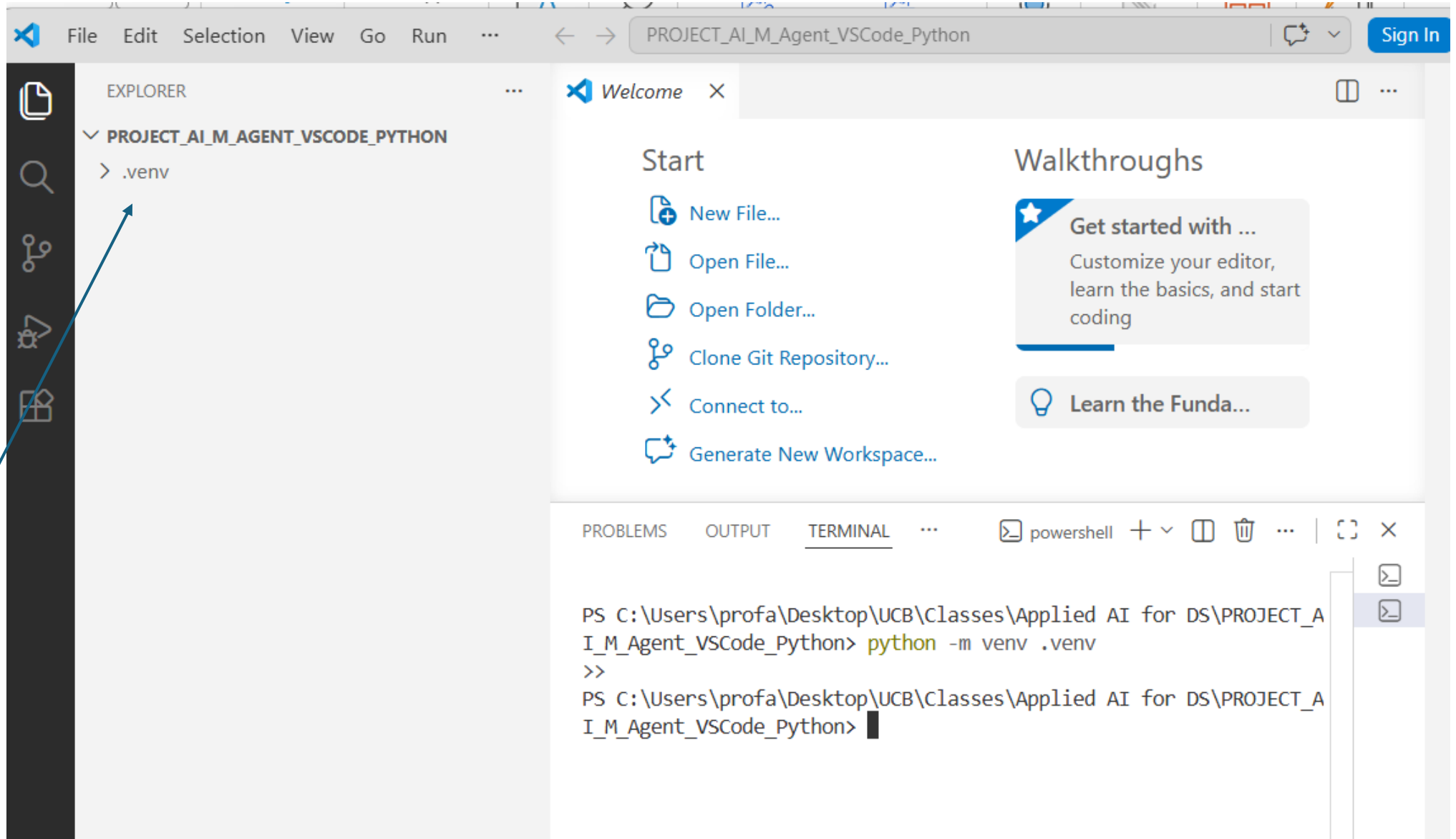
Here is where we are so far.

We installed Python and restarted the computer and VS Code

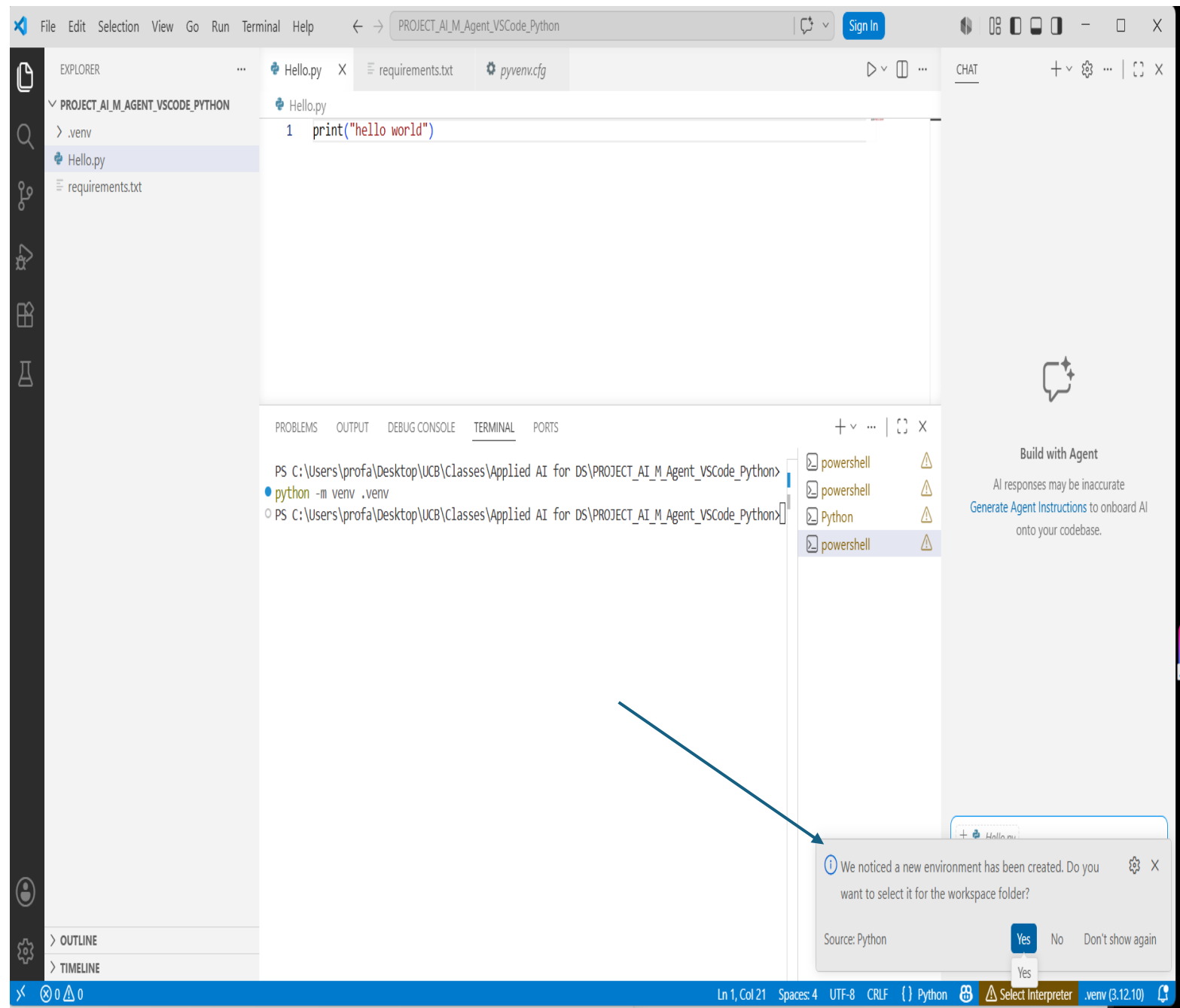
Next, we ran:

`python -m venv .venv` (as you see here)

You will know it worked if you see the `.venv` under your Project Folder name on the left.

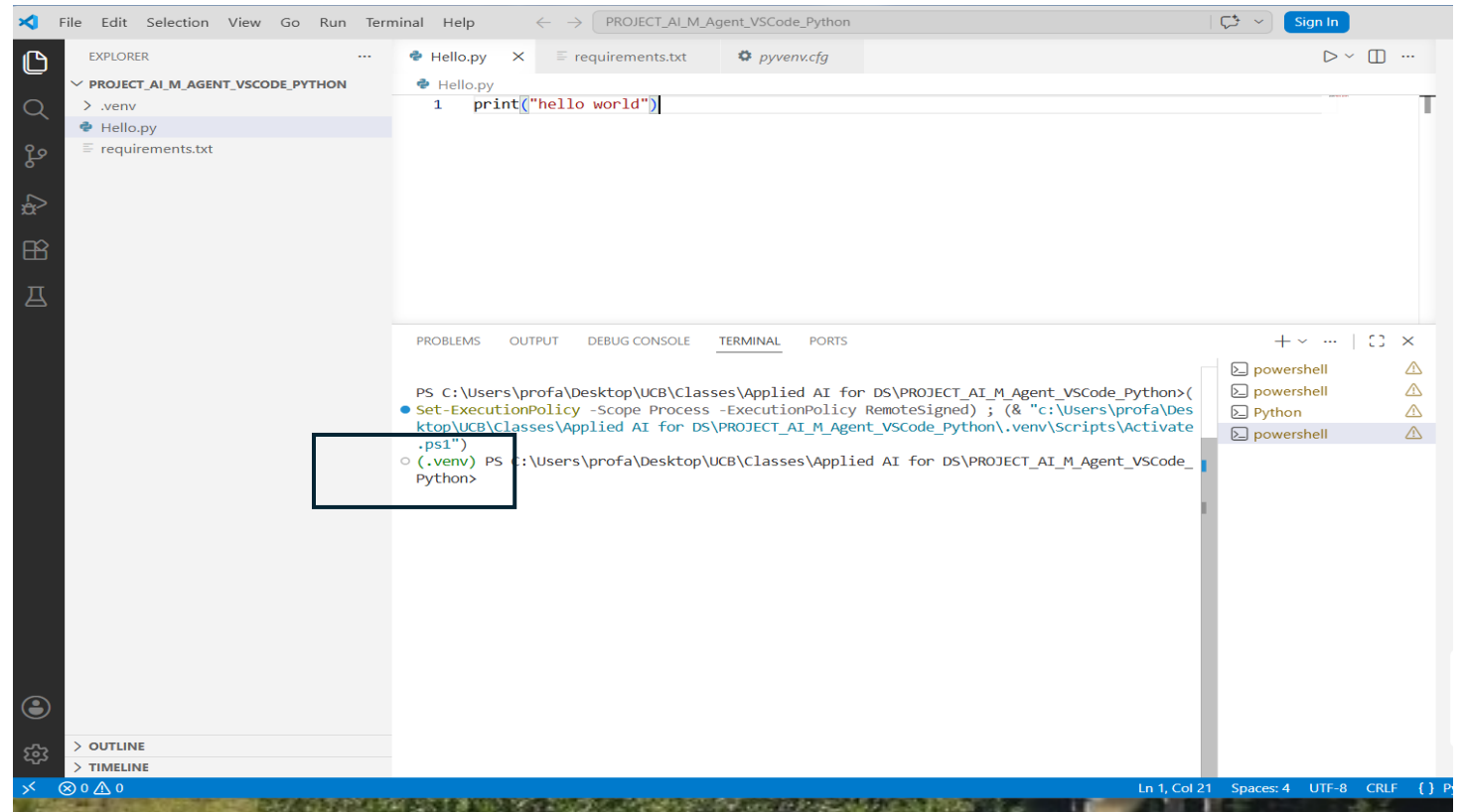


If you see
this pop up
in VS Code
choose yes



It is important to see .venv

If you do not see it (as shown in this image) close and reopen VS Code.



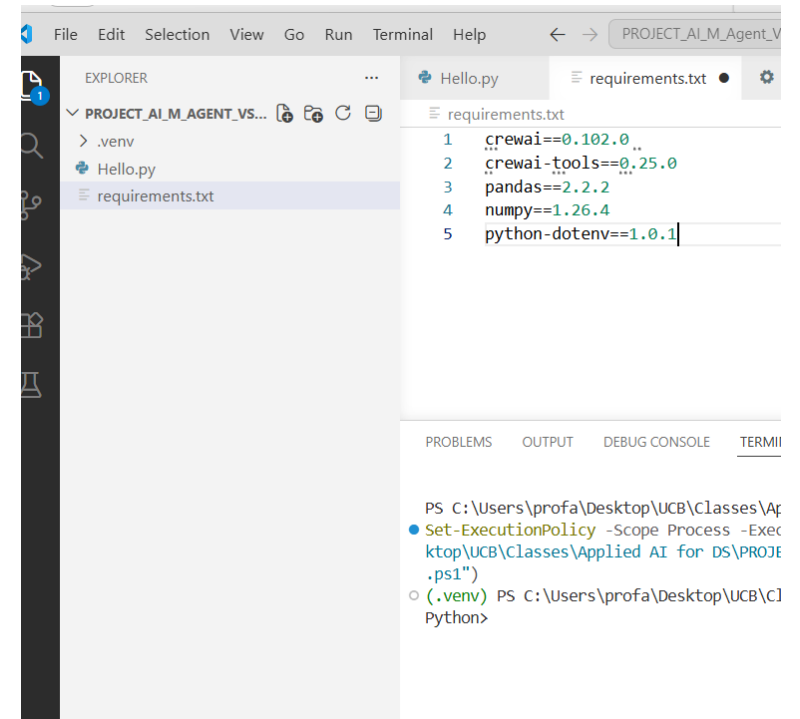
Note: The requirements.txt file

While we have not gotten to this yet, we will have a file in the Project folder called “requirements.txt”. Into that file, place the following. You can do this now or later.

crewai

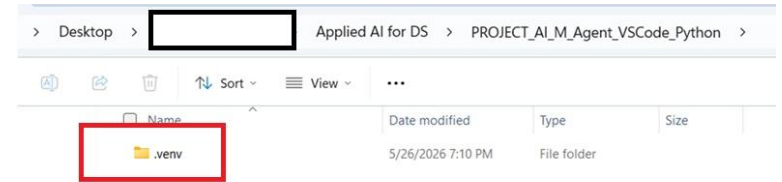
crewai[tools]

python-dotenv



Finding .venv in your Project Folder on your computer

Here, you see .venv in my Project Folder on my computer. You will see something similar on your computer. The idea here is to become familiar with where everything is and that you can see things from multiple locations.



Next – Assure VS Code Can Run Python

- 1) Link VS Code to Local Python Installation
- 2) Configure the Terminal Execution Policy (Windows Only)
- 3) Create and run Hello.py to test it.

IMPORTANT: Everyone's System is a bit different. I am sharing everything I have tested and can think of to help you to prepare a VS Code/Python environment to code AI Agents. **Use my suggestions/examples as they apply to your situation. You can always research any issues you run into.**

FYI (for some cases): Configure the Terminal Execution Policy (Windows Only) [Only in some cases]

- On a Windows machine, the built-in terminal (PowerShell) often blocks scripts from running by default. This will prevent VS Code from activating your .venv.
- Open the Start Menu on your computer, search for **PowerShell**, right-click it, and select **Run as Administrator**.
- Type the following command and press Enter:

Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

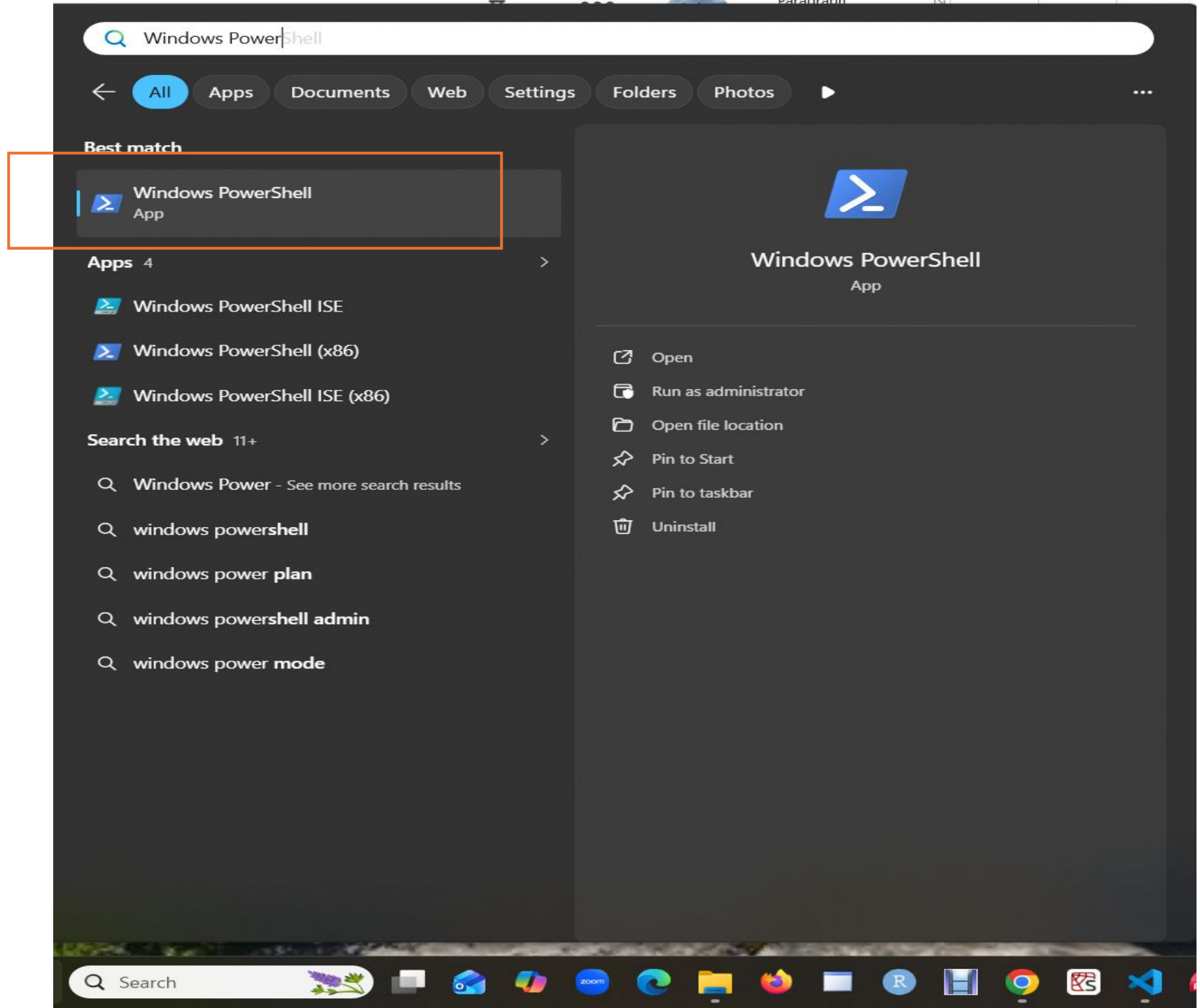
Type Y (Yes) and press Enter to confirm. You can now close PowerShell. This allows VS Code to safely run the internal script required to launch Python.

You can double-check that the setting was applied successfully by running this quick command:

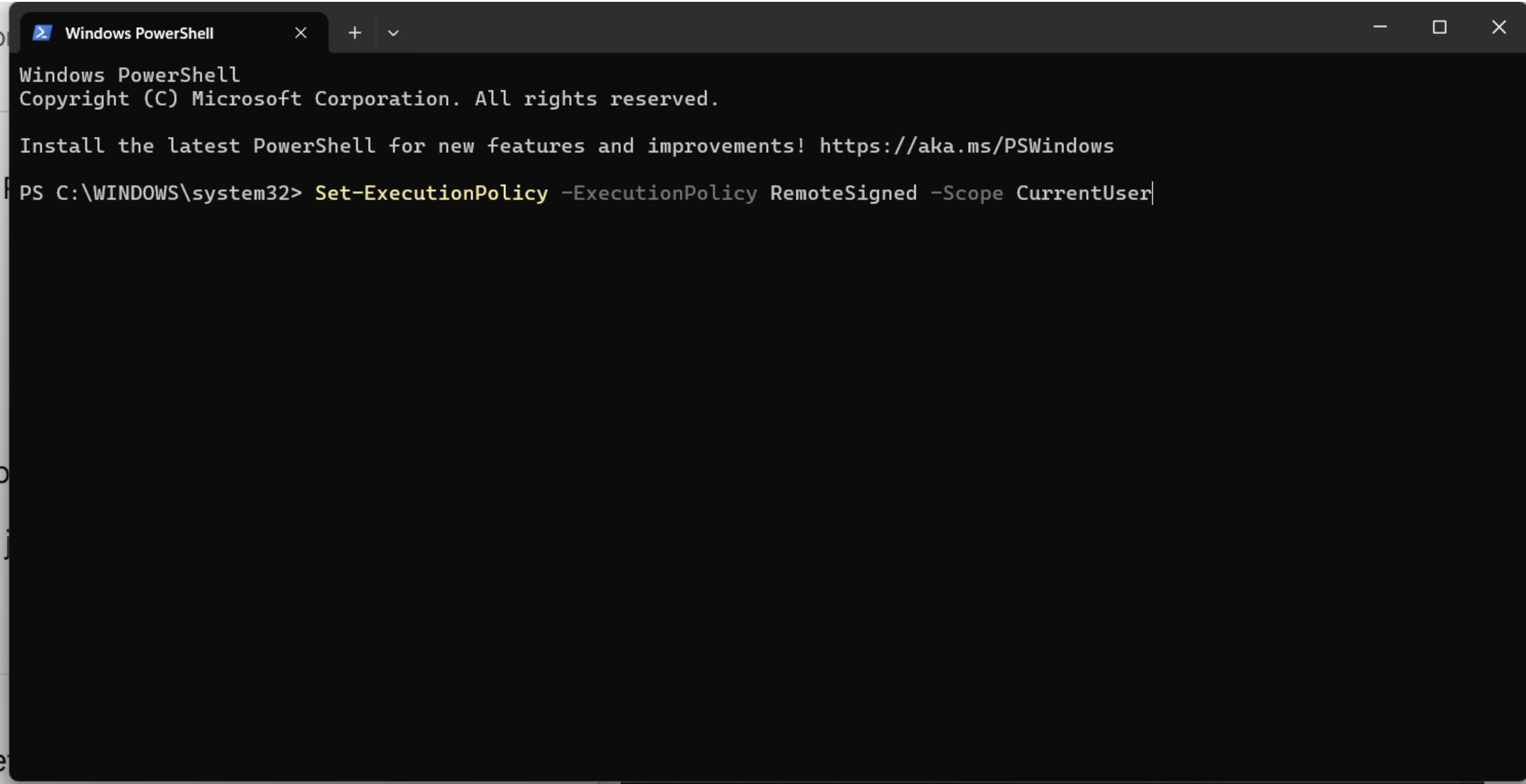
Get-ExecutionPolicy -List

Look at the row labeled CurrentUser. It should now say RemoteSigned next to it. Once you see that, you can safely close PowerShell and return to VS Code!

Use
exactly and
only
Windows
PowerShell
app



Do the following in the Power Shell (windows only users)

A screenshot of a Windows PowerShell window. The title bar shows 'Windows PowerShell' with standard window controls. The console text includes the PowerShell version, copyright notice, and a link to update PowerShell. The command prompt shows the command 'Set-ExecutionPolicy RemoteSigned' being entered at the C:\WINDOWS\system32 directory.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Running Python in VS Code

Our goal here is to enable VS Code to run Python files.

Sadly, VS Code hides language-specific commands until a file of that type is actively open.

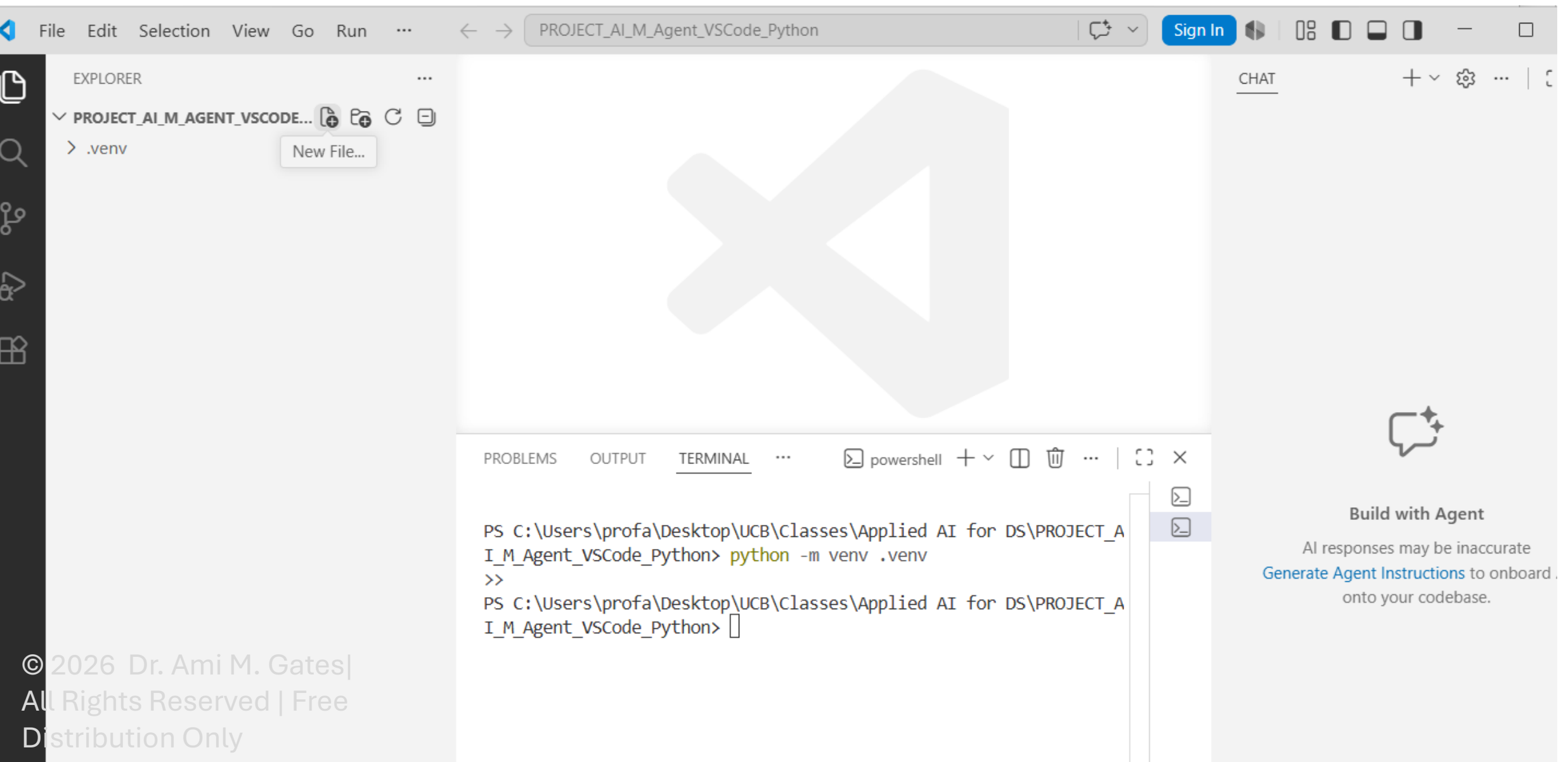
Therefore, first create a Python file called “Hello.py” and place ***print(“Hello World”)*** into the file. Save it inside your Project.

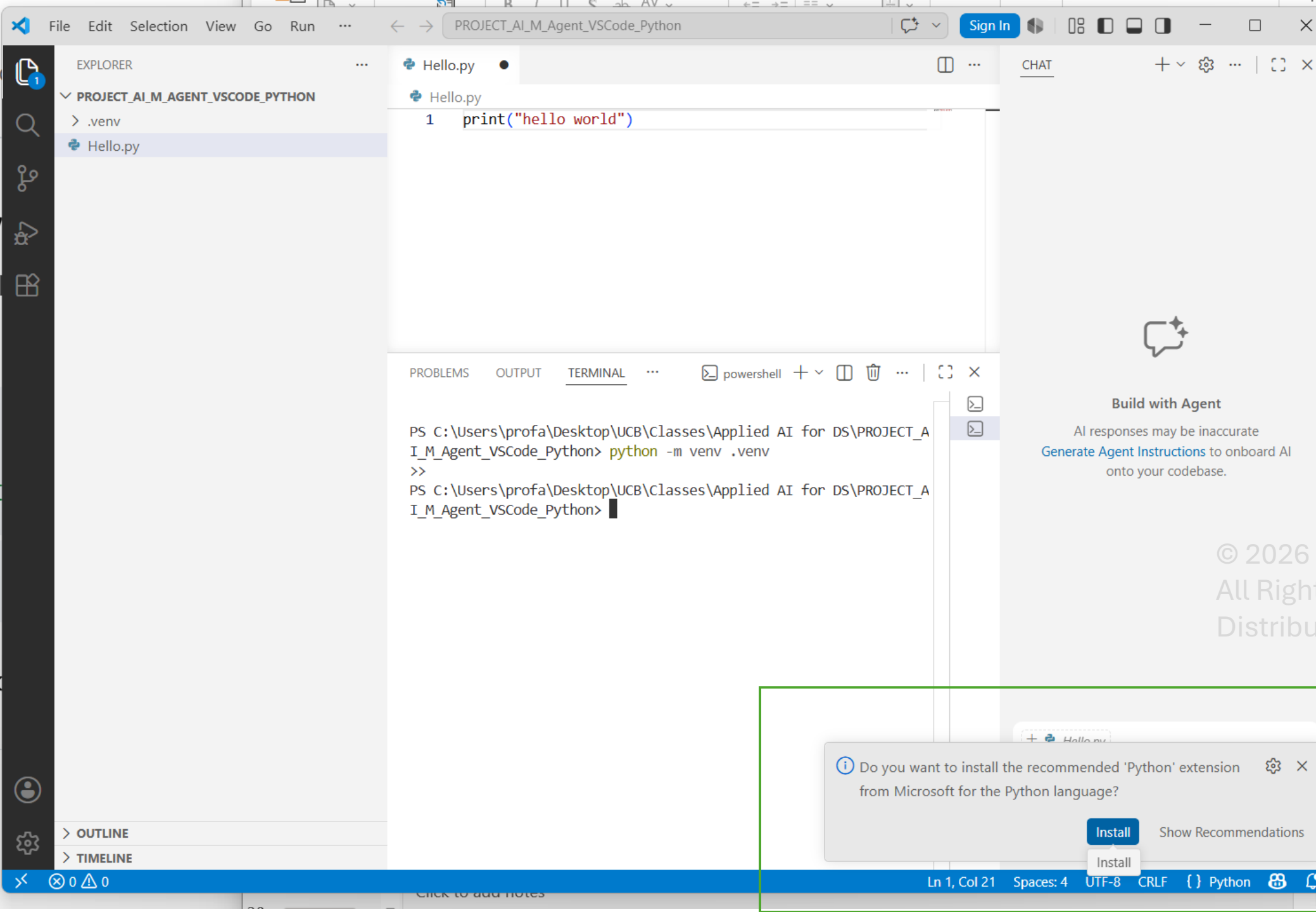
On the left sidebar, click your Hello.py file to open it in the main editor.

Once the file is open and your cursor is blinking inside the code, open the Command Palette (Ctrl+Shift+P for Windows or Cmd+Shift+P for a MAC).

Type ***Python: Select Interpreter***.

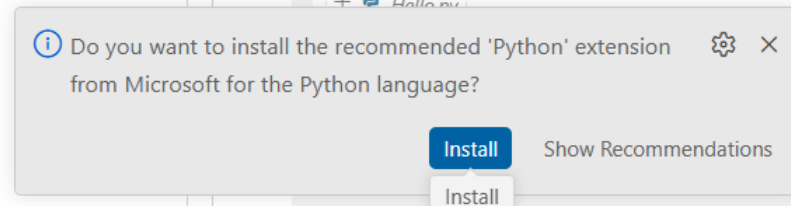
Create the Hello.py file inside of VS Code – in your Project Folder





Yes – Install
the
recommended
Python
extensions....

© 2026 Dr. Ami M. Gates |
All Rights Reserved | Free
Distribution Only



Side Notes

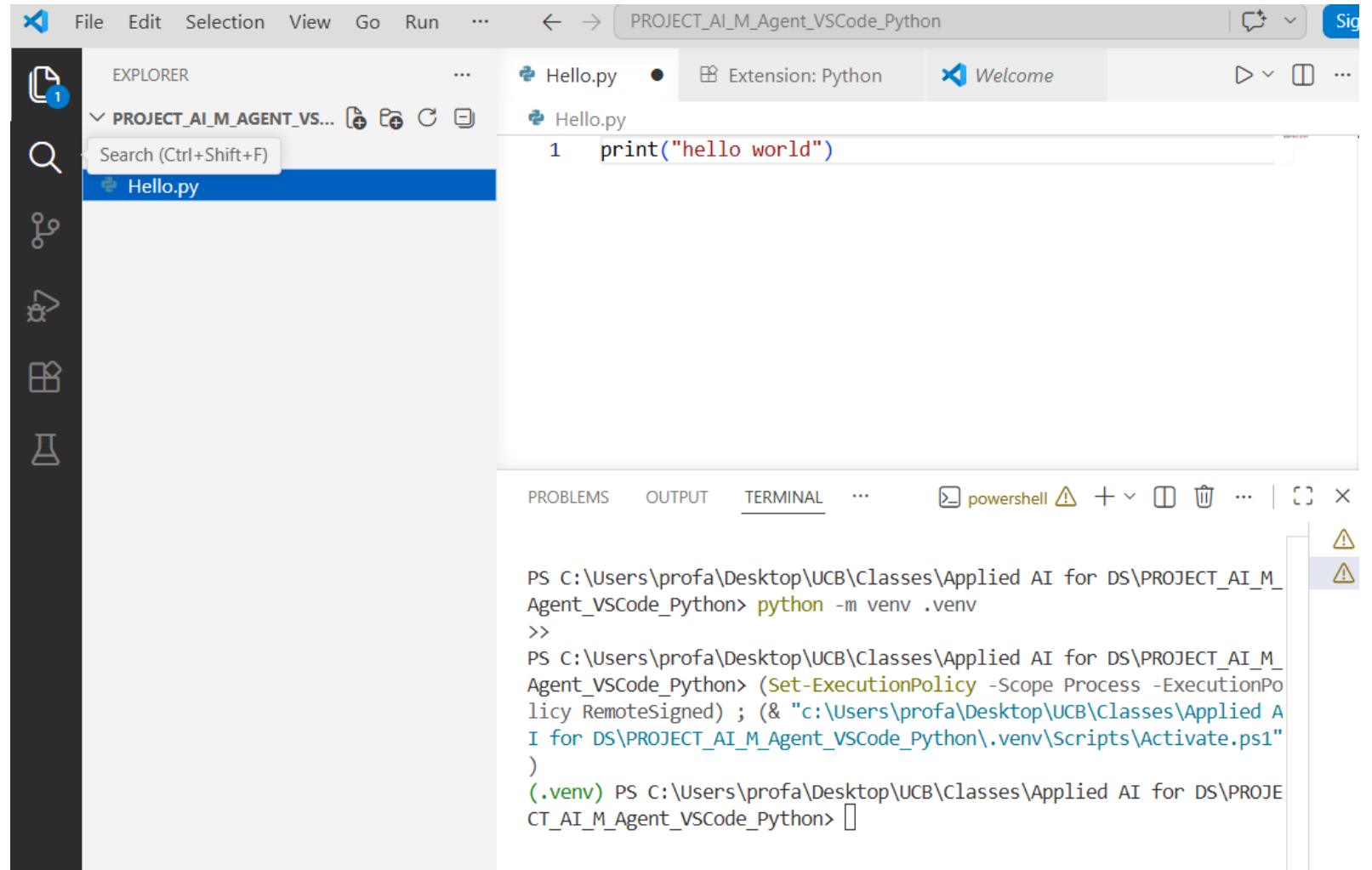
You can access the Command Palette for many things.

One such thing you can do is change the background of your VS Code. The following slides show this quickly before we continue setting up our environment.

To open the
“palette” use **ctrl
shift P** (Windows)

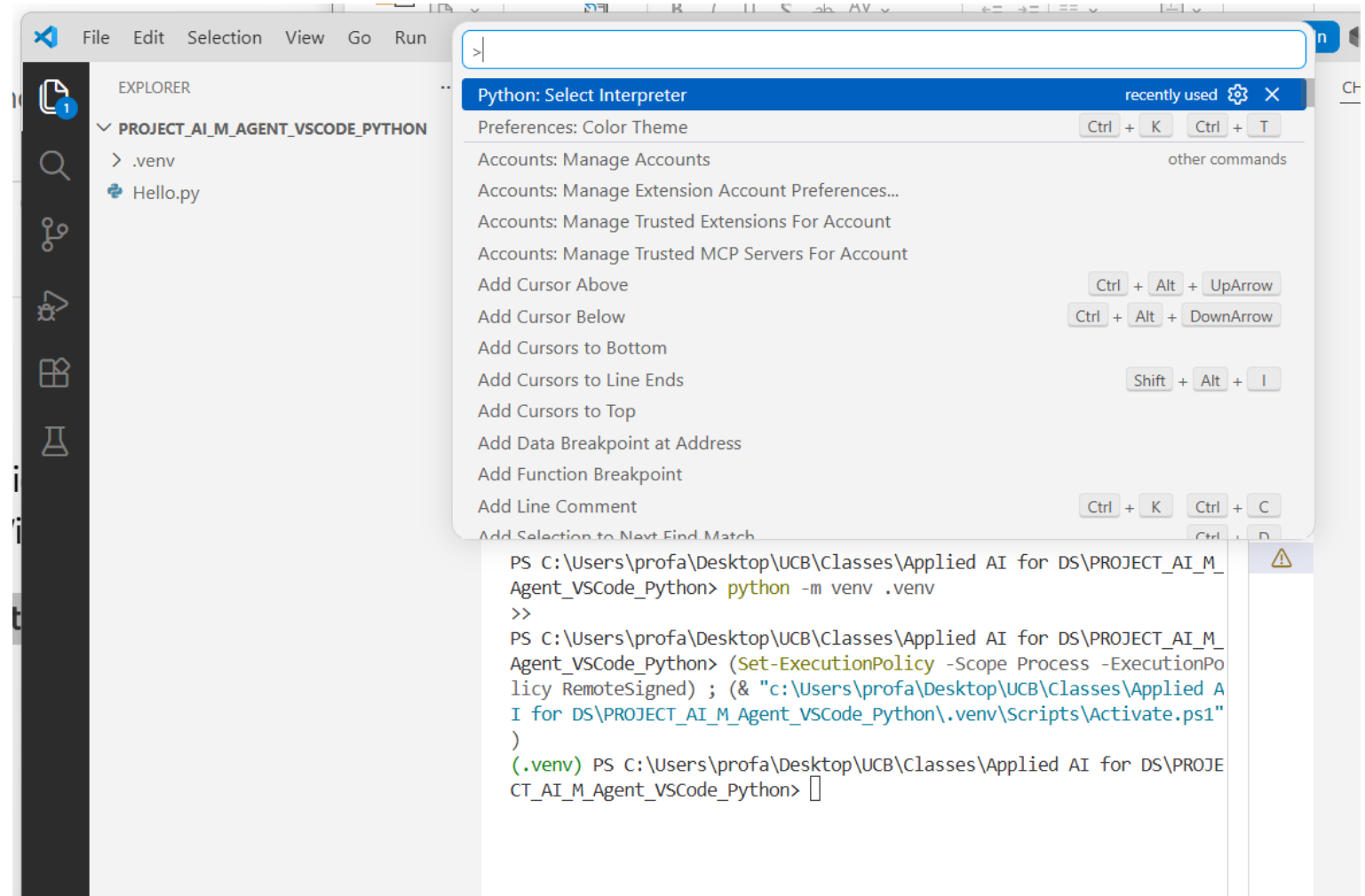
To close the
palette use “Esc”

Also, to re-see
your Project
Folder, find and
click the icon of
two overlapping
sheets of paper
(near the upper
left in VS Code).



Python: Select Interpreter

This is the palette (called the Command Palette) and here we are assuring that we can find: ***Python: Select Interpreter***



Link VS Code to Your Local Python Installation

- VS Code is just a text editor - it needs to be told exactly where your new standalone Python execution file lives.
- In VS Code, open the **Command Palette** by pressing **Ctrl+Shift+P** (Windows) or **Cmd+Shift+P** (Mac).
- Type
Python: Select Interpreter
and click on it when it appears in the dropdown menu.
- A list of Python installations found on your computer will appear. Look closely at the paths:
 - **Do not** choose any path that mentions anaconda, miniconda, or conda.
 - **Select** the path that points to your newly installed standalone Python version, or explicitly select the (.venv): venv environment you just created in your project folder.
- Once selected, look at the bottom-right corner of your VS Code window. **You should see the selected Python environment listed there as visual confirmation.**

OK – Run “Hello.py”

- Now, return to VS Code to verify everything is working perfectly.
- Click on your ***Hello.py*** file in VS Code so it is open on your screen.
- Look at the **top-right corner** of the VS Code window. You will see a small **solid play triangle button** (Run Python File).
- **Click that Play Button.**

File Edit Selection View Go Run ... PROJECT_AI_M_Agent_VSCode_Python Sign In

EXPLORER

PROJECT_AI_M_AGENT_VS...
> .venv
Hello.py

Hello.py

```
1 print("hello world")
```

PROBLEMS OUTPUT TERMINAL ... Python + -

PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python> & C:/Users/profa/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/profa/Desktop/UCB/Classes/Applied AI for DS/PROJECT_AI_M_Agent_VSCode_Python/Hello.py"

hello world

PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python\.venv\Scripts\Activate.ps1")

(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python>

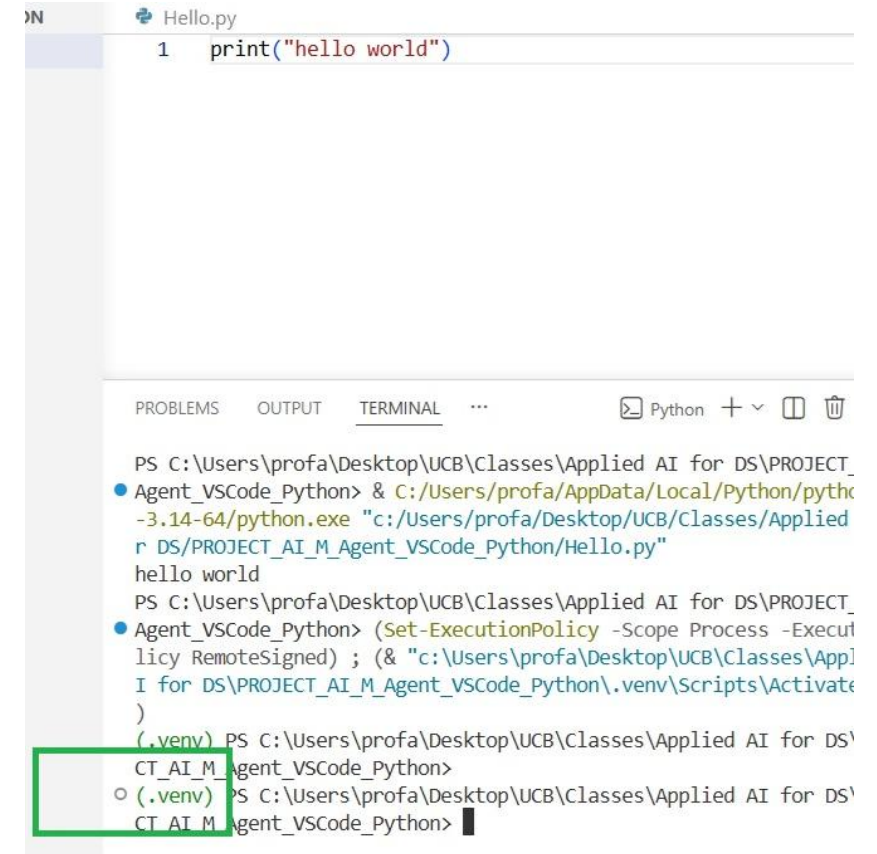
© 2026 Dr. Ami M. Gates | All Rights Reserved | Free Distribution Only

Installing Libraries to Code Agents – In Our Project Environment (only)

- 1) We are going to keep this Project (and all of its goodies) separate from other things on our computers.
- 2) First, assure you have your virtual environment folder structure set up. You will see your Project Folder, and under it, the .venv. Inside .venv you will see things like Lib, and Include, etc.
- 3) The Lib folder you see is the exact specific location where Python will automatically save all the libraries we are about to install.
- 4) You do not need to manually drag or drop any files into those folders. Python's installer (pip) handles everything for you behind the scenes, as long as the environment is active.

Confirming Your Terminal/Environment Is Ready

- Before running the installation commands, look closely at the terminal window at the bottom of your VS Code screen.
- **Look for the label:** You want to see (.venv) at the very beginning of the terminal text line.



```

Hello.py
1 print("hello world")

PROBLEMS OUTPUT TERMINAL ... Python + - 
PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python> & C:/Users/profa/AppData/Local/Python/python3.14-64/python.exe "c:/Users/profa/Desktop/UCB/Classes/Applied AI for DS/PROJECT_AI_M_Agent_VSCode_Python/Hello.py"
hello world
PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:/Users/profa/Desktop/UCB/Classes/Applied AI for DS/PROJECT_AI_M_Agent_VSCode_Python/.venv/Scripts/Activate.ps1")
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python>
o (.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python>
```


Using “requirements.txt” to Install Libraries

Create *requirements.txt* in the right place. To do this - In the **VS Code file explorer** (left side), right-click your main Project Folder and select **New File**.

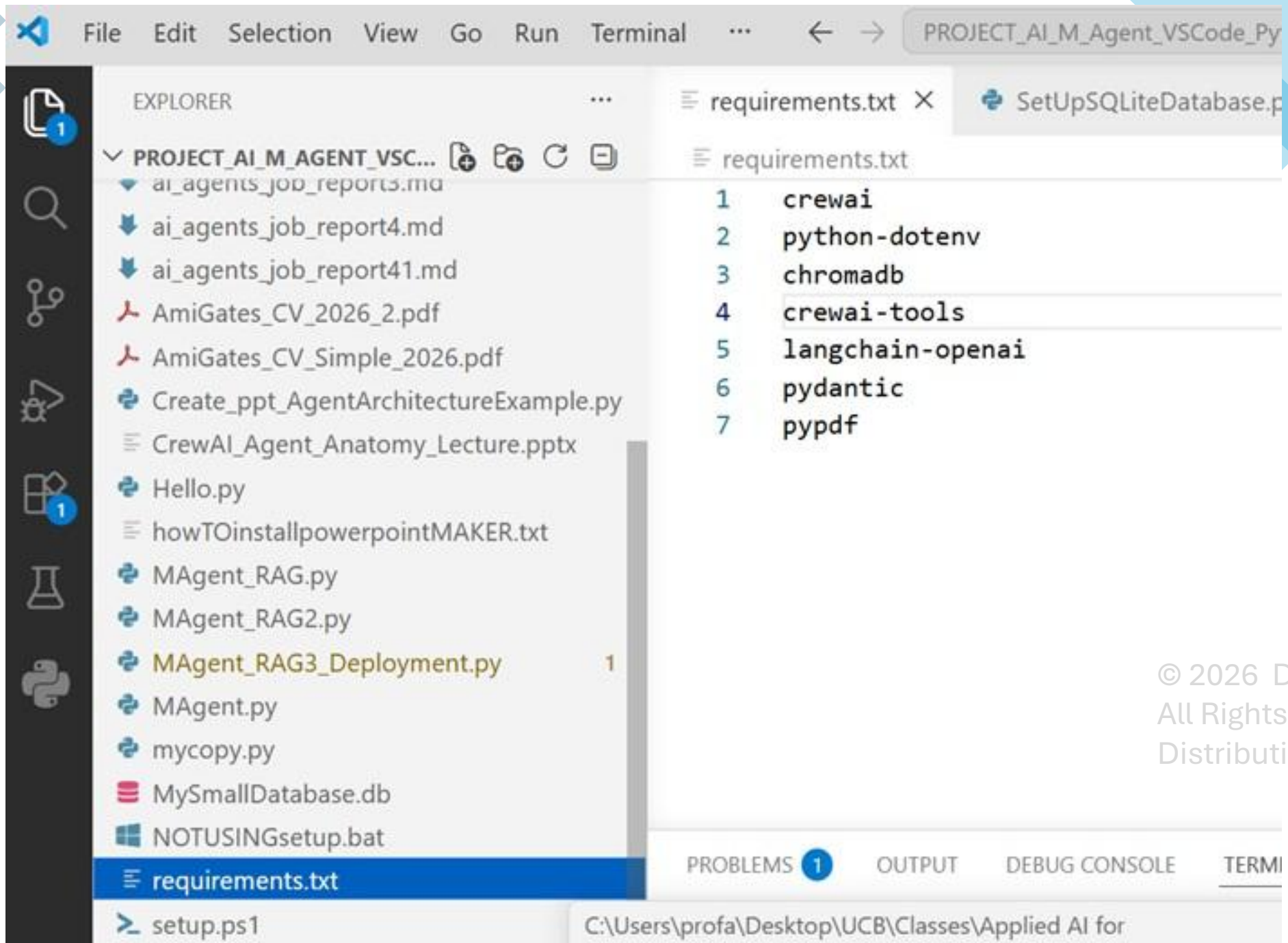
Type “requirements.txt” and press Enter.

Add the Libraries: Copy and paste library names into the new file such as the following and then save the file.

```
crewai  
crewai[tools]  
python-dotenv
```

Run the Installer: Click back down into your terminal window and type this command, then press Enter:

pip install -r requirements.txt

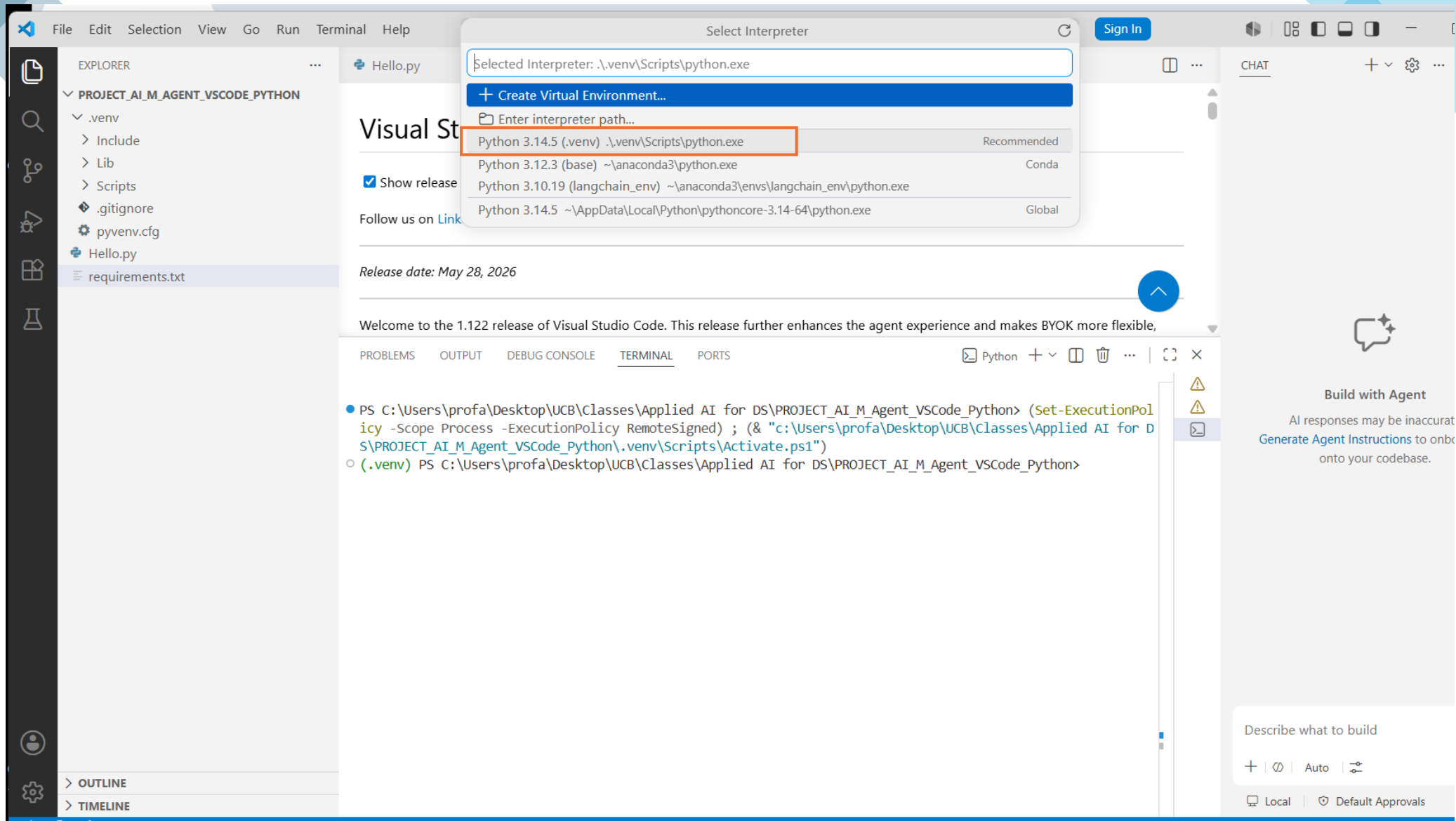


© 2026 Dr. Ami M. Gates |
All Rights Reserved | Free
Distribution Only

Now – Install...

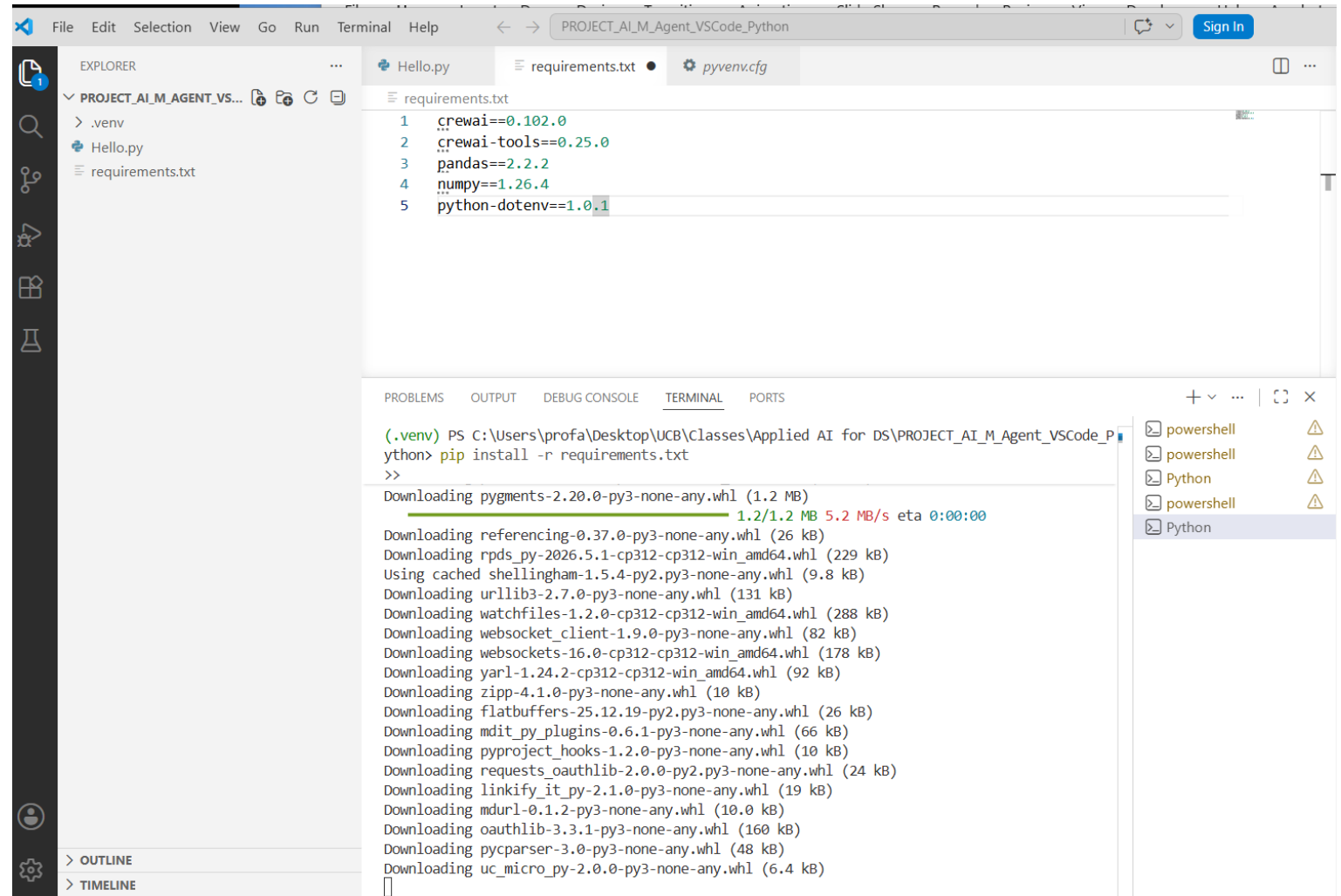
- First, close and then re-open VS Code. Yes there are other ways to do this – but this is fastest 😊 Doing this simply assures that any changes you made get included in our next steps.
- Next, assure you are in `.venv` in your Project.
 - Use the Command Pallet (ctrl shift P) and type in “Python: Select Interpreter”.
 - Then Look for the option that lists **(.venv): ./ .venv/Scripts/python.exe** (or similar path) and select it. This forces VS Code to look inside your local project environment.
- Now, click back down into your terminal window and type this command, then press Enter:

pip install -r requirements.txt



This takes a while so
wait for it to finish.

While
Installing –
you will see
something
like:



The screenshot shows the Visual Studio Code interface. The Explorer panel on the left shows the project structure with a .venv folder and files Hello.py and requirements.txt. The Editor panel shows the requirements.txt file with the following content:

```
1 crewai==0.102.0
2 crewai-tools==0.25.0
3 pandas==2.2.2
4 numpy==1.26.4
5 python-dotenv==1.0.1
```

The Terminal panel at the bottom shows the command `python> pip install -r requirements.txt` being executed. The output shows the progress of downloading various packages, with a progress bar for `pygments-2.20.0-py3-none-any.whl` (1.2 MB) at 1.2/1.2 MB, 5.2 MB/s, eta 0:00:00. Other packages being downloaded include `referencing-0.37.0-py3-none-any.whl` (26 kB), `rpds_py-2026.5.1-cp312-cp312-win_amd64.whl` (229 kB), `Using cached shellingham-1.5.4-py2.py3-none-any.whl` (9.8 kB), `urllib3-2.7.0-py3-none-any.whl` (131 kB), `watchfiles-1.2.0-cp312-cp312-win_amd64.whl` (288 kB), `websocket_client-1.9.0-py3-none-any.whl` (82 kB), `websockets-16.0-cp312-cp312-win_amd64.whl` (178 kB), `yarl-1.24.2-cp312-cp312-win_amd64.whl` (92 kB), `zipp-4.1.0-py3-none-any.whl` (10 kB), `flatbuffers-25.12.19-py2.py3-none-any.whl` (26 kB), `mdit_py_plugins-0.6.1-py3-none-any.whl` (66 kB), `pyproject_hooks-1.2.0-py3-none-any.whl` (10 kB), `requests_oauthlib-2.0.0-py2.py3-none-any.whl` (24 kB), `linkify_it_py-2.1.0-py3-none-any.whl` (19 kB), `mdurl-0.1.2-py3-none-any.whl` (10.0 kB), `oauthlib-3.3.1-py3-none-any.whl` (160 kB), `pycparser-3.0-py3-none-any.whl` (48 kB), and `uc_micro_py-2.0.0-py3-none-any.whl` (6.4 kB).

Errors, Why, and Fixes

Each person has different things on their computers. For example, many folks have the full VS and others have C++, etc. However, if you do not (which is fine), you will get errors when you try to pip install.

Here is how to fix them:

- **The WHY:** Some errors can happen because pip is attempting to download the raw source code - such as for numpy and build (compile) it from scratch on your computer. Since you don't have a C++ compiler (gcc, clang, or Visual Studio build tools) installed on Windows, it crashes.
- Instead of compiling it ourselves, we need to force pip to download the **pre-compiled binary version** (called a "wheel") that matches your Windows system. If you have a MAC, you may need to research this if you get errors. Other also choose to install the entire Visual Studio. I will not use all of VS and will continue to teach and to use VS Code.

NOTE: Learning these types of errors and fixes is part of being a programmer and is very important!

- Here is how to bypass this compiler issue and get everything installed cleanly:

Step 1: Clear the Broken Setup

- Your environment is likely holding onto a half-downloaded, broken build. Upgrade the basic installer tools first. Run this in the VS Code terminal:
- `.venv\Scripts\python.exe -m pip install --upgrade pip setuptools wheel`

Errors and Fixes Cont.

Step 2: Force the Pre-Compiled Binary Installation

- Now, use the `--only-binary` flag. This tells Python to completely skip looking for compilers and only use ready-to-run Windows packages:
- Run in VS Code Terminal:
- `.venv\Scripts\python.exe -m pip install --only-binary :all: -r requirements.txt`

Step 3: Verify It Worked

- If the text scrolls through successfully without errors, type this command to make sure numpy and pandas are safe inside your `.venv`:
- Type in VS Code terminal:
- `.venv\Scripts\python.exe -m pip list`

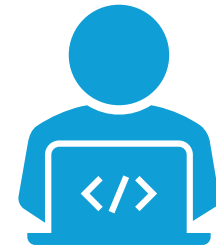
Errors/Fixes



If you run into issues/errors, that is normal and a critical part of learning to set up a new coding environment.



Please use the web including AI to help you to resolve issues.



Everyone's computer system is a bit different.

Connecting to LLMs

The next many slides will go through the steps of being able to use Python, VS Code, and an API to access an LLM so that the Agent we want to code can “think” and communicate.

Step 1: Choose an LLM

There are actually several LLMs one can use. Most are not free and you will need to give them a credit card.

To establish a connection with an LLM. we will create an account with an LLM provider, obtain a secure password called an **API Key**, and save it in a hidden configuration file named ***.env***.

Here we go...

OpenAI is often recommended for beginners and also works well with Crew.

Choose an LLM Provider

Provider	Recommended Model	Best For	Website
 OpenAI	gpt-4o-mini (<i>Cheap/Fast</i>) gpt-4o (<i>Powerful</i>)	Beginners, easiest setup, industry standard.	platform.openai.com
	claude-3-5-sonnet-latest	Advanced coding tasks, deep writing nuances.	console.anthropic.com
	gemini-1.5-flash	Large context windows, cost-efficiency.	aistudio.google.com

**Note: This table summarizes current LLM options for AI Agent development.*

OpenAI

<https://platform.openai.com/login>

Log in/Sign Up

(I log in with my Google account but you can choose any method you like)

- 1) Look at the left-hand navigation sidebar.
- 2) Click on **Settings** (represented by a gear icon ).
- 3) Select **Billing** from the submenu.
- 4) Click **Add funds** (or **Add payment method**). Deposit a minimum balance (such as **\$5.00** or **\$10.00**)

Next, Navigate to **API Keys** (represented by a key icon )

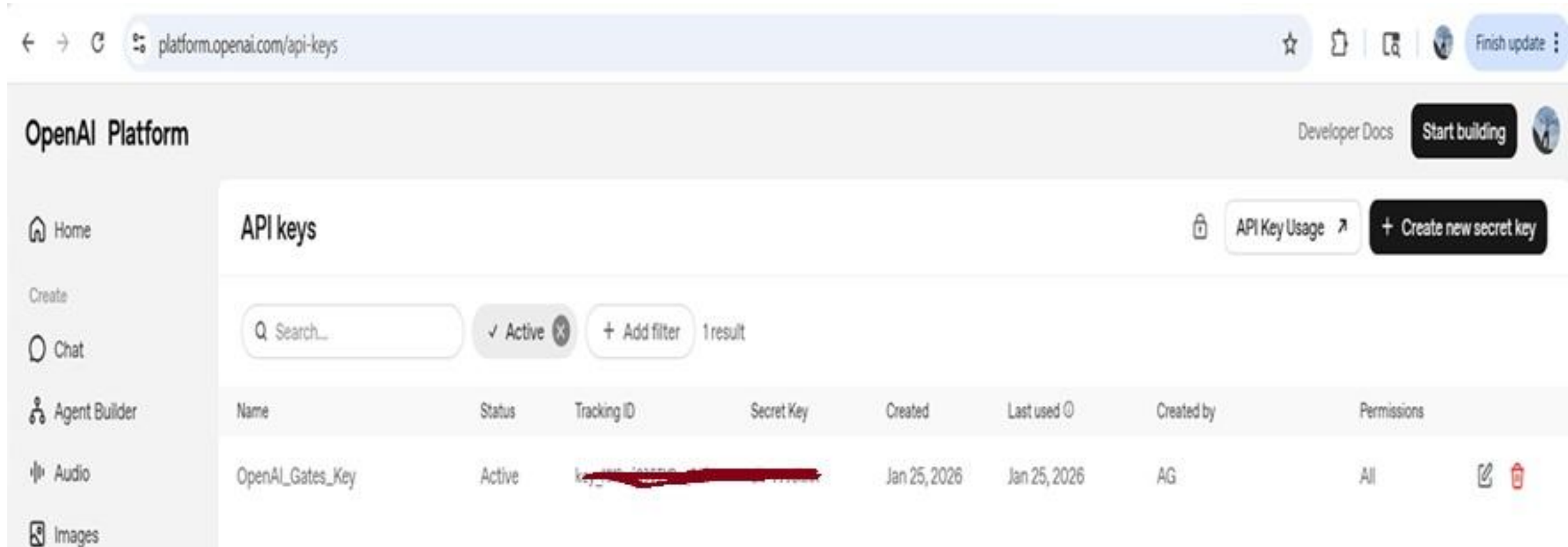
OpenAI Notes

Create the Key

- Click the green button that says **+ Create new secret key**.
- A popup window will appear. In the **Name** field, type a label so you remember what it is for (e.g., VS_Code_CrewAI_Project).
- Leave the ownership and permissions set to their default settings.
- Click the **Create secret key** button at the bottom of the popup.

Copy and Save Your Key

- A new window will show a long string of text starting with **sk-proj-....**
- Click the green **Copy** button next to the key.
- **Critical Detail:** Paste this key into a temporary text file (like Notepad or TextEdit) immediately.
- Click **Done** to close the window.
-  **Critical Security Warning:** You will only see this key **one time**. Once you click "Done", you can never view the text of this key again. If you lose it, you will have to delete it and create a brand new one. Never share this key or push it to public code repositories like GitHub.



Notes About Costs

- **Budget Models: OpenAI vs. Claude**

- Every major AI company provides a "Flagship" model (highly intelligent but expensive) and a "Lightweight/Budget" model (extremely fast and cheap).

- **OpenAI's Budget Model:** gpt-4o-mini

- **Claude's Budget Model:** claude-3-5-haiku [1]

- **The Price Comparison**

- API pricing is measured per **1 Million tokens** (roughly 750,000 words).

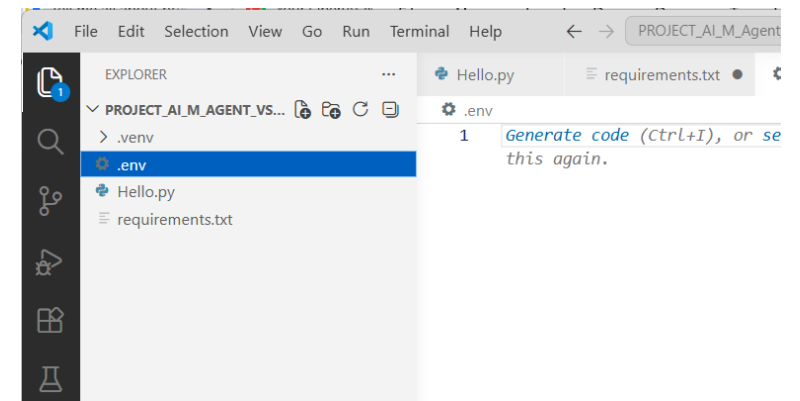
Provider & Model	Cost per 1 Million Input Tokens (Read)	Cost per 1 Million Output Tokens (Write)
BUDGET / LIGHTWEIGHT MODELS		
OpenAI ('gpt-4o-mini')	\$0.15	\$0.60
Claude ('claude-3-5-haiku')	\$0.80	\$4.00
FLAGSHIP / ADVANCED MODELS		
OpenAI ('gpt-4o')	\$2.50	\$10.00
Claude ('claude-3-5-sonnet-latest')	\$3.00	\$15.00

Create the .env file and Place the API Key into it

Step 1: Create a .env File in VS Code

- Open **VS Code**.
- Make sure your main project folder is open in the left-hand Explorer side panel.
- Hover your mouse over the empty space in the Explorer panel (or over your project folder name) and **right-click**.
- Select **New File**.
- Type the name exactly as written below:

.env



Add your Key to the .env file EXACTLY as shown

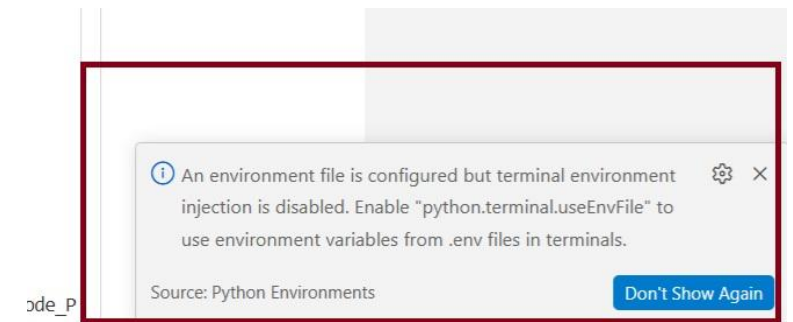
- Click inside the blank .env file editor.
- Type or paste your configuration line exactly like this:

***OPENAI_API_KEY=sk-proj-
yourActualSecretKeyPastedHere***

- It must say OPENAI_API_KEY as shown here
- There must be an = sign
- There are no quotes around the actual key
- **No spaces** before or after the = equals sign
- The entire key must stay on **one single line**

Note About Possible Pop Up Warning

- This pop-up message is a standard warning from VS Code's Python extension.
- It is notifying you that while you have successfully created a .env file, VS Code will not automatically inject those variables directly into your raw terminal window when you open it.
- You can safely **ignore this** warning or click **Close**, because our Python code handles loading this file manually anyway.



Use this
image to
write a quick
script to test
your Key

The screenshot shows the Visual Studio Code interface. In the Explorer sidebar, the file `test_key.py` is selected. The editor displays the following Python code in `test_key.py`:

```
1 import os
2 from dotenv import load_dotenv
3
4 load_dotenv()
5 print("Your saved key starts with:", os.environ.get("OPENAI_API_KEY")[:7])
```

The terminal at the bottom shows the execution of the script. It first runs `pip install --upgrade pip`, which successfully upgrades pip from 25.0.1 to 26.1.1. Then, it runs `python test_key.py`, which outputs: `Your saved key starts with: sk-proj`.

```
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python>
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python> python -m pip install --upgrade pip
Requirement already satisfied: pip in c:\users\profa\desktop\ucb\classes\applied ai for ds\project_ai_m_agent_vocode_python\.venv\lib\site-packages (25.0.1)
WARNING: Cache entry deserialization failed, entry ignored
Collecting pip
  Downloading pip-26.1.1-py3-none-any.whl.metadata (4.6 kB)
  Downloading pip-26.1.1-py3-none-any.whl (1.8 MB)
  1.8/1.8 MB 7.6 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.0.1
    Uninstalling pip-25.0.1:
      Successfully uninstalled pip-25.0.1
  Successfully installed pip-26.1.1
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python> python test_key.py
Your saved key starts with: sk-proj
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python>
```

Creating the **Multi-Agent System (MAS)** or an **Agentic Workflow**

Gates

© 2026 Dr. Ami M. Gates|
All Rights Reserved | Free
Distribution Only

Crew AI

- 1) What is CrewAI?
- 2) Why Use CrewAI? (What it Does)
- 3) What CrewAI Does NOT Do
- 4) What are Similar Options (to crew) in the Ecosystem?
- 5) Why Choose CrewAI Over the Alternatives?
- 6) Is CrewAI Most Used in Industry?

What is CrewAI?

- CrewAI is an open-source Python framework used to orchestrate **Role-Based Multi-Agent Systems**.
- Instead of treating an LLM as a single chatbot that answers questions, CrewAI breaks a complex engineering problem into **distinct professional roles**.
- It mimics a real-world corporate structure, assigning specific personalities, goals, and rules to individual AI agents that work together to accomplish a unified mission.

Why Use CrewAI? (What it Does)

Multi-agent orchestration frameworks solve the "context window" and "reasoning split" limitations of standard LLMs.

CrewAI excels at:

- **Role Isolation:** It forces the LLM to think purely from a specific persona's perspective (e.g., a Quality Assurance engineer vs. a Product Manager), which drastically reduces hallucinations.
- **Autonomous Decision Making:** Agents dynamically decide *which* tool to use and *when* their work meets the required standard.
- **Deterministic Workflows:** It allows developers to enforce structured collaboration methods, such as sequential task handoffs, hierarchical management, or parallel processing.
- **State Management:** It natively handles the complex background memory, system prompts, and history tracking needed as agents pass data back and forth.

What CrewAI Does NOT Do

Students frequently mistake orchestration frameworks for all-in-one AI systems.

CrewAI has strict architectural limitations:

- **It is not an LLM:** CrewAI does not train models or compute text; it is a structural coordinator that sends instructions to models like OpenAI, Anthropic, or local models via Ollama.
- **No Built-In GUI:** It operates as a code-first framework. Any visual frontend or user dashboard must be built separately using tools like Streamlit, Chainlit, or Next.js.
- **Not Ideal for Single Chatbots:** If a project only requires a direct, single-turn question-and-answer interface, CrewAI introduces unnecessary system latency and API computing costs.

What are Similar Options in the Ecosystem?

- The multi-agent ecosystem contains three other primary enterprise and research frameworks:
- **LangGraph (by LangChain):** A low-level framework that models agent networks as cyclical graphs (nodes and edges).
- **Microsoft AutoGen:** An event-driven framework where agents communicate via a flexible conversation-centric paradigm.
- **LlamaIndex Workflows:** An event-driven orchestration system built explicitly around advanced data retrieval (RAG) pipelines.

Why Choose CrewAI Over the Alternatives?

- For graduate-level coursework and rapid enterprise deployment, CrewAI offers specific advantages:
- **Human-Centric Abstraction:** Microsoft AutoGen and LangGraph require developers to manually program state transitions, network loops, and message routing. CrewAI abstracts this into clean, corporate analogies: **Agents, Tasks, and Crews**. This makes code highly readable and structurally intuitive.
- **Production Readiness:** CrewAI has built-in enterprise features like memory systems (short-term, long-term, and entity memory), rate-limiting safeguards to prevent infinite API cost loops, and task delegation controls out of the box.
- **Built-in Tool Ecosystem:** It integrates seamlessly with the crewai-tools library, giving agents immediate access to web scrapers, PDF readers, directory crawlers, and database connectors with single-line implementations.

Is CrewAI Most Used in Industry?

- Yes, CrewAI is currently one of the dominant market leaders for commercial multi-agent application development, sharing the top tier with LangGraph.
- While LangGraph is heavily favored by deep infrastructure engineers who require absolute, low-level control over every microscopic token path, **CrewAI is the industry favorite for rapid prototyping, operational workflows, and enterprise automation.**
- Companies use CrewAI to build internal research systems, automated software engineering pipelines, compliance checking workflows, and marketing content generators because it cuts development time from weeks to hours.

Memory, Management, and CrewAI

Gates

© 2026 Dr. Ami M. Gates|
All Rights Reserved | Free
Distribution Only

Memory Matters

- It is critical to thoroughly grasp how memory functions in an engineering environment in order to understand that memory is not just a passive storage locker.
- In CrewAI for example, memory is a **highly active software layer** that sits directly between the agent's core code and the OpenAI API.
- Here is the deep-dive explanation of how memory is structured, how it operates under the hood, and its explicit architectural connection to an agent.

Topics:

Why Memory is Important: The Engineering Problem

How Memory is Used: The Vector Database Mechanics

How Memory is Explicitly Connected to an Agent

Why Memory is Important: The Engineering Problem

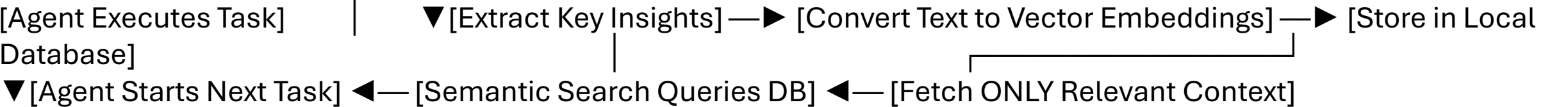
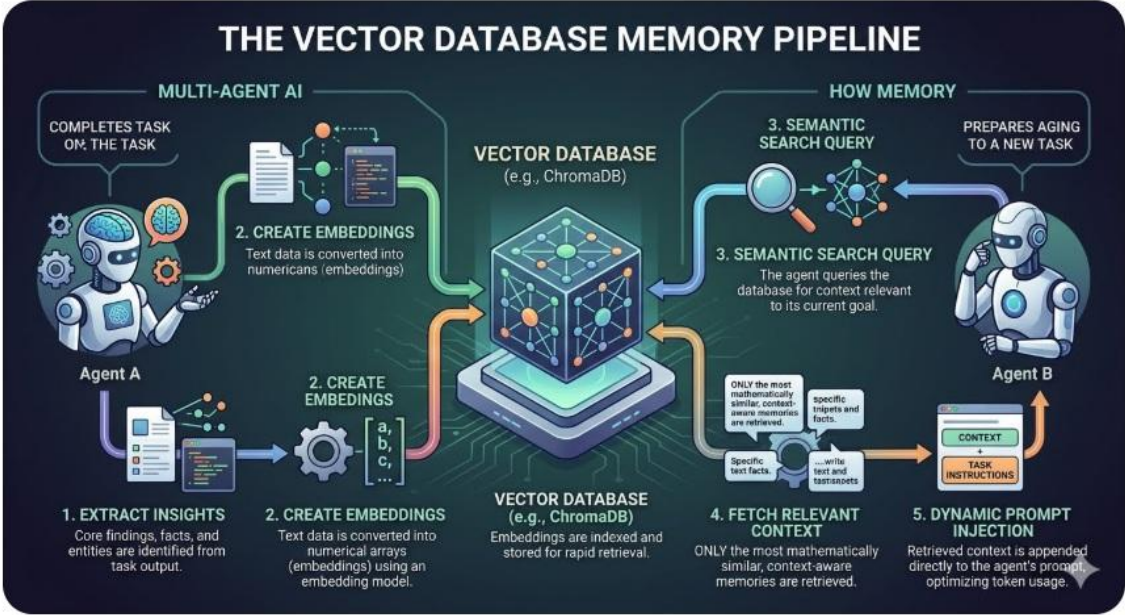
- When coding a multi-agent system, your biggest adversaries are **token limits**, **API latency**, and **hallucinations**.
- If you do not use a memory system, every single agent interaction requires sending the entire conversation history back and forth to OpenAI. This creates a massive compounding problem:
- **Context Exhaustion:** The prompts grow so massive that they eventually exceed the model's context window.
- **Context Dilution:** The more data you stuff into a standard prompt, the less accurately the LLM follows specific instructions.
- **Financial Waste:** You are continuously paying for the exact same input tokens over and over again across different tasks.
- Memory systems fix this by converting a sprawling, unstructured conversation history into a clean, optimized database index that agents can selectively query.

How Memory is Used: The Vector Database Mechanics

When you set *memory=True* in CrewAI, the framework does not just store plain text strings in RAM.

It initializes a localized **Vector Database** (specifically using an embedded library called ChromaDB) directly within your project directory.

Image generated by AI



THE VECTOR DATABASE MEMORY PIPELINE

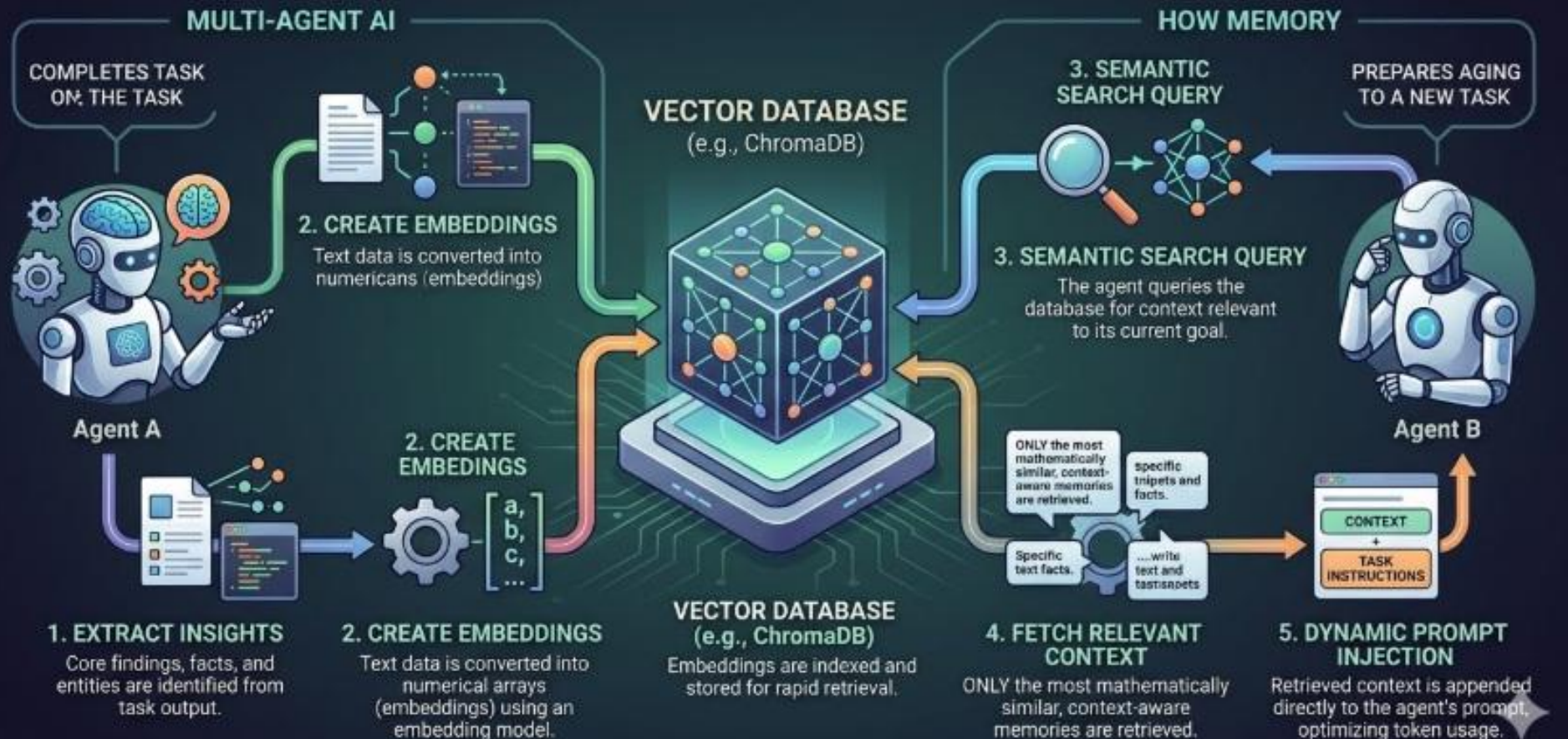


Image generated by AI

The image to the left illustrates an “instructional infographic diagram” that maps out exactly how a **Contextual Prompt Injection** is structured inside an active multi-agent system framework like CrewAI.

This diagram breaks down the specific anatomy of the final prompt payload into its four core layers before it hits the LLM API, ensuring your graduate students can visually trace how background configurations, transient execution memory, permanent historical metadata, and granular task descriptions converge.

How Memory is Explicitly Connected to an Agent

- In your code, the connection between an agent and the memory system is managed automatically by the **Crew Orchestrator**, but you can also configure individual agent behaviors.
- Here is exactly how they interlock architecturally:

The Contextual Prompt injection

Every time an agent prepares to execute a task, CrewAI compiles a hidden, master system prompt. Because memory is enabled, the orchestrator injects a specific block into that prompt (see the next slide)

Example

[SYSTEM PROMPT]

You are a Senior Research Analyst...

Your goal is to...

[CONTEXT FROM SHORT-TERM MEMORY]

The Lead Technical Writer previously established this format constraint: ...

[CONTEXT FROM ENTITY MEMORY]

Here are verified facts regarding the technology discussed: ...

[CURRENT TASK INSTRUCTIONS]

Conduct a thorough analysis of...

The Code Implementation Setup

How the configuration bridges from the macro level (The Crew) down to the micro level (The Agent).

```
from crewai import Agent, Crew, Process
#####Individual Agent Configuration

researcher = Agent(
    role="Senior Research Analyst",
    goal="Uncover cutting-edge developments in AI",
    backstory="Expert research analyst...",
    verbose=True,

#Individual agents can be explicitly blocked from or
#allowed to use memory
    allow_delegation=True )

#####The Orchestration Connection
#This is where the actual memory database #infrastructure is
#spun up and hooked into the agents

ai_crew = Crew(
    agents=[researcher],
    tasks=[...],
    process=Process.sequential,
    memory=True
# MANDATORY: This turns on the vector database that all #
#agents above will share )
```

Execution Isolation

- Because the memory system handles routing under the hood, it ensures **role isolation**.
- The researcher agent's memory database holds raw data, sources, and findings, while the writer agent's memory database focuses on stylistic rules, formatting preferences, and communication notes.
- They only share what is explicitly required by the sequential task handoff.

THE ANATOMY OF A CREWAI AGENT OBJECT

Gates (and AI and Python and VS Code)

© 2026 Dr. Ami M. Gates|
All Rights Reserved | Free
Distribution Only

AGENT OBJECT STRUCTURAL MAP

The five explicit operational components configured inside the Python instantiation wrapper

1. THE COGNITIVE BRAIN

Code: `llm=ollm`

Handles core token computation, text pattern analysis, semantic evaluations, and logical response parsing.

Under the Hood:
`openai`, `litellm`

2. THE PERSONA ENGINE

Code: `role=`, `goal=`,
`backstory=`

Compresses general knowledge boundaries into clear professional contexts using automated system prompt wrappers.

Under the Hood:
`crewai` core

3. CONTEXTUAL MEMORY

Code: `memory=True`

Indexes localized execution context layers, runtime task handoffs, and learned entity facts.

Under the Hood:
`chromadb`, `pydantic`

4. REAL-WORLD HANDS

Code: `tools=[...]`

Gives the agent API access hooks to scrape websites, query vector systems, read directories, or write code files.

Under the Hood:
`crewai-tools`

5. REFLECTOR LOOP

Code: `verbose=True`

Governs the ReAct evaluation iteration step before outputting a validated solution block to the user.

Under the Hood:
`Python re / json`

THE DYNAMIC PAYLOAD ASSEMBLY

How the framework translates component variables into a singular prompt sent to the LLM API

[PERSONA BLOCK]	→	Injects role, goal, and backstory constraints directly to skew the base alignment model.
[MEMORY RETRIEVAL] vector database lookups.	→	Stitches relevant semantic historical context fragments recovered from local
[TOOL REGISTRY] like file system readers.	→	Exposes the strict text structural arguments required to call interface hooks
[CURRENT TASK BOUNDS] expected data schemas.	→	Appends isolated operational goal scopes, specific criteria filters, and
[REACT INTERACTION TRIGGER] ('Thought:', 'Action:')	→	Forces the cognitive runtime loop engine to open its internal evaluation step

Building Your First Agent

- Let's transition directly from theory into live execution.
- To understand how the “Brain” (the LLM), the “Persona” (in this case the CrewAI Config), the “Memory” (here we use ChromaDB), the “Hands” (also known as Agent Tools), and the “Reflector” (ReAct Loop) interact, we are going to build and run an Agent in Python in our Project.
- We will explicitly equip the “**Senior Research Analyst**” (**of our Project**) with its "Hands"—a native CrewAI Web Search tool.
- This forces the agent into a dynamic loop: it will have to read the task, form a search query, evaluate the raw search results, store findings in memory, and pass a refined payload to the writer.

Further Environment SetUp

- Because we are introducing **live web browsing**, we need to provide a specialized **search tool api key** alongside our OpenAI key.
- For this extra setup, we will use **Serper**, which is the industry standard for programmatic Google Search integration.
- Go to ***serper.dev*** and sign up for a free account (it takes 30 seconds and gives you 2,500 free searches).
- Open your hidden **.env** file in VS Code and append the new key right beneath your OpenAI key.

Ensure it looks exactly like this:

```
OPENAI_API_KEY=sk-proj-YourActualOpenAIKeyGoesHere  
SERPER_API_KEY=YourActualSerperKeyGoesHere
```

The World's Fastest & Cheapest Google Search API

No credit card required

Customize

6

Dashboard

All usage metrics at a glance

API keys

Top up

Credits left

2,500

Credits



05/19/2026 - 06/02/2026

Type filter



Total
0

19 May

20 May

21 May

22 May

23 May

24 May

25 May

26 May

27 May

28 May

29 May

30 May

31 May

Jun '26

Copy the Serper API Key

Place the key into your .env file

Ensure .env looks exactly like this:

```
OPENAI_API_KEY=sk-proj-  
YourActualOpenAIKeyGoesHere
```

```
SERPER_API_KEY=YourActualSerperKeyGoesHere
```

What is Serper?

Serper (specifically ***Serper.dev***) is a fast, low-cost **Google Search** API built for developers.

It allows applications, software pipelines, and AI models to programmatically (such as via Python) search Google and receive real-time **Search Engine Results Page** (SERP) data in a structured format.

The Multiagent: The Researcher and the Writer

These are two Agents that will work together to research and to write/create.

The steps/code will follow.

To start, create a new .py file in your Project in VS Code.

Call it: MAgent_YourName.py

We will also create a setup file.

The setup file (MAC on next slide)

For Windows – this is the code that goes in the setup.ps1 file in your Project:

TO run the following use: .\setup.ps1

Temporarily bypass execution policies just for this terminal process

Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass

Activate the virtual environment natively in PowerShell

.venv\Scripts\Activate.ps1

Upgrade pip to prevent installation warning messages

python -m pip install --upgrade pip

Double-check and install the pinned requirements

pip install -r requirements.txt

Execute the MAgent workflow within the correct environment

python MAgent.py

Keep the window open

Read-Host -Prompt "Workflow complete. Press Enter to exit"

For MAC

The Mac Alternative: Create a *setup.sh* Script

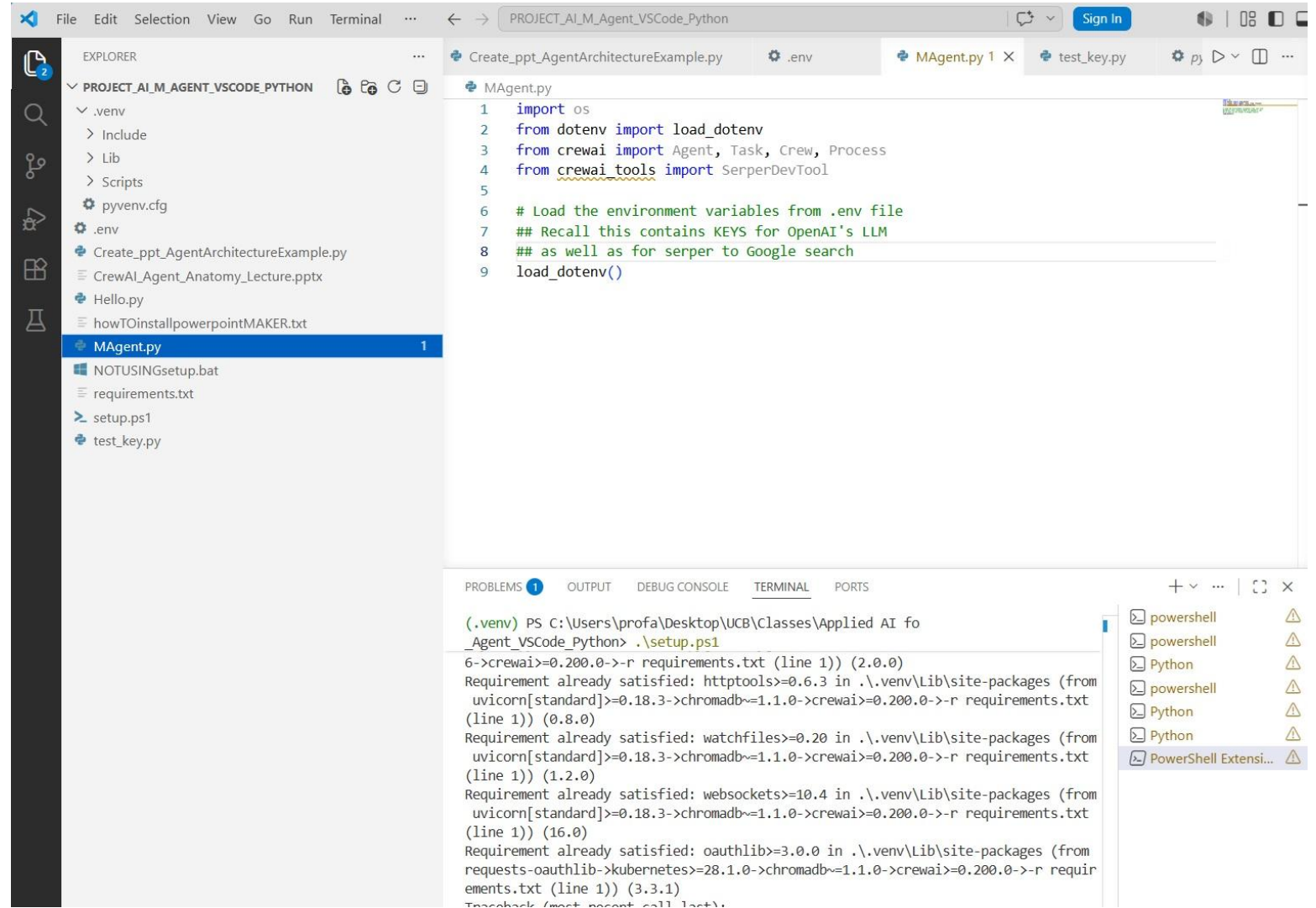
Create a separate script file in the root Project folder in VS Code named **setup.sh** and paste this code:

```
#!/bin/bash  
  
# Activate the local virtual environment on macOS  
source .venv/bin/activate  
  
# Download and install dependencies  
pip install -r requirements.txt  
  
# Execute the multi-agent workflow  
python crew_workflow.py
```

To run it: `chmod +x setup.sh`

NOTE: I do not have a MAC. If you do, you may need to do some research to make things work.

This is what we have so far...



```
File Edit Selection View Go Run Terminal ... PROJECT_AI_M_AGENT_VSCODE_PYTHON
Create_ppt_AgentArchitectureExample.py .env MAgent.py 1 X test_key.py
MAGENT.py
1 import os
2 from dotenv import load_dotenv
3 from crewai import Agent, Task, Crew, Process
4 from crewai_tools import SerperDevTool
5
6 # Load the environment variables from .env file
7 ## Recall this contains KEYS for OpenAI's LLM
8 ## as well as for serper to Google search
9 load_dotenv()
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI fo
_Agent_VSCode_Python> .\setup.ps1
6->crewai>=0.200.0->-r requirements.txt (line 1)) (2.0.0)
Requirement already satisfied: httpxtools>=0.6.3 in .\venv\Lib\site-packages (from
uvicorn[standard]>=0.18.3->chromadb~=1.1.0->crewai>=0.200.0->-r requirements.txt
(line 1)) (0.8.0)
Requirement already satisfied: watchfiles>=0.20 in .\venv\Lib\site-packages (from
uvicorn[standard]>=0.18.3->chromadb~=1.1.0->crewai>=0.200.0->-r requirements.txt
(line 1)) (1.2.0)
Requirement already satisfied: websockets>=10.4 in .\venv\Lib\site-packages (from
uvicorn[standard]>=0.18.3->chromadb~=1.1.0->crewai>=0.200.0->-r requirements.txt
(line 1)) (16.0)
Requirement already satisfied: oauthlib>=3.0.0 in .\venv\Lib\site-packages (from
requests-oauthlib->kubernetes>=28.1.0->chromadb~=1.1.0->crewai>=0.200.0->-r requir
ements.txt (line 1)) (3.3.1)
Traceback (most recent call last):
```

powerShell powershell Python powershell Python PowerShell Extensi...

Code for MAgent.py

(Note: Yours is called MAgent_YourName.py. Mine is just called MAgent.py)



The following slides will take you through coding your first Multi Agent Workflow.



Let's build it!



Place all the following code into



MAgent.py

One More Addition to .env

You will find that we make additions to .env as we go.
That's OK! Coding is “building”.

Let's now add the following to .env:

```
OPENAI_MODEL_NAME=gpt-4o-mini
```

NOTE: In industry, we would likely use ***gpt-4o***, but we want to keep our costs very low here.

(Anytime you restart VS Code or update .env or any other code in the Project, you should run `.\setup.ps1`)

Step 1: Imports and Loading your env

```
import os from dotenv
import load_dotenv from crewai
import Agent, Task, Crew, Process from crewai_tools
import SerperDevTool
from crewai.tools import tool
```

```
# Load the environment variables. So far this include
## The Keys to OpenAI LLM as well as
## serper.dev for Google searches
from .env file load_dotenv()
# Initialize the Serper search tool
google_search_tool = SerperDevTool()
llm = "gpt-4o-mini"
```

Here, we first load information stored in our .env.

Next, we instantiate the SerperDevTool() so we can use it.

We are using CrewAI – here is the reference <https://docs.crewai.com/en/concepts/agents>

Step 2: Code Agent 1

Define the Job Hunter Agent (Agent 1) with safety built in

```
JobHunter = Agent(  
    role="DS Job Hunter",  
    goal="Find current and open jobs in the area of data science, machine learning, and AI",  
    backstory="""You are job hunter. You search through  
online job data, specifically in the areas of data science,  
machine learning, and AI, to find open jobs.""" ,  
  
    verbose=True,  
    llm=llm,  
    allow_delegation=False,  
    tools=[google_search_tool],  
  
    # Cost & Safety Settings  
    max_iter=5,  
  
    # NOTICE: max_iter=15 to 25 -> Normally Production agents are given  
    # more "thinking room" to retry failed web queries, but this increases API token costs.  
  
    max_rpm=10  
    # NOTICE: max_rpm=None -> Normally, accounts use high-tier API plans with  
    # custom rate-limits, so they do not artificially throttle agent speed.  
)
```

Step 3: Code Agent 2

```
# Define the Writer Agent (Agent 2)
```

```
JobLister = Agent(  
    role="Job Options Lister",  
  
    goal="Create useful and applicable information about job options and listing \  
in the areas of data science, machine learning, and AI",  
  
    backstory="""You are a skilled writer who transforms dense job listings and job related  
information  
into clear, useful, and applicable blog posts. You make job requirements and focusses  
easy for to understand and to apply for.""" ,  
  
    verbose=True,  
    llm=llm,  
    allow_delegation=False,  
  
    max_iter=3,  
    max_rpm=10  
  
    ## Again, your max_iter and max_rpm can be higher later  
)
```

Step 4: Define an Agent “Task” (this one searches Google)

```
# Define the Job Search Task
```

```
job_search_task = Task(  
    description="""Investigate the latest job listings in the areas of data science,  
machine learning, and AI for the last month.  
Focus on job openings that are seeking a recent graduate in the areas of data science,  
machine learning, or AI.""" ,  
    expected_output="A bulleted summary of at least 5 " \  
"job openings with source context and for each at least 4 skills required for that job.",  
    agent=JobHunter,  
    tools=[google_search_tool]
```

```
##Recall that you already have the following in above code:
```

```
## Initialize the Serper search tool
```

```
##google_search_tool = SerperDevTool()
```

```
)
```

Step 5: Define another Agent “Task” (This one write markdown to a file saved as .md)

```
# Define the Writing Task
```

```
job_write_task = Task(  
    description="""Using the job search summary provided, write a bullet list that  
        describes the focus of the job, and at least 5 skills required."""  
    \ expected_output="A focused and easy to review formatted markdown blog post with a clear"  
    \ " list of jobs and their key topics or requirements."  
    agent=JobLister,  
    output_file="ai_agents_job_report.md"  
)
```

An .md file stands for a **Markdown file**. It is a lightweight, plain-text document that uses simple text symbols to format text (like making words bold, creating lists, or adding headers). When a Python script creates an .md file inside Visual Studio Code (VSCode), it means that your Python code wrote plain text formatted with Markdown syntax into a file ending in .md.

Step 6: Define your “Crew” (Group of workers and tasks they do)

```
# Define the Crew
```

```
job_crew = Crew(  
    agents=[JobHunter, JobLister], ## Our Agents  
    tasks=[job_search_task, job_write_task], ## Tasks our Agents can perform  
    process=Process.sequential,  
    verbose=True  
  
    # NOTICE: memory=True -> Enterprise crews often use an underlying  
    # vector database (like ChromaDB)  
    # to remember past execution details, which increases setup complexity  
    # and infrastructure cost. We will look at this and RAG later on.  
)
```

The line “process=Process.sequential” configures an AI agent framework (here - CrewAI) to execute tasks one after the other in a strict, linear order.

Task B will not start until Task A is completely finished. and the output and results of the first task are automatically passed to the next agent as context. Here, Agents cannot choose when to work and must wait for their designated turn in the queue.

Step 7: Run main and capture the results

```
if __name__ == "__main__":  
    print("\n## Starting the Multi-Agent Workflow via setup.ps1...")  
    result = job_crew.kickoff()  
    print("\n## Workflow Completed Successfully! Check ai_agents_job_report.md")  
    print(result.raw)
```

Recall – to run this code, run the following:

1) Run setup.ps1 (or similar) if you are just getting started
If you already ran it, there is no need to run it each time.
This file sets up your environment.
To run it: `.\setup.ps1`

2) The run your multiagent Python code. Mine is called MAgent.py
To run it: `python MAgent.py` in the VS Code terminal

FYI

The line of code:

```
if __name__ == "__main__":
```

ensures that specific blocks of code only run when the Python file is executed directly, and not when it is imported as a module into another file.

Each time Python runs a file, it automatically sets a few special built-in variables. One of these variables is ***__name__***.

In “Direct Execution”, if you run the file directly (say by typing ***python script.py*** in VSCode), Python sets ***__name__*** to equal ***"__main__"***.

The ***if statement then*** evaluates to ***True***, and the code inside it runs.

Otherwise, if the file is imported as a Module, then Python sets ***__name__*** to the actual filename instead. The if statement evaluates to False, and the code inside it is safely skipped.

Results

Recall that we have this line of code in one of our Tasks:

```
output_file="ai_agents_job_report.md"
```

So, let find this newly created file...

> Lib
> Scripts
pyenv.cfg
.env

ai_agents_job_report.md

Create_ppt_AgentArchitectureExample.py

CrewAI_Agent_Anatomy_Lecture.pptx

Hello.py

howTOinstallpowerpointMAKER.txt

MAgent.py

mycopy.py

NOTUSINGsetup.bat

requirements.txt

setup.ps1

test_key.py

science, machine learning, or artificial intelligence, here are some exciting job openings along with their key focuses and required skills.

1. [Entry Level Machine Learning Engineer jobs in the United States]

(<https://www.linkedin.com/jobs/entry-level-machine-learning-engineer-jobs>)

- **Focus**: Building and deploying machine learning models.
- **Skills Required**:
 - Python and R programming
 - Familiarity with machine learning libraries (e.g., TensorFlow, PyTorch)
 - Data preprocessing and analysis
 - Knowledge of algorithms and statistical techniques
 - Ability to work closely with data sets to derive insights

2. [New Grad Data Science, AI & ML Jobs - Simplify](<https://simplify.jobs/1/New-Grad-Data-Science-AI-ML>)

- **Focus**: Analyzing data and developing insights using AI and machine learning techniques.
- **Skills Required**:
 - Proficiency in Python and SQL
 - Experience with data visualization tools (e.g., Tableau, Power BI)
 - Understanding of machine learning concepts and algorithms
 - Ability to communicate complex findings to non-technical stakeholders
 - Strong teamwork and collaboration abilities

3. [New Grad Data Scientist Jobs - Employment - Indeed](<https://www.indeed.com/new-grad-data-scientist-jobs>)

These job listings represent excellent opportunities for recent graduates eager to build their careers in the innovative fields of data science, machine learning, and artificial intelligence. Don't forget to explore each link for application details and further information!



Build with Agent

AI responses may be inaccurate
[Generate Agent Instructions](#) to onboard AI onto your codebase.

© 2026 Dr. Ami M. Gates | All Rights Reserved | Free Distribution Only

> OUTLINE

> TIMELINE

PROBLEMS 1

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

+ v ... | [] x

powershell

PowerShell Extension

+ ai_agents_job_repo

Describe what to build

+ | | ... ↑

L... Def...

x 0 1

Ln 1, Col 1 Spaces: 2 UTF-8 CRLF { } Markdown

7:09 PM

Inside the .md file

© 2026 Dr. Ami M. Gates|
All Rights Reserved | Free
Distribution Only

Job Listings for Recent Graduates in Data Science, Machine Learning, and AI

If you are a recent graduate looking to jumpstart your career in data science, machine learning, or artificial intelligence, here are some exciting job openings along with their key focuses and required skills.

1. [Entry Level Machine Learning Engineer jobs in the United States](<https://www.linkedin.com/jobs/entry-level-machine-learning-engineer-jobs>)

- ****Focus****: Building and deploying machine learning models.
- ****Skills Required****:
 - Python and R programming
 - Familiarity with machine learning libraries (e.g., TensorFlow, PyTorch)
 - Data preprocessing and analysis
 - Knowledge of algorithms and statistical techniques
 - Ability to work closely with data sets to derive insights

2. [New Grad Data Science, AI & ML Jobs - Simplify](<https://simplify.jobs/1/New-Grad-Data-Science-AI-ML>)

- ****Focus****: Analyzing data and developing insights using AI and machine learning techniques.
- ****Skills Required****:
 - Proficiency in Python and SQL
 - Experience with data visualization tools (e.g., Tableau, Power BI)
 - Understanding of machine learning concepts and algorithms
 - Ability to communicate complex findings to non-technical stakeholders
 - Strong teamwork and collaboration abilities

FYI – for Later – To Build Your Own Tool(s)

```
from crewai.tools import BaseTool
from pydantic import BaseModel, Field
from typing import Type
```

```
# Define the student-facing input schema using modern Pydantic v2
```

```
class MyCustomToolInput(BaseModel):
```

```
    """Input schema specifications for the custom tool."""
```

```
    query_text: str = Field(..., description="The search string or parameter text passed into the tool.")
```

```
# Construct the tool class by inheriting directly from BaseTool
```

```
class MyCustomTool(BaseTool):
```

```
    name: str = "My_Custom_Utility_Tool"
```

```
    description: str = "A clear description explaining what this tool does so the AI agent knows when to use it."
```

```
    args_schema: Type[BaseModel] = MyCustomToolInput
```

```
    def _run(self, query_text: str) -> str:
```

```
        """The internal processing execution logic. Place your raw Python code here."""
```

```
        # Example teaching placeholder logic:
```

```
        output_result = f"Custom tool successfully processed input text: {query_text}"
```

```
        return output_result
```

OK! Let's Summarize What We Did

- 1) We set up a new Programming Environment for coding Agentic Workflows using LLMs.
- 2) There were MANY steps need to do this including things like getting VS Code, getting Python (the right version), getting keys for APIs we plan to use (like for OpenAI), etc. etc.
 - 1) *** Review the set up that we did!
- 3) Next, and after choosing which libraries we wanted to use, we coded Agents and Tasks and then built our Crew.
- 4) Please review the slides (there are nearly 100!) so assure that all the steps make sense. While the exact steps are rarely the same, the concepts of setting up an environment are critical.

RAG and Vector Databases

Gates

© 2026 Dr. Ami M. Gates|
All Rights Reserved | Free
Distribution Only

Vector Databases vs. RAG

- 1) RAG (Retrieval-Augmented Generation) is a design pattern.
- 2) A vector database is a component often used to build RAG and other designs for AI workflows.
- How they differ and how they work together:
 - **Vector Database:** A specialized **storage system** that holds data that has been converted into numerical formats (embeddings) so one can perform fast mathematical similarity searches. An embedding is when a word (or other token differentiation) is represented by a vector so that similar words/tokens are “near” to each other in vector space.
 - **RAG:** RAG is a method/process that retrieves relevant data from an external source (like a vector database or an API) and hands it to an LLM to generate an accurate, context-aware answer.
 - **The Relationship:** A vector database is often used *inside* a RAG system to find the right information quickly.

More on RAG (Retrieval- Augmented Generation)

- 1) RAG is an **AI framework** that connects a Large Language Model (LLM) to external, **private** data sources to deliver **accurate, up-to-date answers/information**.
- 2) A common analogy is giving an open-book exam to an AI. Instead of relying purely on its memory or general training, the AI looks up the facts in the book first, then writes the response using the most current and accurate information.

Why is RAG Important

- 1) Standard LLMs (and to Agentic Systems or Workflows that use LLMs) suffer from two major limitations:
 - 1) **Knowledge Cutoffs:** They only know information available up to their last training date.
 - 2) **Hallucinations:** They often invent plausible-sounding but completely false facts when they lack accurate data.
- 2) RAG fixes both problems by forcing the LLM to search for its answers in real, verifiable documents that “we” provide directly.

How Does RAG Work? (The three steps)

Step 1 - Retrieval: When a question is asked (say by a User or by AI Agents, etc.), a **specialized search** system scans your **data repository** (PDFs, internal wikis, SQL databases) to pull up the most relevant and recent paragraphs/information.

Step 2 - Augmentation: The system “glues” the original question and those recovered paragraphs together into a single, comprehensive prompt. (Improved prompt engineering)

Step 3 - Generation: The LLM reads this **combined prompt** and uses the provided context to write a highly precise and accurate response.

RAG – The Technical Pipeline

Building a production-ready RAG system generally requires a standard **data ingestion workflow** including:

- 1) **Document Chunking:** Large manuals or files are chopped into smaller, bite-sized sections (e.g., 500-word blocks).
- 2) **Embedding Models:** These text chunks pass through an embedding model that converts words into mathematical vectors representing their semantic meaning.
- 3) **Vector Databases:** Tools like Chroma, Pinecone, or FAISS store these embeddings (vectors).
- 4) **Result:** When a user asks a question (such as a person or an Agent), the database performs a **mathematical similarity search** to instantly find matching (and more current/relevant) data.

RAG – Further Core Benefits

- 1) **Data Privacy:** You can query private company data without using it to retrain or expose public models.
- 2) **Cost Efficiency:** Updating your data repository is cheap and instantaneous whereas retraining a base foundation model or LLM costs millions.
- 3) **Verifiable Sources:** Answers can include citations or links directly back to the source document, making validation fast and feasible.

RAG References

The RAG Triad: A Learner's Guide to Building Truthful AI by Dr Akshika Wijesundara

<https://www.linkedin.com/pulse/rag-triad-learners-guide-building-truthful-ai-wijesundara-phd-sktxe/>

Introduction to RAG by Moer Amer

<https://cohere.com/llmu/introduction-to-rag>

What is Retrieval Augmented Generation (RAG)? A comprehensive guide by Leeway Hertz

<https://www.leewayhertz.com/what-is-retrieval-augmented-generation/>

How RAG works: An essential guide by Leapfrog Blog

<https://www.lftechnology.com/blogs/how-RAG-works-essential-guide>

NOTE: These are just a few. Always research your own reviews, tutorials, videos, and guides. Learning how to learn new things is critical in the areas of AI, DL, and DS.

More on Vector Databases and Embedding

- 1) In the context of **AI Agents** (like crews built with CrewAI, LangChain, or AutoGen), **embeddings** and **vector databases** act as the agent's **long-term memory** and **semantic reasoning engine** as well as sources for information.
- 2) While standard RAG uses these tools often just to answer questions, **AI agents** use them to **remember past conversations**, look up specialized tools, and maintain a consistent "personality" or knowledge base over time.

What are Embeddings?

- 1) An **embedding model** turns human language into a collection of **vectors** that capture the **conceptual meaning** of the text.
- 2) **Similarity**: Tokens, Words or sentences with similar meanings are placed **close** together in a multi-dimensional vector space.
Example: The sentence "The server is down" and "We have a service outage" look completely different to a traditional keyword search. However, an embedding model creates vectors for both that are “close” to each in vector space.
- 3) **Why Agents Need Embeddings**: Agents must understand user intent, even if the user uses typos, slang, or indirect language. There are many other reasons as well ranging from space and time to context and accurate communication.

More on Embedding

- An **embedding** is a technique used to translate complex data—like words, images, or audio—into a list of numbers (a vector), allowing machines to understand both the "meaning" and relationships between items.
- **How the Process Works**
- **Translating to Numbers:** An AI model evaluates the data and represents its traits as a high D vector.
- **The "Embedding Space":** Every piece of data (token) is represented as a coordinate (a point in high D space) such that context and similarity are maintained.
- **Grouping by Meaning:** Items with similar meanings or characteristics are plotted close to each other in this space, while unrelated items are placed far apart.
- More: VIDEO:
<https://www.youtube.com/watch?v=vr0MHUMZPv8>

Word Embeddings – Past and Present – From word2vec to

Today's embedding tools generate dynamic, context-aware representations tailored for large-scale information retrieval and AI systems.

They vastly outperform past models by capturing nuanced meaning across sentences and documents.

Reference:

<https://jeftaylo.medium.com/a-deep-dive-into-nlp-embeddings-from-word2vec-to-text-embedding-ada-002-and-why-you-might-want-9f470f184271>

Most Commonly Used Embedding Tools (2026)

Commercial APIs

1. **OpenAI:** Leading the industry with models like ***text-embedding-3-small*** and ***text-embedding-3-large***. They natively support **Matryoshka representation learning**, which allows users to truncate vectors to save on storage and processing costs.
2. **Google:** The new ***gemini-embedding text models*** (available via the Google Gemini API) consistently top the Massive Text Embedding Benchmark (MTEB) for multilingual retrieval.
3. **Cohere & Voyage AI:** Highly popular third-party enterprise providers offering domain-specific and multilingual-tuned embeddings optimized for complex enterprise RAG (Retrieval-Augmented Generation) applications

Open Source Models

- **Qwen:** The **Qwen** family of embedding models (e.g., Qwen3-Embedding-8B) provides top-tier open-source multilingual performance rivaling closed APIs.
- **BAAI General Embedding (BGE):** The BGE-M3 model is highly favored because it simultaneously handles dense, sparse, and multi-vector retrieval.
- **Sentence-Transformers (Hugging Face):** Frameworks utilizing models like all-MiniLM-L6-v2 remain the standard for lightweight, local, and edge computing environments.

Modern vs Past Embedding Tools

Context Sensitivity

1. **Past Tools** (Word2Vec & GloVe) created static word embeddings.
 1. A word has exactly **one vector representation** regardless of its surrounding text.
 2. For example, the word "bank" is given the same vector representation whether it means a riverbank or a financial institution.
2. **Modern Tools:** Modern models (such as BERT, GPT, and modern dense retrievers) use Transformers to produce context-sensitive embeddings.
 1. The vector for "bank" dynamically changes based on the words in the sentence, allowing the AI to naturally understand polysemy and linguistic nuance.

Scope of Representation

- **Past Tools:** Operate primarily at the **token or word level**. To find the vector for a sentence or document, developers had to average or sum the vectors of all the individual words in it, which results in a loss of syntactic meaning.
- **Modern Tools:** Operate as **universal document and sentence encoders**. They intake whole paragraphs or pages at once and assign a single, highly accurate dense vector that captures the whole meaning of a phrase.

More on Vector Databases

Vector databases are specialized storage engines designed specifically to hold and search high-dimensional vector embeddings.

Unlike **traditional databases** that match keywords exactly, vector databases use mathematical distance to find data based on conceptual similarity.

Vector Search:

Traditional databases search for exact matches (e.g., "red shoes").

Vector databases search for semantic meaning (e.g., "maroon sneakers" or "crimson footwear") by calculating how close vectors are to each other in a multi-dimensional geometric space.

Most Common Vector Databases

Dedicated (Native) Vector Databases

Pinecone: A fully managed, cloud-native SaaS database heavily favored for production AI apps because it requires zero infrastructure setup.

Milvus: An enterprise-grade, highly scalable, open-source distributed database designed for handling billions of vectors.

Qdrant: A fast, open-source vector search engine written in Rust, known for high resource efficiency and powerful filtering capabilities.

Weaviate: An open-source database that natively integrates machine learning models into its storage pipeline to automatically vectorize data upon ingestion.

Chroma: A lightweight, open-source database designed to be easily embedded directly into Python applications for quick prototyping.

How Vector Databases Work

Ingestion: Raw data (text, images, audio) is converted into a string of numbers (a vector) using an embedding model.

Storage: The database stores the vector alongside reference pointers to the original raw data (called "metadata").

Indexing: The database builds a complex mathematical index. Instead of searching every vector sequentially, it clusters similar vectors together using Approximate Nearest Neighbor (ANN) algorithms like HNSW (Hierarchical Navigable Small World).

Querying: When a user searches, their query is converted into a vector, and the database instantly returns the closest vectors in that geometric space

Primary Use Cases for Vector Databases

- **Retrieval-Augmented Generation (RAG):** Supplying Large Language Models (LLMs) with custom, up-to-date company data to eliminate AI hallucinations.
- **Recommendation Engines:** Finding products, music, or videos that are conceptually similar to what a user previously liked.
- **Anomaly Detection:** Identifying fraud or cyber threats by detecting data points that sit too far away from normal behavioral clusters.
- **Image & Video Search:** Matching visual assets based on their content rather than file names or manual tags.

References: Vector Databases

- **Vector Databases Explained (2026) - VIDEO**

Source: [Vector Databases Explained: The Complete Guide for 2026 on YouTube](#)

- **For Hands-on Python Developers: *ChromaDB & LangChain Practice* - Practical Code Tutorial**

Source: [Complete Tutorial on Vector Database - Learn ChromaDB on YouTube](#)

- **The Video Guide (Hands-on Framework Mastery) - Deep-dive video tutorial**

Resource: [CrewAI RAG Deep Dive: Basic & Advanced Examples on YouTube](#)

- **The Text Guide (Production Design & Deployment) - Step-by-step engineering blog**

Resource: [How to Build Multi-Agent AI with CrewAI Python in 13 Steps on Tech-Insider \[1\]](#)

- **For System Architects: *The 2026 Vector Database Decision Matrix* - Engineering Guide & Production Analysis**

Source: [Top 15 Vector Databases in 2026: A Production Guide on Medium](#)

- **For Enterprise AI Engineers: *Building an End-to-End RAG Pipeline* - Deep-Dive Technical Blog**

Source: [How to Build a RAG Pipeline from Scratch in 2026 by Kapa.ai](#)

Adding on to our Multi Agent Workflow

Gates

© 2026 Dr. Ami M. Gates|
All Rights Reserved | Free
Distribution Only

What We Can Add to Our Existing Workflow

- Since our agents currently scrape live data science jobs via Serper, we can add a RAG component to **match those scraped jobs against our own internal data.**
- **Use Case:**
- **Candidate Matching:** Store a pool of student or applicant resumes in a vector database. Have our agents query that database to find the best match for the newly scraped jobs.
- **Historical Deduplication:** Store previously found jobs in the database. Before summarizing, have agents check if the "new" job is already in your database to avoid duplicate work.
- **Skills Gap Analysis:** Store a standard data science curriculum or common skillsets. Have the agents compare the scraped job descriptions against such skills to highlight what may be missing, etc.

OK – Pause and Test

To make sure that everything is working as expected so far

–

- 1) Close VS Code
- 2) Open VS Code and run setup.ps1 (.\setup.ps1 for Windows)
- 3) Then run your Agent code (mine is called MAgent.py)

python MAgent.py

Everything should run fine and you should see a new .md file you can look at.

If something does not work – trace it back and resolve it. Otherwise, you are ready to move forward into adding RAG/VD into your code and workflow.

Adding RAG For a Resume or CV on Our Computer (as a pdf)

Let's update our code to add a RAG option that uses a resume (as pdf) that we have on our computer.

SUGGESTION: Create a new Python file and paste your current MAgent.py code into it. This preserves the old MAgent.py and also allows you to make updates. I am calling my new file MAgent_RAG.py.

Next, we will use the **PDFSearchTool** and will include it **into our** second agent (JobLister). This lets the writer agent read your local **my_resume_name.pdf** file dynamically, compare it to the web search results, and generate a tailored report.

Let's see how this works...

How to Code RAG into CrewAI

CrewAI provides native “**Tools**” designed specifically for RAG.

Let’s start with “PDFSearchTool”

Include this in your code:

```
from crewai_tools import SerperDevTool,  
PDFSearchTool
```

DO the following:

Into the ***requirements.txt*** file, add these lines:

```
chromadb  
langchain-openai  
pypdf  
pydantic  
llama-parse
```

Now requirements.txt contains:

```
crewai  
python-dotenv  
chromadb  
crewai-tools  
langchain-openai  
pydantic  
pypdf
```

Placing your pdf Resume in the right location

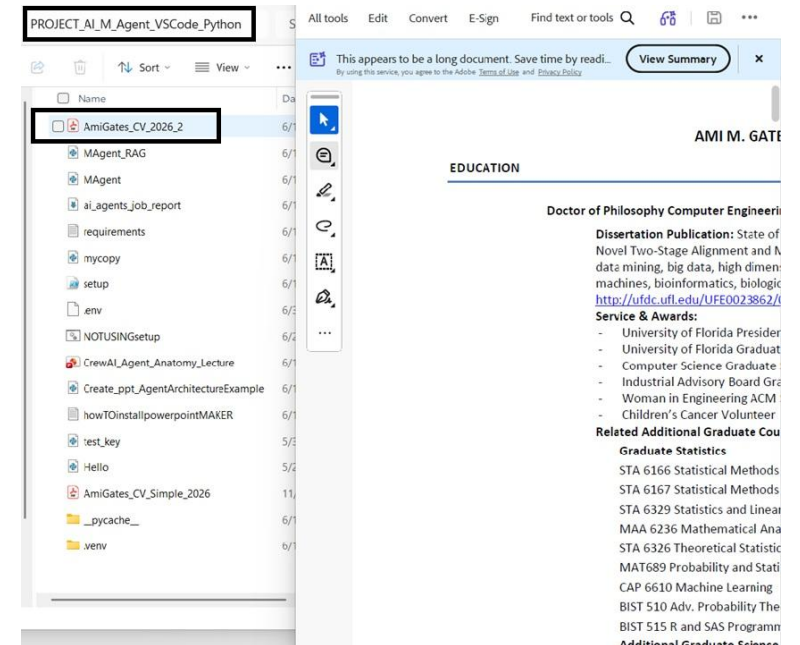
First, you will need to have a resume 😊

Assure it is saved as pdf.

Place it into your Project folder

Do not make it over-complicated. Avoid images, tables, and other fancy element for now (we can get to that later).

The more clear and readable your resume pdf is, the better it will be interpreted.



Add in this block of code to you MAgent_RAG.py

Place it just after

```
load_dotenv()
```

```
llm = "gpt-4o-mini"
```

```
# Initialize the Serper search tool
```

```
google_search_tool = SerperDevTool()
```

```
# =====  
# RAG: Initialize the pdf Search Tool  
# =====  
# This tool opens a raw PDF, chunks the text, and sets up a local  
# Chroma vector index automatically behind the scenes.  
# Ensure a file named "my_resume_name.pdf" (whatever YOUR resume name is)  
# exists in your VS Code workspace directory - in your Project Folder.  
resume_rag_tool = PDFSearchTool(  
    pdf="AmiGates_CV_2026_2.pdf",  
    config={  
        "llm": {  
            "provider": "openai",  
            "config": {"model": "gpt-4o-mini", "temperature": 0.1}  
        },  
        "embedder": {  
            "provider": "openai",  
            "config": {"model": "text-embedding-3-small"}  
        }  
    }  
)
```

Next, Update your JobLister Agent to the following:

```
#=====
# Define the Writer Agent
#=====
JobLister = Agent(
    role="Job Options Lister",
    goal="Create useful and applicable information about job options and listing \
in the areas of data science, machine learning, and AI",
    backstory="""You are a skilled writer who transforms dense job listings and job related
information into clear, useful, and applicable blog posts. You make job requirements and
focusses easy for to understand and to apply for.""" ,
    verbose=True,
    llm = llm,
    allow_delegation=False,
    # Give this agent access to the PDF RAG tool
    tools=[resume_rag_tool],
    max_iter=3,
    max_rpm=10
)
```

Next, update your job_write_task and your job_crew

```
#=====
# Define the Writing Task
#=====
job_write_task = Task(
    description="""Using the job search summary provided by JobHunter, use your
resume rag tool to read the candidate's 'my_resume.pdf'.
Cross-reference the found jobs with the skills on the resume. Write a markdown blog post
that highlights these roles,
explains how well they match the resume stack, and notes any skill gaps."""
    ,
    expected_output="A focused and easy to review formatted markdown blog post with a clear "
    \
    "list of jobs, their key requirements, and a personalized resume
alignment analysis.",
    agent=JobLister,
    output_file="ai_agents_job_report3.md"
)

#=====
# Assemble the Crew
#=====
job_crew = Crew(
    agents=[JobHunter, JobLister],
    tasks=[job_search_task, job_write_task],
    process=Process.sequential,
    verbose=True
)
```



> Include

> Lib

> Scripts

pyenv.cfg

.env

ai_agents_job_report.md

ai_agents_job_report3.md

AmiGates_CV_2026_2.pdf

AmiGates_CV_Simple_2026.pdf

Create_ppt_AgentArchitectureExample.py

CrewAI_Agent_Anatomy_Lecture.pptx

Hello.py

howTOinstallpowerpointMAKER.txt

MAgent_RAG.py

MAgent.py

mycopy.py

NOTUSINGsetup.bat

requirements.txt

setup.ps1

test_key.py

> OUTLINE

> TIMELINE

35

5. [Data Scientist New Grad @ SentiLink - Simplify](https://simplify.jobs/p/5486f8fc-e958-437b-934e-ad3295e9a178/Data-Scientist-New-Grad)

36

- ****Skills Required:****

37

- Knowledge of machine learning and statistical analysis

38

- Experience with big data technologies (e.g., Hadoop, Spark)

39

- Ability to code in Python or R

40

- Experience with data manipulation libraries (e.g., pandas, NumPy)

41

42

Resume Alignment Analysis

43

44

Candidate: Ami Gates

45

46

Ami Gates has an extensive academic and professional background in data science, machine learning, and related fields. Here's how her skills align with the job openings:

47

48

****1. Graduate Data Scientist****

49

- ****Alignment:**** Ami is proficient in Python and R, has taught data analysis and statistical techniques, and has a strong understanding of SQL from her past database management experience.

50

- ****Gaps:**** No significant gaps noted, as her qualifications exceed the requirements.

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

+

▼

...

|

↺

↻

×

📄 powershell

📄 Python

📄 PowerShell Extensi...

Trace Batch Finalization

✓ Trace batch finalized with session ID:
c98c62d9-8f8d-4136-ae30-449070c49469

🔗 View here:
https://app.crewai.com/crewai_plus/ephemeral_trace_batches/c98c62d9-8f8d-4136-ae30-449070c49469?access_code=TRACE-1a4a48ac53

🔑 Access Code: TRACE-1a4a48ac53

○ (.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python>

Ln 1, Col 1

Spaces: 2

LTE: 8

GBLE

© 2026 Dr. Ami M. Gates|
All Rights Reserved | Free
Distribution Only

Pause, Review, and Improve

Now, pause and review.

1) Notice that you have update things like Agent goal and backstory as well as Task description and expected output. Updating these will change the results.

Experiment with this.

2) Next, notice that we can use the web to get information (here we are using Serper/Google). We can also use forms of RAG.

3) So far, we use RAG (as PDFSearchTool) to review and use a resume in pdf format.

RAG Option: Searching a Specific Website

- 1) Next, we will use another RAG option.
- 2) Let's use RAG to look for information from a **specific webpage** (rather than just the entire web). We might do this because we want to return specific information – such as courses offered by a specific University.
- 3) To do this, we will create WebSearchTool options (as many as we want websites).
- 4) Then, we will create Agent(s) and Task(s) that use these new RAG tools.

Code for a Multi-Agent RAG Workflow

Gates

© 2026 Dr. Ami M. Gates|
All Rights Reserved | Free
Distribution Only

Bring in the Libraries

```
import os
from dotenv import load_dotenv
from crewai import Agent, Task, Crew, Process
from crewai_tools import SerperDevTool, PDFSearchTool, WebsiteSearchTool
from crewai.tools import tool
```

Load the environment and define the llm and Serper tool

```
# Load the environment variables from .env file
## Recall this contains KEYS for OpenAI's LLM
## as well as for serper to Google search
load_dotenv()
```

```
llm = "gpt-4o-mini"
```

```
# Initialize the Serper search tool
google_search_tool = SerperDevTool()
```

The RAG pdf tool

```
# =====  
# RAG: Initialize the pdf Search Tool  
# =====  
# This tool opens a raw PDF, chunks the text, and sets up a local  
# Chroma vector index automatically behind the scenes.  
# Ensure a file named "my_resume_name.pdf" (whatever YOUR resume name is)  
# exists in your VS Code workspace directory - in your Project Folder.  
resume_rag_tool = PDFSearchTool(  
    pdf="AmiGates_CV_2026_2.pdf",  
    config={  
        "llm": {  
            "provider": "openai",  
            "config": {"model": "gpt-4o-mini", "temperature": 0.1}  
        },  
        "embedder": {  
            "provider": "openai",  
            "config": {"model": "text-embedding-3-small"}  
        }  
    }  
)
```

NEW: Define the RAG websites we will access

```
# =====  
# Define the University Website Searcher tool for DS and CS  
# =====  
  
ds_courses_tool = WebsiteSearchTool(  
    website="https://catalog.colorado.edu/courses-a-z/dtsa/",  
    config={"llm": {"provider": "openai", "config": {"model":  
"gpt-4o-mini"}}}  
)  
  
# Tool for the Computer Science curriculum page  
cs_courses_tool = WebsiteSearchTool(  
    website="https://catalog.colorado.edu/courses-a-z/csci/",  
    config={"llm": {"provider": "openai", "config": {"model":  
"gpt-4o-mini"}}}  
)
```

Notice that we use
WebSearchTool for this.

Recall:
`from crewai_tools`
`import SerperDevTool,`
`PDFSearchTool,`
`WebsiteSearchTool`

You can find documentation, examples, and explanations for everything we use on various sites. Here is an example:

<https://docs.crewai.com/v1.14.7/en/tools/search-research/websitesearchtool>

Now create some Agents

```
# =====  
# Define a Course Advisor AGent Based on Job/skill deficits  
# =====  
CourseAdvisor = Agent(  
    role="University Course Advisor",  
    goal="Identify specific university courses that will bridge a candidate's technical skill  
gaps for data science and AI roles",  
    backstory="You are an academic advisor specializing in aligning university curricula with  
tech industry demands. You analyze job skill gaps and search university course listings to  
find exact course matches.",  
    verbose=True,  
    llm=llm,  
    allow_delegation=False,  
    tools=[ds_courses_tool, cs_courses_tool],  
    max_iter=5,  
    max_rpm=10  
)
```

```

# =====
# Define the Job Hunter Agent with safety built in
# =====
JobHunter = Agent(
    role="DS Job Hunter",
    goal="Find current and open jobs in the area of data science, machine learning, and AI",
    backstory="""You are job hunter. You search through
find online job data, specifically in the areas of data science, machine learning, and AI, to
    open jobs."""',
    verbose=True,
    llm = llm,
    allow_delegation=False,
    tools=[google_search_tool],
    # Cost & Safety Settings
    max_iter=5,
    # NOTICE: max_iter=15 to 25 -> Normally Production agents are given
    # more "thinking room" to retry failed web queries, but this increases API token costs.
    max_rpm=10
    # NOTICE: max_rpm=None -> Normally, accounts use high-tier API plans with
    # custom rate limits, so they do not artificially throttle agent speed.
)

```

```

#=====
# Define the Writer Agent
#=====
JobLister = Agent(
    role="Job Options Lister",
    goal="Create useful and applicable information about job options and listing \ in the
areas of data science, machine learning, and AI",
    backstory="""You are a skilled writer who transforms dense job listings and job
related information into clear, useful, and applicable blog posts. You make job
requirements and focusses easy for to understand and to apply for.""" ,
    verbose=True,
    llm = llm,
    allow_delegation=False,

    # Give this agent access to the PDF RAG tool
    tools=[resume_rag_tool],

    max_iter=3,
    max_rpm=10
)

```

About the Agents

Notice that we create three Agents.

Think about what each one does.

Think about whether you want them to work in serial or parallel.

Think about how to improve each.

Now, Let's Create Tasks for the Agents

```
#=====
# Define the Job Search Task
#=====
job_search_task = Task(
    description="""Investigate the latest job listings in the areas of data science,
machine learning, and AI for the last month.
Focus on job openings that are seeking a recent graduate in the areas of data science,
machine learning, or AI."""
    expected_output="A bulleted summary of at least 5 " \
"job openings with source context and for each at least 4 skills required for that
job.",
    agent=JobHunter,
    tools=[google_search_tool]
)
```

```

#=====
# Define the Course Recommendation Task
#=====
course_recommendation_task = Task(
    description="""1. Read ONLY the 'skill gaps' section from the
previous report.
2. Completely ignore any courses or history listed on the
candidate's resume.
3. Use your ds_courses_tool, cs_courses_tool to actively query the CU
Boulder website catalog.
4. Find NEW, un-taken courses that match those missing skills.

CRITICAL: Do not recommend any course the candidate has already
taken or taught. You must pull live data from the website tool.""",
    expected_output="A list of 3 brand-new CU Boulder courses from " \
    "the website tool matching the skill gaps.",
    agent=CourseAdvisor,
    tools=[ds_courses_tool, cs_courses_tool]
)

```

```

#=====
# Define the Writing Task
#=====
job_write_task = Task(
    description="""Using the job search summary provided by JobHunter as well as course
recommendations
    from the CourseAdvisor, and using your resume_rag_tool to read the resume. AND
using ds_courses_tool, cs_courses_tool
    to determine areas where the candidate can fill in missing gaps in knowledge with
respect to
    a comparison between their resume and the jobs listed
    Cross-reference the found jobs with the skills on the resume. Write a markdown blog
post that highlights these roles,
    explains how well they match the resume stack, and notes any skill gaps.""",
    expected_output="A focused and easy to review formatted markdown blog post with a
clear "\
    "list of jobs, their key requirements, and a personalized resume
alignment analysis.",
    agent=JobLister,
    output_file="ai_agents_job_report41.md"
)

```


Next, let's CREW UP the Agents and Tasks

```
#=====
# Assemble the Crew
#=====
job_crew = Crew(
    agents=[JobHunter, JobLister, CourseAdvisor],
    tasks=[job_search_task, course_recommendation_task, job_write_task],
    process=Process.sequential,
    verbose=True
)

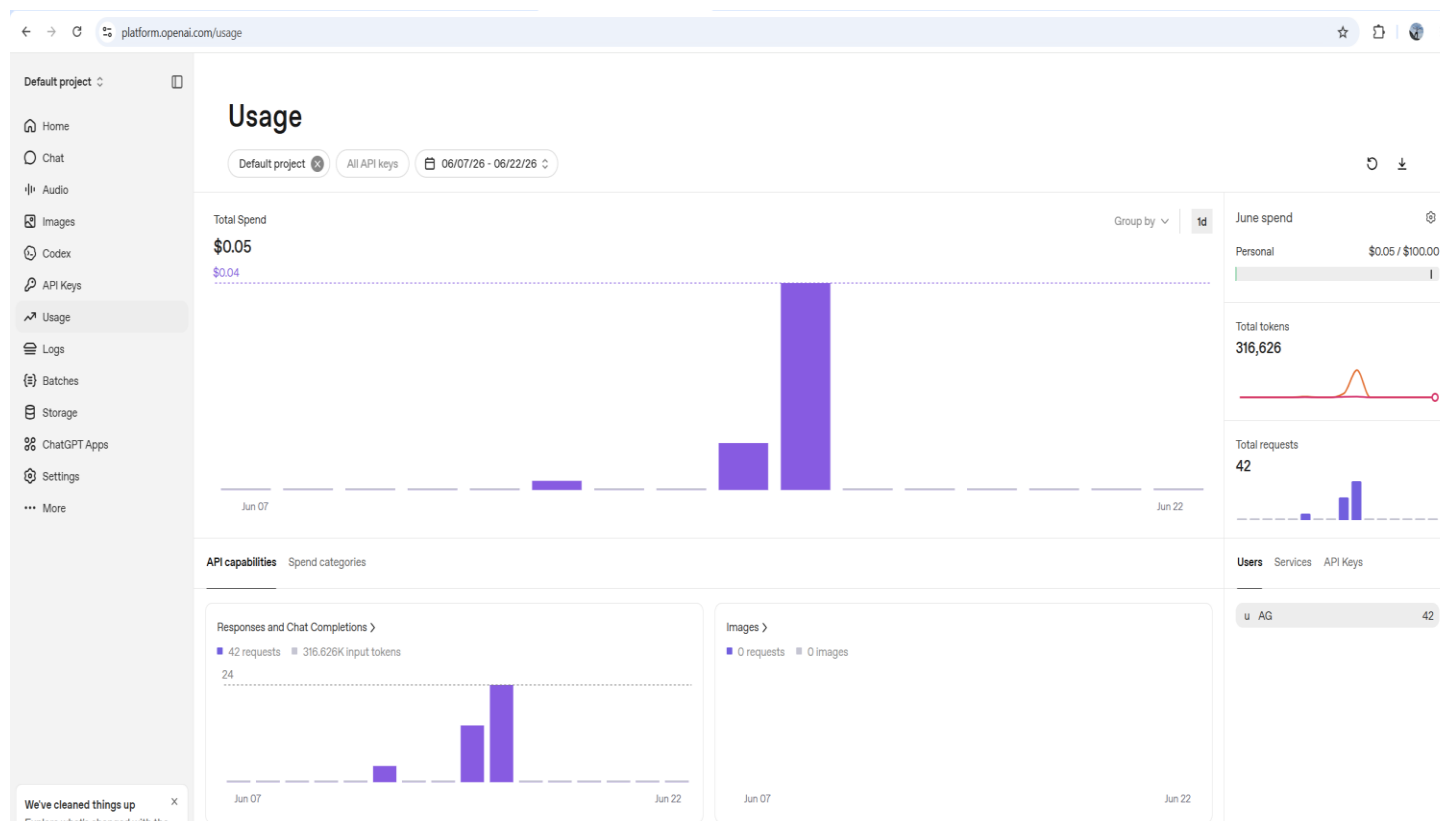
if __name__ == "__main__":
    print("\n## Starting the Multi-Agent Workflow via setup.ps1...")
    result = job_crew.kickoff()
    print("\n## Workflow Completed Successfully! Check ai_agents_job_report41.md")
    print(result.raw)
```

Notice that right now, we are using the Agents sequentially.
This is not a requirement.
There are actually many options.

Run the Code ONCE

(If you have
errors, correct
and re-run.)

Investigate the results.



Remember – each time you run this code, you will use OpenAI value - it will cost you money.

Not a lot of money – but you want to be careful and check your OpenAI balance.

View the Markdown (.md) file

Your code creates a .md file.

Open it and have a look.

Read it.

- Does it make sense?

Is it doing a “good job”?

- Is it doing what you want it to do?

How can you improve it?

- What other Agents and Tools can you create?

Advanced RAG

data structures - query styles - advanced agent behaviors

© 2026 Dr. Ami M. Gates|
All Rights Reserved | Free
Distribution Only

Structured Data RAG (Text-to-SQL / Text-to- Dataframe)

- A lot of real-world enterprise data is stored in databases rather than in “paragraphs”.
- Agents can interact with structured data such as databases or structured files (like csv).
- **SQL Database RAG:** Agents can use tools that connects to a SQLite or PostgreSQL databases for example. In such a case, the agent takes a natural language question, generates a precise SQL query, executes it against the database, and reads the table output to form a response.
- **CSV / Pandas RAG:** An agent can use a tool that loads a local .csv or .xlsx file (or similar) into a Pandas DataFrame (or similar). In this case, the agent uses Python code [in this example] to query, filter, and/or calculate statistics from the data to answer user questions.

Multi-Modal & Specialized Media RAG

- Modern RAG is no longer text-exclusive.
- Adding visual or audio data retrieval is an option that enables Agents to engage with multi-modal models.
- **Image / Vision RAG:** Here, a **vector database** can be used to store embeddings of images alongside their metadata or descriptions. An agent can retrieve relevant images or use a Vision LLM to search through charts, diagrams, or diagrams in slides.
- **Audio Transcription RAG:** Here, agents can use a tool that takes audio files (like .mp3 podcasts or lectures), run them through an API like OpenAI's Whisper, chunk the text transcript, and perform standard semantic retrieval on the spoken text.
- **Video / YouTube RAG:** Agents can also use a tool where a YouTube URL can be used as input. The tool fetches the auto-generated video transcript, embeds it, and allows the agent to answer questions about specific video timestamps.

Knowledge Graph RAG (GraphRAG)

- In some cases, a standard “vector search” can struggle with complex relationship tracking,
- For example: "Find all employees who worked under Manager X before 2024".
- **Entity-Relationship RAG:** Here, Agents can use a tool that connects to a **graph database** (like Neo4j) so that instead of finding text chunks based on raw similarity, the agent navigates **nodes and edges** to extract deep context and interconnected/related information.

Codebase RAG (Repo QA)

- **Local Repository Search:** Here, a tool can be created that lists files, parses code structure, and searches through a local Python project directory.
- Using this tool, an agent can scan function definitions and docstrings across multiple Python files to explain how the application works or to try to debug an issue.

API-Driven Live Data RAG (References here are links provided by AI)

- Traditional RAG relies on static vector stores. Live API RAG teaches students how to retrieve fluid, real-time context that changes minute by minute. [[1](#), [2](#)]
- **Weather or Market API RAG:** A tool that pulls live data from free endpoints (like Open-Meteo or CoinGecko). The agent uses this real-time data as context to answer time-sensitive user prompts. [[1](#), [2](#), [3](#)]
- **Internal App RAG:** A tool that interfaces with a mock CRM or Jira board, pulling live user profiles, tickets, or order histories into the LLM prompt context. [[1](#)]

Architectural RAG Patterns for Multi- Agent Systems

- Agents can also strategically select and optimize their retrieval methods.
- **Router Agent:** An introductory multi-index concept. Rather than giving one agent access to all tools, build a "Triage Agent" or "Router" that reads the user's intent and dynamically hands the task over to the single best sub-agent (e.g., routing to the PDF Agent vs the Web Search Agent). [[1](#), [2](#), [3](#), [4](#), [5](#)] (References from AI)
- **Corrective RAG (CRAG):** Implement a self-correcting loop. Agent A retrieves the PDF text chunks. Agent B (an evaluator agent) grades the quality of those chunks. If the chunks are irrelevant, Agent B automatically triggers a web search tool to find better information.

Resources and References

It is strongly recommended that you search the web and YouTube for the specific areas of interest to you and to your Project. The following offer just a few potential resources.

CREW: <https://docs.crewai.com/v1.15.1/en/concepts/tools>

VIDEO:

https://www.youtube.com/watch?v=QmUsG_3wHPg

<https://www.youtube.com/watch?v=Rxwjoeegpl98>

<https://www.youtube.com/watch?v=uN7X819DUIQ>

WEB:

<https://pub.towardsai.net/advance-your-rag-with-a-multi-index-agent-4789478d4bec>

MORE:

<https://rasa.com/blog/agent-orchestration-tools>

crewai

v1.15.1

English

Home

Documentation

AMP

SSE Transport

Streamable HTTP Transport

Connecting to Multiple MCP Servers

MCP Security Considerations

Tools

Tools Overview

File & Document

Web Scraping & Browsing

Search & Research

Database & Data

Overview

MySQL RAG Search

PG RAG Search

Snowflake Search Tool

NL2SQL Tool

Qdrant Vector Search Tool

Weaviate Vector Search

MongoDB Vector Search

crewai

v1.15.1

English

Search...Ctrl K

Home

Documentation

AMP

API Reference

Examples

Changelog

SSE Transport

Streamable HTTP Transport

Connecting to Multiple MCP Servers

MCP Security Considerations

Tools

Tools Overview

File & Document

Overview

File Read

File Write

PDF RAG Search

DOCX RAG Search

MDX RAG Search

XML RAG Search

TXT RAG Search

JSON RAG Search

CSV RAG Search

Directory RAG Search

Integration: Boosts agent capabilities

Customizability: Provides the flexibility to meet the specific needs of agents.

Error Handling: Incorporates robust error handling mechanisms.

Caching Mechanism: Features intelligent caching to optimize performance.

Asynchronous Support: Handles both synchronous and asynchronous operations.

Typed Outputs: Uses optional Pydantic models to ensure structured and typed outputs, allowing agents to receive the tool's normal return value.

Using CrewAI Tools

To enhance your agents' capabilities with CrewAI tools, you can install the `crewai[tools]` package using pip:

```
pip install 'crewai[tools]'
```

Here's an example demonstrating their use:

Code

Ask a question...

© 2026 Dr. Ami M. Gates| All Rights Reserved | Free Distribution Only

SQL Tools – Using SQLite

RAG Example

SQLite

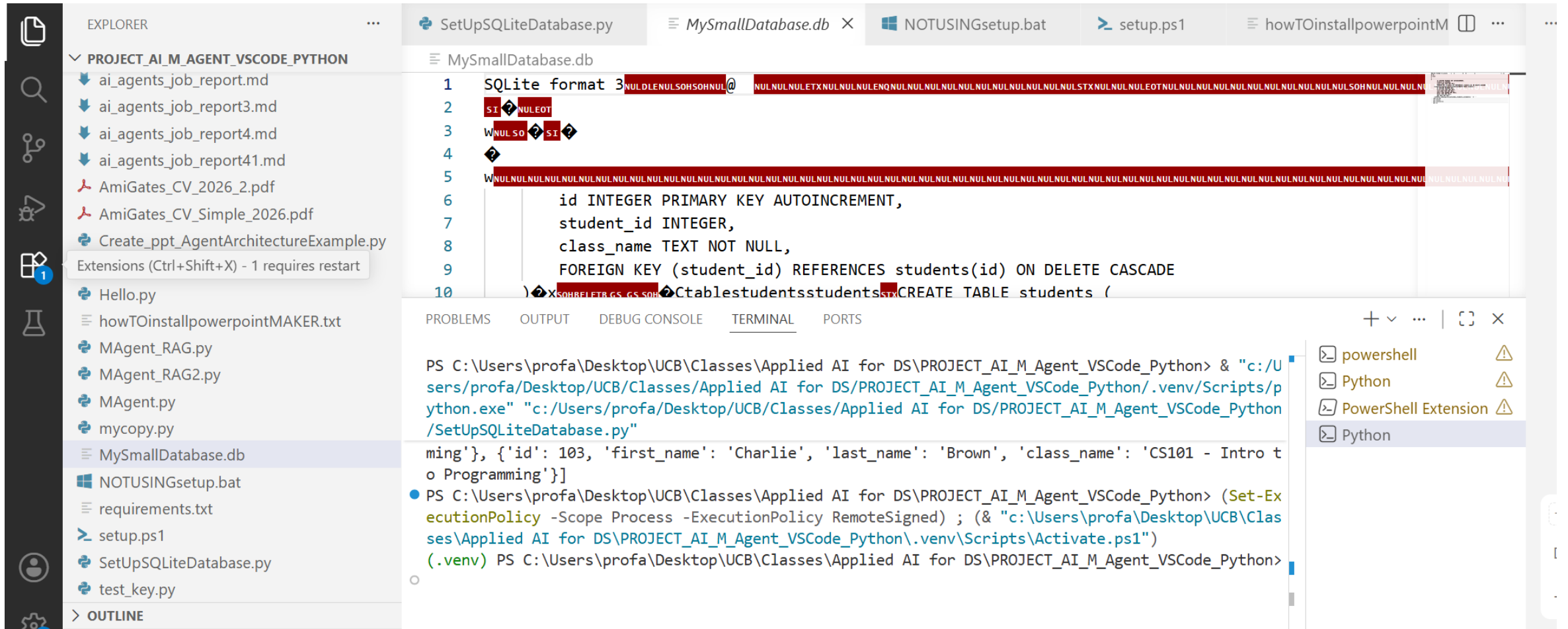
- Python has a built-in SQL database engine called **SQLite** that requires **zero installation**.
- It runs entirely out of a single local file (such as – for example - ***MySmallDatabase.db***) right inside the VS Code workspace.

Install the SQLite Viewer into VS Code

This will enable you to view the databases (.db) files we will create and work with. See these notes and the images on the next few slides.

Install the SQLite Viewer Extension

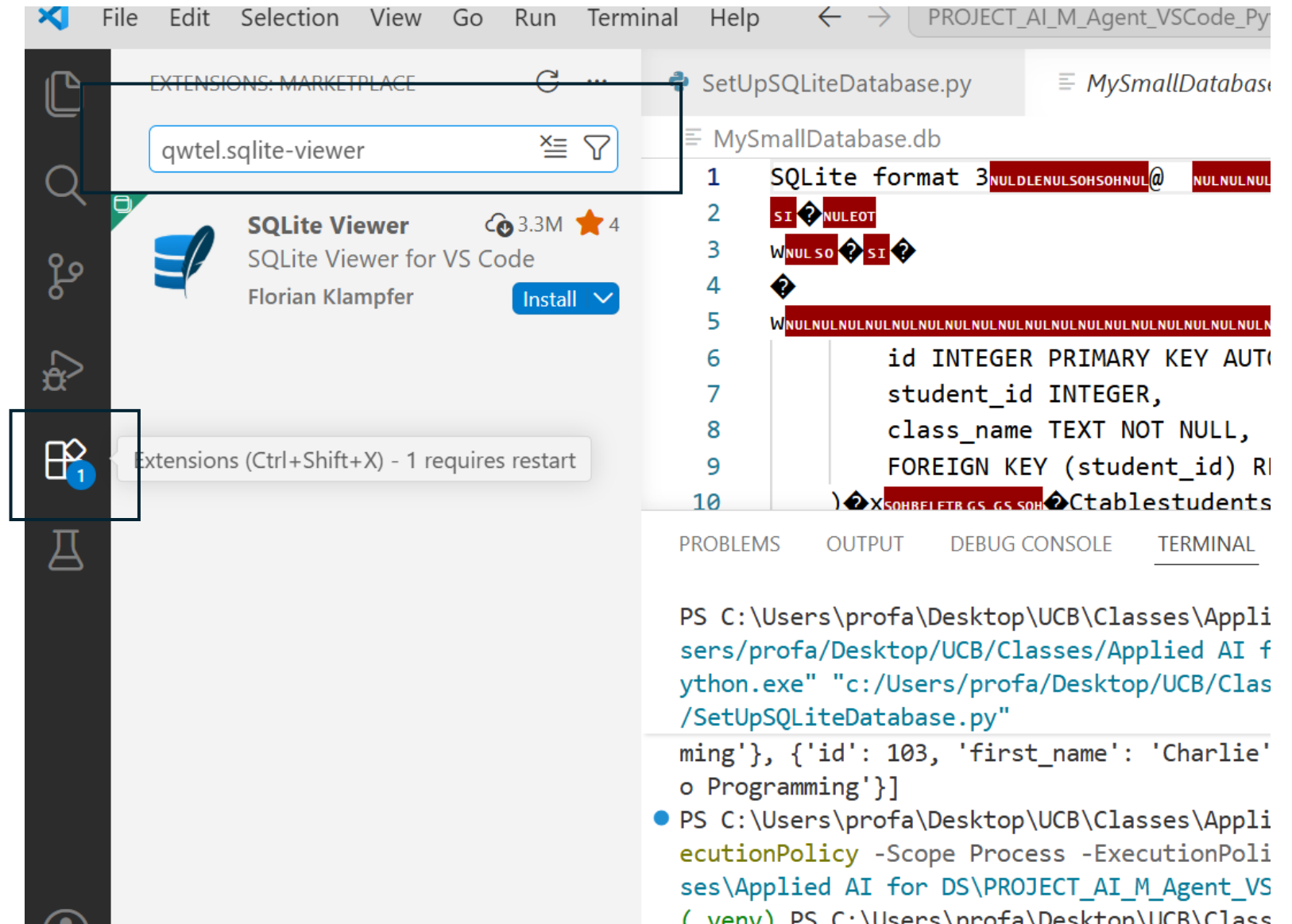
- Open the **Extensions** view in VS Code (click the square blocks icon on the left sidebar, or press Ctrl+Shift+X / Cmd+Shift+X).
- Type ***qwtel.sqlite-viewer*** into the search bar (this targets the popular **SQLite Viewer** extension by *qwtel*).
- Click **Install**



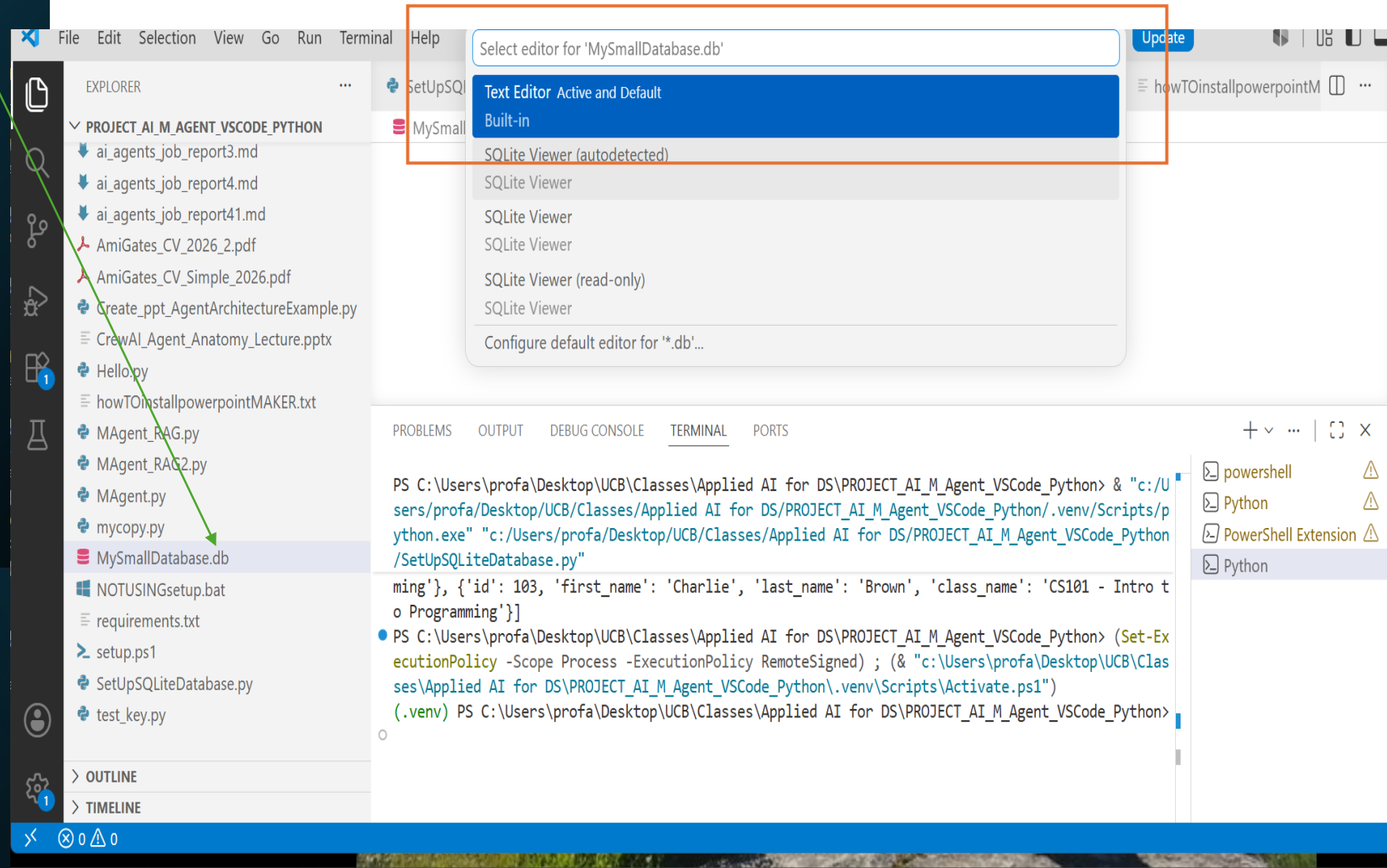
This is what things can look like
without the viewer.

© 2026 Dr. Ami M. Gates |
All Rights Reserved | Free
Distribution Only

Type
qwtel.sqlite-viewer
and install.

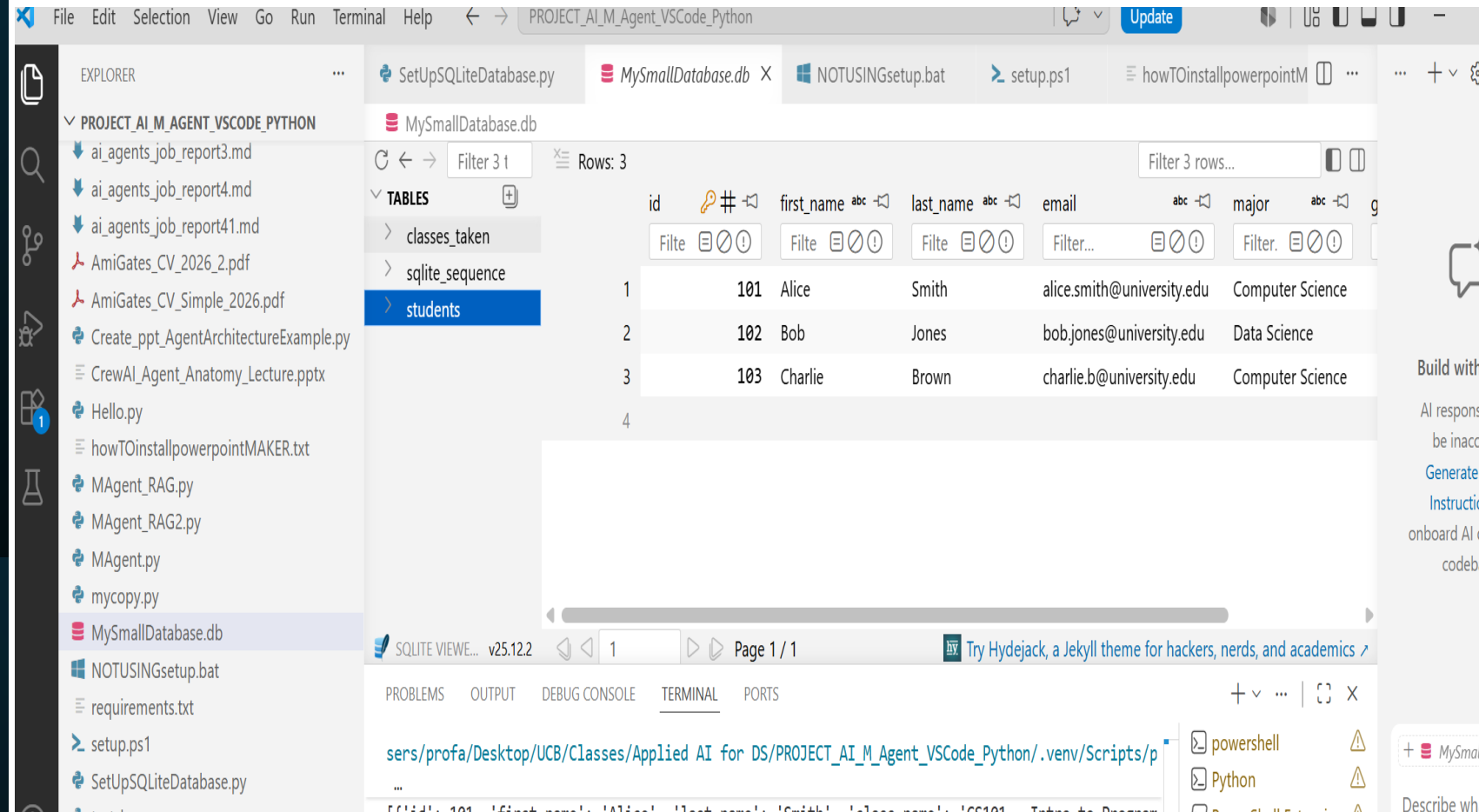


Now we have not created a .db file yet – but when we do, you can now click it and can view it by choosing the SQLite Viewer.



Now you can see a
“clean” database file.

Again, we have not
created the .db file yet –
that’s next!



The screenshot shows the Visual Studio Code interface with a SQLite database viewer open. The Explorer sidebar on the left shows the project structure for 'PROJECT_AI_M_AGENT_VSCODE_PYTHON'. The database viewer on the right shows a table named 'students' with 3 rows. The table has columns: id, first_name, last_name, email, and major. The data is as follows:

id	first_name	last_name	email	major
1	Alice	Smith	alice.smith@university.edu	Computer Science
2	Bob	Jones	bob.jones@university.edu	Data Science
3	Charlie	Brown	charlie.b@university.edu	Computer Science

The bottom of the screen shows the terminal with a PowerShell prompt, indicating the current directory is the project's virtual environment.

Deployment Launching Your Agent

Gates

Thinking About it and Researching Options

- 1) Whenever you learn or do something new, it helps to research options.
- 2) Ask AI some questions – like which might be more valuable in industry.
- 3) Think about your goal(s). Learn more first.
- 4) While we will consider many choices (to better understand) we will eventually use a combination of Render, GitHub, and Streamlit Python libraries.
- 5) The following MANY slides will go over options and then what is specifically needed to use Render, Git, and Streamlit to deploy our Project.

Our Goals in General

We now have a multi- agentic AI workflow that uses RAG.

It is coded using CrewAI, OpenAI, Python, and VSCode.

We want to deploy it “live” on the web so that anyone can use it.

BE CAREFUL – you will have to pay each time its used!

Things to Consider

- 1) **Environment Keys, Secrets, Passcodes, related.**
 - Never hardcode your OpenAI or Serper API keys (or any other API keys, codes, etc).
 - Use the platform's UI dashboard to inject them securely as environment variables.
- **Terminal vs. UI options – like Streamlit or Gradio UI**
 - CrewAI runs in the terminal by default. To make it a web app, you should wrap the execution loop in a lightweight framework like Streamlit so users can input prompts via a browser text box
- 1) **Server time-outs (for free options)**
 - Free tier hosting platforms often spin down (stall or pause) after 15 minutes of inactivity, or timeout during long-running multi-agent debates. You can think about how to optimize agent count and token limits to keep execution swift.

Organization How Are Things Defined?

- 1. Frontend Python Libraries**
(How the UI is Coded)
- 2. Frontend Cloud Hosting**
Platforms (Where the UI
Runs)
- 3. Multi-Agent Orchestration**
Libraries (How the Agents
are Coded)
- 4. Agentic Backend Cloud**
Platforms (Where the
Agents Execute)

Frontend Python Libraries (How the UI is Coded)

These are local open-source Python packages that you import into your script to build user interfaces (buttons, file uploaders, and graphs) using pure Python. They do **not** provide hosting on their own.

A Few Examples:

- **Streamlit Python Library:** Best for building clean, data-driven web dashboards in under 50 lines of code.
- **Gradio:** A direct alternative, heavily used in AI communities for rapid prototype deployment.
- **Dash (by Plotly):** A powerful alternative for building highly complex, production-grade analytical web applications

Frontend Cloud Hosting Platforms (Where the UI Runs)

These are **cloud-based computing platforms** where you deploy your frontend Python code (like your Streamlit app) so the public can access it via a web URL. [[1](#), [2](#)]

- **Render:** A general-purpose cloud provider. Its \$7/month tier hosts your Streamlit/Python application with guaranteed, always-on performance.
- **Streamlit Community Cloud:** Streamlit's own official, free hosting platform. (Excellent for basic student homework, but prone to sleep modes and resource limitations). [[1](#), [2](#), [3](#), [4](#), [5](#)] (AI references)
- **Hugging Face Spaces:** A massive, community-driven cloud hosting platform designed specifically to host open-source Gradio and Streamlit AI applications for the public.

Multi-Agent Orchestration Libraries (How the Agents are Coded)

These are **local Python development frameworks** you install on your machine to design agent logic, memory properties, and multi-tool RAG configurations. They do **not** include hosting.

- **CrewAI Framework:** Your active framework for defining specialized roles, sequential or hierarchical processes, and task dependencies.
- **LangChain / LangGraph:** A highly complex alternative framework utilized to build cyclic state-machine architectures.
- **Microsoft AutoGen:** An alternative framework specializing in multi-agent conversational structures.

Agentic Backend Cloud Platforms (Where the Agents Execute)

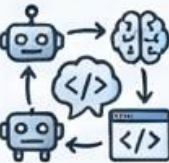
These are **specialized, long-running cloud server environments** built specifically to host raw multi-agent execution code. They run asynchronously as background APIs so that intensive agent web browsing and data scraping tasks never trigger web browser timeout crashes.

- **CrewAI Cloud:** The custom-built infrastructure environment designed to host your exact MAgent_RAG2.py agent script as a background API worker.
- **Langflow / Flowise:** Visual, drag-and-drop cloud backend execution engines for RAG and agent patterns.
- **AWS Lambda / Google Cloud Functions:** General-purpose serverless cloud backends (require manual setup for multi-agent workloads).

CrewAI Cloud handles the "Heavy Lifting" (Backend): <https://app.crewai.com>

- More On Multi_Agentic Systems with CrewAI:
<https://sachinsudheer.medium.com/developing-multi-agent-systems-using-crewai-a2f54e3be621>
- Multi-agent workflows are notorious for timing out on standard web servers because web agents can take 1 to 3 minutes to browse Google, parse PDFs, and scrape website catalogs.
- Standard free or low-tier servers (like Streamlit Community Cloud) will aggressively drop the connection after 30 seconds.
- **CrewAI Cloud** is custom-built to host crews as long-running, asynchronous background worker APIs that never time out

Architectural Stack Summary: Your Selected Toolset

COMPONENT LAYER	YOUR CHOSEN TOOL	FUNCTIONAL ROLE	EXECUTION ENVIRONMENT
 Frontend Framework	 Streamlit	Code that creates the upload buttons, text boxes, and visual dashboard for the user.	Runs in the user's web browser (locally or deployed).
 Frontend Web Host	 Render Render	Serves the public web address (e.g., <code>app.onrender.com</code>) that hosts the Streamlit UI.	Public Cloud Platform (Render Infrastructure).
 Agent Framework	 CrewAI	Code that defines the 3 specialized agents, their hierarchical processes, and RAG tools.	Runs as Python code (locally or within a container).
 Agentic Backend Host	 CrewAI Cloud	Executes the heavy, asynchronous agent RAG loops and data scraping tasks without timeouts.	Dedicated Backend Cloud Infrastructure (CrewAI Managed).

For MAgent_RAG2.py Project.

Render (\$7/month "Web Service") hosts the UI (Frontend): <https://render.com/>

- We will use Render's \$7/month "Starter" tier.
- Offers: **no spin-up delays** (no 5-minute loading screen for users waking up a free instance) and dedicated RAM.
- Users get a fast, beautiful, responsive interface that polls your CrewAI Cloud backend API for updates.

What is Render: <https://pangea.app/glossary/render>



Let's Deploy!

Streamlit Python Libraries – GitHub - Render

Step 1: Update the Code using Streamlit

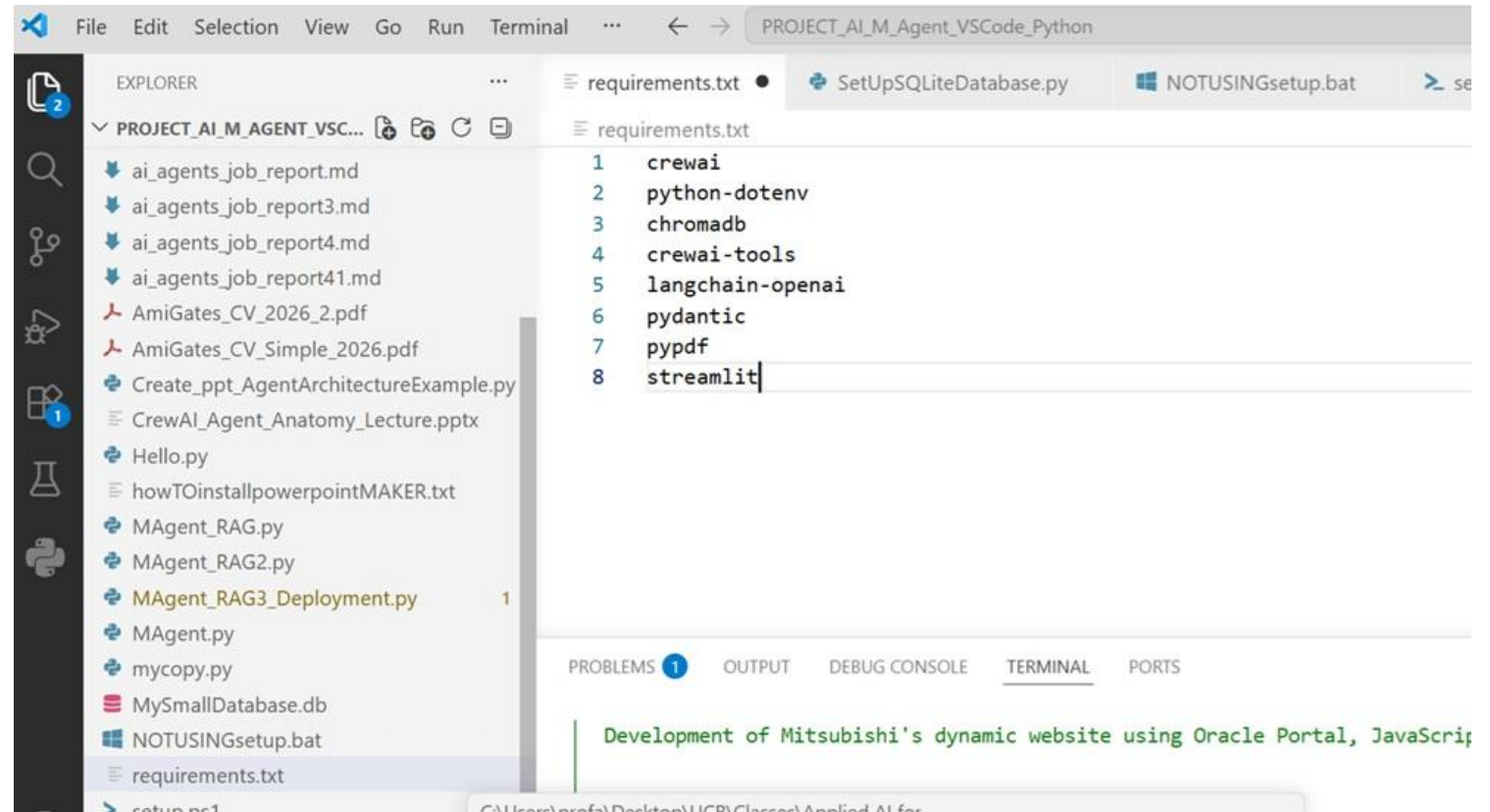
- 1) Update the Multi-Agentic Code using Streamlit. You will do this in VSCode.
- 2) You can then test the new code.
- 3) You will find that there are IMPORTANT and SPECIAL ways you will need to run the new code.

You can find
all the Code
(that uses
Streamlit)
here

<https://github.com/agates001/Al-agent-career-advisor>

- 1) Open the Project Link in Your Web Browser
- 2) Click the green `<>Code` button
- 3) Select Download Zip
- 4) Extract the folder, open it in VSCode, check and update any portions needed. Remember - this is my code.

Update requirements.txt to use Streamlit



After you update requirements.txt, then – re run it:
`pip install -r requirements.txt`

IMPORTANT:

Running Streamlit in VSCode/Python

Once your code is all ready, you will need to run/test it in VSCode.

Normally, when running Python code in VSCode, we can just type “python filename.py” or we can click “run”.

However, because we are using Streamlit now in our code, we cannot do this. Try it – see what happens.

Instead, we need to type:

`streamlit run filename.py`

into the venv terminal

After doing this, many things will happen.

- A Browser window will come up with your App. Right now, this is still local to your computer.
- You can ctrl (or cmd MAC) click the <http://localhost:8501/> to see your App. You can run it from there.

DO NOT FORGET that every time you run your MAgent System, it costs a few cents for OpenAI.

To stop it (stop the App), use ctrl C or cmd C.

Step 2: GitHub

At this point, you should have fully functional multi-agentic Python code that we wrote in VSCode.

You also have files such as your requirement.txt and your .env, etc.

Next, because we are using RAG for pdf and our resume/CV, you will need that file as well.

Be sure you have a github account (if not – get one now).

Next, we will review everything we need to get our files into git properly.

The next MANY slides take you through all the github steps.

GitHub – Pushing Files and Setting Up the Environment

1) Get inside github (log in).

2) Create the Cloud Repository on GitHub

- Open your web browser and go to your GitHub Dashboard.
- Click the green **New** button in the top-left area (or click the + icon in the top right and select *New repository*).
- Set your repository details:
 - **Repository name:** Type exactly – for example: AI-agent-career-advisor
 - **Public/Private:** Select **Public** (or Private—either works perfectly). I am using Public.
 - **Initialize this repository with: Leave every single checkbox completely blank** (Do *not* add a README, .gitignore, or license).
- Click the green **Create repository** button at the bottom.

You will see a setup screen filled with black code boxes. **Keep this browser tab open!**

- Look for your unique repository link in the text box at the top. It will look like this:
<https://github.com>

1 General

Owner *

 agates001

Repository name *

AI-agent-career-advisor

✓ AI-agent-career-advisor is available.

Great repository names are short and memorable. How about [fantastic-enigma](#)?

Description

An example multi-agent AI Workflow with RAG tools written in Python/VSC

71 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

 Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off ☐

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

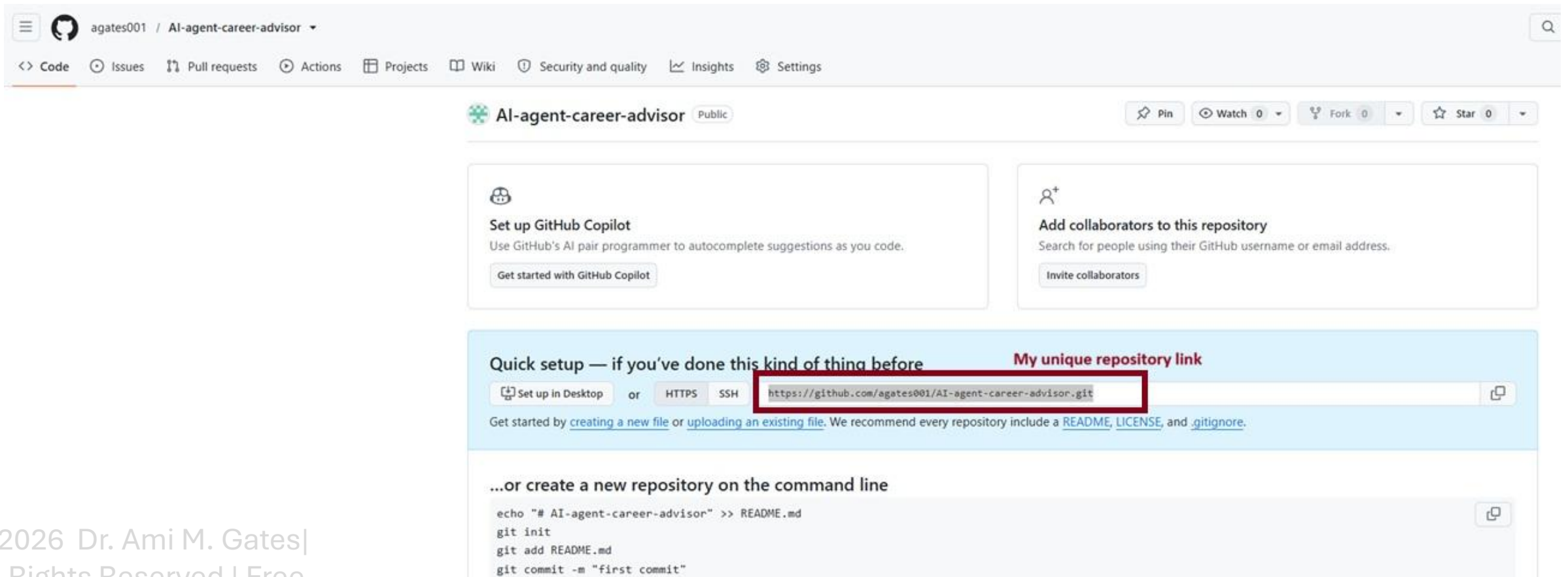
Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

Find Your Repo Link



agates001 / AI-agent-career-advisor

<> Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

AI-agent-career-advisor Public

Pin Watch 0 Fork 0 Star 0

Set up GitHub Copilot
Use GitHub's AI pair programmer to autocomplete suggestions as you code.
Get started with GitHub Copilot

Add collaborators to this repository
Search for people using their GitHub username or email address.
Invite collaborators

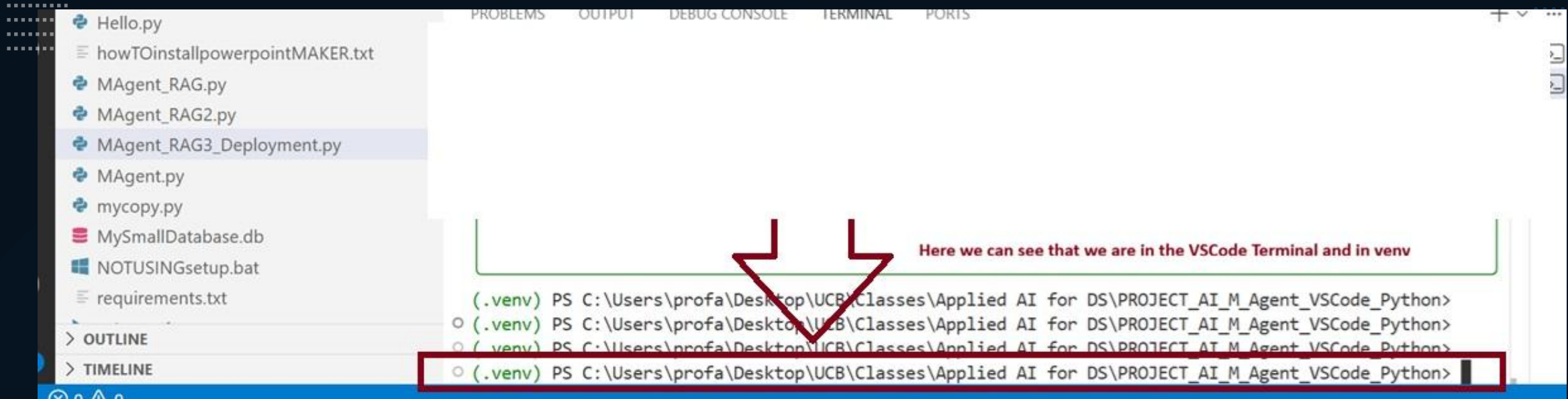
Quick setup — if you've done this kind of thing before **My unique repository link**

Set up in Desktop or HTTPS SSH <https://github.com/agates001/AI-agent-career-advisor.git>

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# AI-agent-career-advisor" >> README.md
git init
git add README.md
git commit -m "first commit"
```



Push Your VSCode Workspace to GitHub

Go back into **VSCode**. Make sure your terminal panel is open at the bottom and ensure your `(.venv)` environment marker is visible.

Push to Git – type each
of these into your
VSCode Terminal one
at a time and press
enter after each one.

READ THE NEXT SLIDE
FIRST _ BEFORE
DOING THIS...

1. Turn your project folder into a local Git tracker

git init

2. Stage your files (YourAgent.py, requirements.txt, and your PDF resume/CV) to be saved

git add .

3. Create a local snapshot save point of your workspace

git commit -m "Initial commit: Unified agent and streamlit app"

4. Set the primary branch name to main

git branch -M main

5. Link your local VSCode folder to your empty GitHub project online

NOTE: Remember to REPLACE the URL below with **your exact link** from your open GitHub browser tab

git remote add origin https://github.com

6. Push your files up to the GitHub cloud folder

git push -u origin main

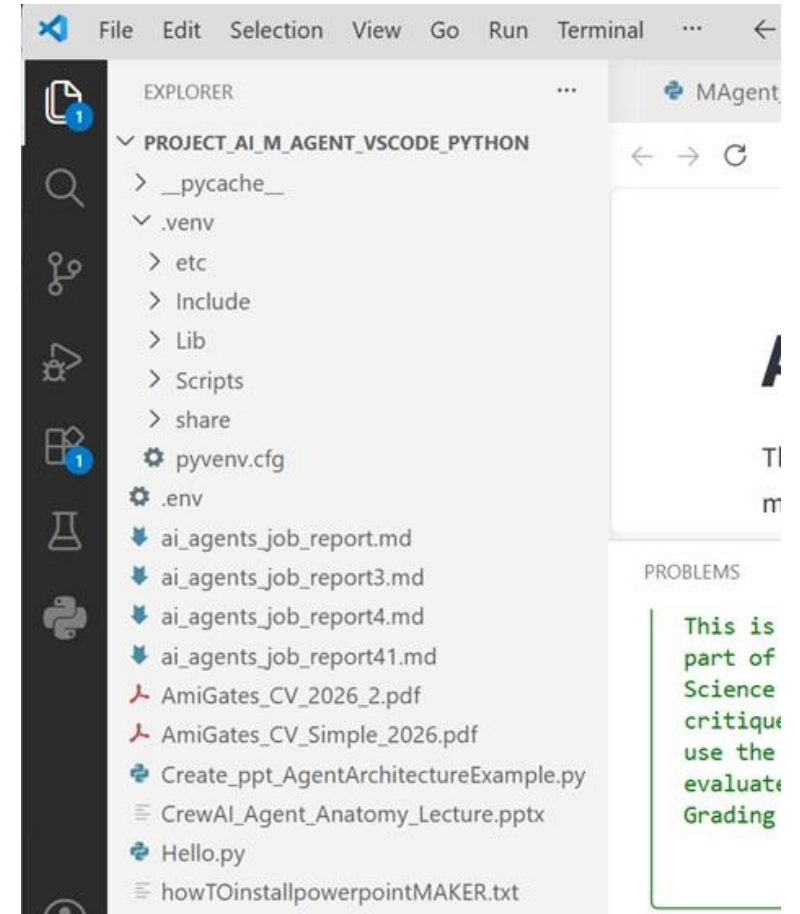
Push Only What You Need

Inside your AI Project you likely have many files and perhaps many versions of your Multi-Agentic Workflow Python file.

Only push to GitHub what is needed (so not only versions, not your env, etc.)

To the right, you can see all the goodies in my Project folder. However, I only plan to push a few items.

YOU have control over what you push to github.



Create a
.gitignore file in
your VSCode
Project Folder.
This is one way
to push all that is
not ignored.

1) Create a new file in your VSCode Project folder called **.gitignore**

The “.” needs to be there!

2) Into that file, put anything you want ignored.

3) Also – important – first create a new folder in your Project folder to hold any files you know will not be pushed. This way, your gitignore is small.

The next slide shows an example of mine.

Go Run Terminal ... PROJECT_AI_M_Agent_VSCode_Python Update

MAgent_RAG3_Deployment.py ~~AmiGates_CV_Simple_2026.pdf~~ AI Agent Career Advisor .gitignore

```
1 # Ignore these items - do not push to git. This is example and is not needed
2 # because I instead created a folder in this Project to hold all older or other files
3 # that will not be pushed. So just FYI on how you would do this.
4 *.md
5 # The above will ignore any file that ends in .md
6 # Also ignore these
7 .env
8 # It is critical NOT to push your passcodes to git!
9 .venv/
10 __pycache__/
11 # Note: the "/" tells it to ignore the entire folder and everything in it (like other fo
12 # Ignore the folder created to house any other stuff. Mine is called Indirect_Project_Fi
13 Indirect_Project_Files/
```

Now – Push to
GitHub
Do these one at
a time....
Remember to
use **YOUR**
GitHub URL

```
git config --global credential.helper manager
git init
git add .
git commit -m "Initial commit: Multi Agent
with RAG Python Workflow Example 1"
git branch -M main
git remote add origin
https://github.com/agates001/AI-agent-
career-advisor.git
git push -u origin main
```


Bugs

- Sometimes this process can be odd.
- Its OK to ask AI if you get stuck
- The following slide shows my exact steps and an image of what pops up for Git.

```
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python> git config --global credential.helper manager
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python> git init
Reinitialized existing Git repository in C:/Users/profa/Desktop/UCB/Classes/Applied AI for DS/PROJECT_AI_M_Agent_VSCode_Python/.git/
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python> git add .
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python> git commit -m "Initial commit: Multi Agent with
RAG Python Workflow Example 1"
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python> git branch -M main
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python> git remote add origin
https://github.com/agates001/AI-agent-career-advisor.git
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python> git push -u origin main
info: please complete authentication in your browser...
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python> git push -u origin main
```

Enumerating objects: 7, done.

Counting objects: 100% (7/7), done.

Delta compression using up to 8 threads

Compressing objects: 100% (7/7), done.

Writing objects: 100% (7/7), 371.29 KiB | 20.63 MiB/s, done.

Total 7 (delta 0), reused 0 (delta 0), pack-reused 0

To https://github.com/agates001/AI-agent-career-advisor.git

* [new branch] main -> main

branch 'main' set up to track 'origin/main'.

(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for DS\PROJECT_AI_M_Agent_VSCode_Python>

© 2026 Dr. Ami M. Gates |
All Rights Reserved | Free
Distribution Only

The screenshot shows the Visual Studio Code interface. The Explorer panel on the left displays the project structure for 'PROJECT_AI_M_AGENT_VSCODE_PYTHON', including files like 'pydeck.json', 'pyvenv.cfg', '.env', '.gitignore', 'AmiGates_CV_2026_2.pdf', 'MAgent_RAG3_Deployment.py', 'requirements.txt', and 'setup.ps1'. The central editor shows the 'MAgent_RAG3_Deployment.py' file with a code snippet:

```
72 JobHunter = Agent(  
73     role="DS Job Hunter",  
74     goal="Find current and open jobs in
```

The integrated terminal at the bottom shows the following commands and output:

```
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI  
config --global credential.helper manager  
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI  
init  
Reinitialized existing Git repository in C:/Users/profa/D  
AI_M_Agent_VSCode_Python/.git/  
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI  
add .  
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI  
commit -m "Initial commit: Multi Agent with RAG Python Wo  
On branch main  
nothing to commit, working tree clean  
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI  
branch -M main  
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI  
remote add origin https://github.com/agates001/AI-agent-career-advisor.git
```

A 'Connect to GitHub' dialog box is open on the right, titled 'GitHub Sign in'. It offers two options: 'Browser/Device' and 'Token'. The 'Sign in with your browser' button is highlighted with a red star. Below it, the 'Sign in with a code' button is also highlighted with a red star. At the bottom of the dialog, it says 'Don't have an account? Sign Up'.

Example of GitHub After Successful Push

 agates001 / AI-agent-career-advisor

agates001 / AI-agent-career-advisor

Type / to search











[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Wiki](#) [Security and quality](#) [Insights](#) [Settings](#)

 **AI-agent-career-advisor** Public

 Pin  Watch 0  Fork 0  Star 0

 main  1 Branch  0 Tags

Go to file

Add file

Code

 **agates001** Initial commit: Organized multi-agent web application f173a3a · 17 minutes ago 1 Commit

 .gitignore	Initial commit: Organized multi-agent web application	17 minutes ago
 AmiGates_CV_2026_2.pdf	Initial commit: Organized multi-agent web application	17 minutes ago
 MAgent_RAG3_Deployment.py	Initial commit: Organized multi-agent web application	17 minutes ago
 requirements.txt	Initial commit: Organized multi-agent web application	17 minutes ago
 setup.ps1	Initial commit: Organized multi-agent web application	17 minutes ago

About

An example multi-agent AI Workflow with RAG tools written in Python/VSC

 Activity

 0 stars

 0 watching

 0 forks

Releases

No releases published

[Create a new release](#)

Step 3: Render

Get a Render Account

The Sign-Up: Go to render.com and create an account by clicking “Start for Free”.

- For the smoothest workflow, **sign up using your GitHub account**. This allows Render to instantly see your code repositories later without complex SSH key configurations.
- Next, choose “Web Services”

Billing: To unlock the **\$7/month Starter Web Service tier**, you will need to add a credit card under your account billing settings. Render charges on a predictable monthly billing cycle.

Why it's needed now: Render requires a valid card on file to provision dedicated computing instances and to prevent platform abuse from automated spam bots.



Create a new Web Service

1 Choose service > 2 Configure > 3 Deploy

📄 Static Sites

Static content served over a global CDN. Ideal for frontend, blogs, and content sites.

[New Static Site →](#)

🌐 Web Services

Dynamic web app. Ideal for full-stack apps, API servers, and mobile backends.

[New Web Service →](#)

🔒 Private Services

Web app hosted on a private server. Ideal for apps that require a private network or access to other Render services.

[New Private Service →](#)

🕒 Cron Jobs

Short-lived tasks that run on a periodic schedule.

🗄️ Postgres

Relational data storage. Supports point-in-time recovery, read replicas, and high availability.

📦 Key Value

Managed Redis®-compatible shared cache, message broker, or stream processor.



M My Workspace

🌐 New Web Service

🔍 Search ^ K

Configure and deploy your new Web Service



Choose service



Configure



Deploy

Source Code

Git Provider

Public Git Repository

Existing Image

Connect Git provider

Connect your Git provider to deploy from your existing repositories.



GitHub



GitLab



Bitbucket

Next Steps

Render will pop up an “Install Render” window.

Choose “Only select repositories and choose your AI Agent repo.

Then, install.

Once authorized, the Render page will refresh, and you will see your **AI-agent-career-advisor** repository sitting at the top of your Render list.

Click the **Connect** button next to it.

The screenshot shows the 'Install Render' window. At the top, it says 'Install on your personal account agates001'. Below this, it asks 'for these repositories:'. There are two radio button options: 'All repositories' and 'Only select repositories'. The 'Only select repositories' option is selected and highlighted with a red box. Below the options, there is a button labeled 'Select repositories'. Underneath, it says 'Selected 1 repository:' and shows a list with 'agates001/AI-agent-career-advisor' and a close button 'X'. Below this, it says 'with these permissions:' and lists two permissions: 'Read access to Dependabot alerts, code, and metadata' and 'Read and write access to actions, checks, commit statuses, deployments, environments, issues, pull requests, repository hooks, and workflows'. At the bottom, there is a section for 'User permissions' which states 'Render can also request users' permission to the following resources. These permissions will be requested and authorized on an individual-user basis.' and lists 'Read access to email addresses'. At the very bottom, there are two buttons: 'Install' (highlighted with a red box) and 'Cancel'. Below the buttons, it says 'Next: you'll be directed to the GitHub App's site to complete setup.'



M

My Workspace



New Web Service

Search



Configure and deploy your new Web Service



Choose service



Configure



Deploy

Click on the repo name.
When you do, other fields
will com up.

Source Code

Git Provider

Public Git Repository

Existing Image



Search



agates001 / AI-agent-career-advisor 11h ago

Steps to Complete the Render Set Up

- The following several slides will show images of what it looks like and what you need to type to get Render set up.
- REMEMBER – some items need to be changed to YOUR names/stuff.
- You will also need a credit card to do this.



M My Workspace

New Web Service

Configure and deploy your new Web Service



Choose service



Configure



Deploy

It looks like you're using **Python**, so we've autofilled some fields accordingly.

Source Code



agates001 / AI-agent-career-advisor • 11h ago

Name

A unique name for your web service.

AI-agent-career-advisor-ui

Language

Choose the **runtime environment** for this service.

Python 3

Branch

The Git branch to build and deploy.

 main

Region

Your services in the same [region](#) can communicate over a [private network](#).

Ohio (US East)

Root Directory Optional

If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a [monorepo](#).

e.g. src

Build Command

Render runs this command to build your app before each deploy.

```
$ pip install -r requirements.txt
```

Start Command

Render runs this command to start your app with each deploy.

```
$ streamlit run MAgent_RAG3_Deployment.py --server.port $PORT --server.address 0.0.0.0
```

Instance Type

For hobby projects

Free	512 MB (RAM)
\$0 / month	0.1 CPU

For professional use

For more power and to get the most out of Render, we recommend using one of our paid instance types. All paid instances support:

- Zero Downtime
- SSH Access
- Scaling
- One-off jobs
- Support for persistent disks

Starter	512 MB (RAM)
\$7 / month	0.5 CPU

Standard	2 GB (RAM)
\$25 / month	1 CPU

Pro	4 GB (RAM)
\$85 / month	2 CPU

Pro Plus	8 GB (RAM)
\$175 / month	4 CPU

Pro Max	16 GB (RAM)
\$225 / month	4 CPU

Pro Ultra	32 GB (RAM)
\$450 / month	8 CPU



M My Workspace ↕

🌐 New Web Service ▾

Environment Variables

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)

OPENAI_API_KEY

.....

SERPER_API_KEY

.....

OPENAI_MODEL_NAME

gpt-4o-mini

+ Add Environment Variable

📄 Add from .env

> **Advanced**

Deploy Web Service

My Workspace

← Environment

AI-agent-career-advisor-ui

Deploys

Settings

MONITOR

Events

Logs

Metrics

MANAGE

Environment

Changelog

Invite a friend

Contact support

Render Status

My project / Production / AI-agent-career-advisor-ui

WEB SERVICE

Al-agent-career-advisor-ui

Python 3 Starter

Service ID: `srv-d9hm75ernols73a8e0mg`

agates001 / AI-agent-career-advisor main

<https://ai-agent-career-advisor-ui.onrender.com>

Initial commit: Organized multi-agent web application

Deployed on July 24, 2026 at 7:07:36 AM MDT

dep-d9hm766rnols73a8e19g

Status

Building

Source

f173a3a

All logs

Search logs

Jul 24

07:07:45 AM

Getting requirements to build wheel: started

07:07:45 AM

Getting requirements to build wheel: finished with status 'done'

© 2026 Dr. Ami M. Gates | All Rights Reserved | Free Distribution Only

Why \$PORT and why 0.0.0.0

Why we use \$PORT

- Render dynamically allocates a new network port number every time your application container spins up or reboots. By writing `--server.port $PORT`, you are instructing Streamlit to read Render's internal system variable automatically and bind itself to whichever port Render opens for it.

Why we use 0.0.0.0

- In networking, setting an address to 0.0.0.0 is a universal command that tells the web server to listen to **all incoming external network requests** from the public internet, rather than locking itself down to your private local machine. This is what allows your students to access your website from their own devices at home.

Why This Fails

- When you deploy, you will see that it eventually fails.
- It is important to see this and to understand why.

In this case, recall that we are using specific versions of Python (3.12 in our case) because CrewAI and many other Agent coding elements are not yet ready for Python 3.13 or higher.

- Therefore, this causes a cascade of errors because Render is assuming Python 3.13 (the current version at the time of these slides) and not the required version we must use.

How to Fix it

Double check that you are using 3.12 in your VSCode Project. This Tutorial specifies 3.12.x so you should be using 3.12.x.

To Check:

```
(.venv) PS C:\Users\profa\Desktop\UCB\Classes\Applied AI for  
DS\PROJECT_AI_M_Agent_VSCode_Python> python --version
```

Python 3.12.10

Now, make updates in Render: IMPORTANT

- 1) You should be looking at the failed deploy log and if you look at the URL it should start with `dashboard.render.com/...` This means you are in Render dashboard area. If you are not – get there first.
- 2) On the left-hand sidebar navigation for “Environment” and click it. Once there you will see “Environment Variables”.
- 3) Click edit on the right and then click the + Add Environment Variable button.
- 4) Click Save, Rebuild, and Deploy. This will go back to try to render/deploy your app again

Now Your App Should be LIVE!

The screenshot displays the Render dashboard for a project named 'AI-agent-career-advisor-ui'. The left sidebar contains navigation options: Environment, Deploys, Settings, MONITOR (Events, Logs, Metrics), and MANAGE (Environment, Shell, Changelog, Invite a friend, Contact support, Render Status). The main area shows the deployment status as 'Live' with a source ID of 'f173a3a' and a manual trigger. Below this, a log viewer shows the deployment process. Two specific log entries are highlighted with red boxes: 'Your service is live' and 'Available at your primary URL https://ai-agent-career-advisor-ui.onrender.com'. At the bottom right, a note suggests using the Render CLI, MCP Server, or log stream integration for better log management.

My Workspace

My project / Production / AI-agent-career-advisor-ui

Status: ✓ Live Source: f173a3a Trigger: Manual

All logs Search logs

Jul 24

07:31:23 AM [h75nq]

07:31:24 AM [h75nq] 2026-07-24 13:31:24.188 Uvicorn server started on 0.0.0.0:10000

07:31:24 AM [h75nq]

07:31:24 AM [h75nq] You can now view your Streamlit app in your browser.

07:31:24 AM [h75nq]

07:31:24 AM [h75nq] Local URL: <http://localhost:10000>

07:31:24 AM [h75nq] Network URL: <http://10.24.235.205:10000>

07:31:24 AM [h75nq] External URL: <http://74.220.50.220:10000>

07:31:32 AM [h75nq] **--> Your service is live 🎉**

07:31:32 AM [h75nq] **--> Available at your primary URL <https://ai-agent-career-advisor-ui.onrender.com>**

07:31:33 AM [h75nq]

07:31:33 AM [h75nq]

Need better ways to work with logs? Try the [Render CLI](#), [Render MCP Server](#), or set up a [log stream integration](#)

Your App is Live!

ai-agent-career-advisor-ui.onrender.com



Multi-Agent Job & Course Advisor

This application runs a live multi-agent workflow to search for jobs, scan your resume, and recommend matching CU Boulder courses.



Run AI Agent Analysis

A Note About Costs

1. Render Fee (The Server)

- **Cost:** Flat \$7/month.
- **How it works:** You are paying for the physical server space in Render's data center to keep your website address alive 24/7. This price is fixed. It does not change based on how many people visit your website or how hard your agents work.

2. OpenAI & Serper Fees (The Agent API Keys)

- **Cost:** Pay-as-you-go (per analysis run).
- **How it works:** These costs only happen **when someone actually clicks the "Run" button** on your live site.
- **The math:** Running your specific crew on a resume costs roughly **\$0.02 to \$0.05 per click** in OpenAI tokens (gpt-4o-mini is incredibly cheap) and consumes exactly 1 free search credit on Serper. If the button is not clicked, your API token usage is exactly zero.

The Updated Code

Gates

You can find
all the Code
here

<https://github.com/agates001/Al-agent-career-advisor>

- 1) Open the Project Link in Your Web Browser
- 2) Click the green `**"<>Code"*` button
- 3) Select Download Zip
- 4) Extract the folder, open it in VSCode, check and update any portions needed. Remember - this is my code.

Evaluations

Ground Truth and Evals

Evaluation Common Options

Option 1: Code a Basic "LLM-as-a-Judge" Eval

Here, we can write a short, lightweight Python script that reads the final Markdown report your crew generates, hands it to a separate GPT-4o-mini judge, and scores it from 1 to 5 on metrics like *Hallucination* and *Relevance*.

Option 2: Explore CrewAI's Built-In Testing Tools

Use/apply the official terminal commands (crewai test and crewai train) to see how the framework natively measures agent performance.

Option 3: Study Industry Metrics (Ragas & DeepEval)

Formal industry metrics like *Context Precision* and *Faithfulness*.

Heuristic / Deterministic Evals (Code- Based)

- **How it works:** One writes a traditional Python Program (if/else statements, string matching, or Regular Expressions) to check the text output.
- **For Example:** Depending on the expected output, one can check for certain words or phrases. Checking if the output contains the string like "Python Programming", or ensuring the agent output is valid Markdown by checking for ### headers.
- **Pros:** 100% free, runs instantly (milliseconds), and completely predictable.
- **Cons:** Completely blind to meaning. If an agent writes a beautiful, accurate report but misses a specific keyword, a heuristic test will fail it. It cannot grade nuance, tone, or helpfulness.

LLM-as-a-Judge (Automated Semantic Evals) — *INDUSTRY PREFERRED*

- **How it works:** You take the agent's output and feed it to a powerful model (like GPT-4o-mini) with a strict, specialized grading rubric prompt.

Tutorials:

<https://www.linkedin.com/pulse/what-evals-everything-traditional-experimenters-need-know-longden-lbbte>

<https://aishwaryasrinivasan.substack.com/p/ai-evals-explained-for-builders>

- **For Example:** *"You are an expert academic evaluator. Read this candidate's resume and this agent's course recommendation. On a scale of 1-5, score the 'Faithfulness' of the recommendation. Give a 1 if the agent recommended a course the student has already taken."*
- **Pros:** Unbelievably fast compared to humans, highly scalable, and can understand deep semantic meaning, context, and nuance.
- **Cons:** Costs API tokens to run the judge model, and the judge itself can occasionally be inconsistent (though this is mitigated by using strict system prompts and low temperature settings).

Human-in-the-Loop (HITL)

- **How it works:** A human expert sits down, reads the agent's report, and manually scores it or provides a thumbs-up/thumbs-down.
- **Pros:** The ultimate "ground truth." Humans are still the best at spotting subtle logical flaws or evaluating real-world helpfulness.
- **Cons:** The ultimate bottleneck. It is incredibly slow, expensive, does not scale, and introduces human fatigue and bias.

Which One Does Industry Prefer and Why? (According to AI 😊)

- The industry overwhelmingly prefers **LLM-as-a-Judge** for production development.
- When engineers change a prompt, swap an underlying LLM, or tweak an agent's tools, they need to know *instantly* if that change broke anything across 100 different test scenarios. Waiting for humans to review 100 reports takes days. Running heuristic keyword checks is too dumb.
- An **LLM-as-a-Judge** can evaluate all 100 scenarios in 30 seconds for less than a dollar, giving the engineering team a concrete, mathematical average score (e.g., *"Your prompt tweak increased overall Faithfulness from 72% to 89%"*).

Grader Type	Anthropic's Definition	How We Apply This to Your Project
Code-Based <i>(Deterministic)</i>	Fast, cheap, objective checks like regex, string matching, or state changes.	Duplicate Detection: Use pure Python to cross-reference the report's course recommendations against the input resume string to ensure a 100% fail-proof check against recommending a class the student already took.
Model-Based <i>(LLM-as-a-Judge)</i>	Flexible, rubric-based scoring that captures semantic nuance and handles open-ended output.	Relevance & Faithfulness Scoring: Use a second isolated LLM to read the report and score it on a 1–5 scale to verify the recommendations are grounded in the actual CU Boulder catalog links.
Human Graders	Gold-standard quality used primarily to calibrate and validate your model-based judges.	Expert Calibration: You, acting as the professor/subject matter expert, spot-check 5% of the automated grading outputs to calibrate the LLM judge's accuracy.

Use Python to Build an LLM-As-a-Judge

Suppose we want to create a Python Program that will evaluate our Multi-Agent with RAG Application that we now have running live via Render.

There are many options for evals.

Let's create one that “can” run when (and only if) we want it to. This will save us space and money. It will also help us to learn more about evals.

Here are the overall goals:

- 1) Our main Multi-Agent/RAG application creates a report. It does this already.
- 2) We want to create code that we can deploy live and (via a button) can *choose* to run it or not to gain insight into how the Multi-Agent/RAG application is doing. In other words – to evaluate it (and then make it better, etc.)
- 3) Our Eval Python Code will inspect and audit out Multi-Agent/RAG application output as an OPTION.

File-Based Inspection Pipeline

- Our MAgent/RAG App creates an “.md” (markdown) output.
- Therefore, the Eval Program will need to access this output. It will also need access to our CV/Resume.
- We will need to update (add to and re-push to github) a more extensive requirements.txt file that contains all the elements on the right.
Let's do that first.

--- Core Application Stack ---

streamlit

crewai

crewai-tools

langchain-openai

pydantic

python-dotenv

chromadb

pypdf

--- Local & Live Evaluation Suite ---

deepeval

pytest

nest-asyncio

pytest-asyncio

pandas

openpyxl

Update requirements.txt in VSCode and Git

- 1) Make sure your requirements.txt file in your VSCode Project area contains all of the elements as noted on the previous slide.
- 2) Push the changes (updated requirements.txt) into github by running these three Git commands in your VSCode terminal. Note that in this example, the main branch is called “main”. If you named your main branch differently, you will need to adjust.

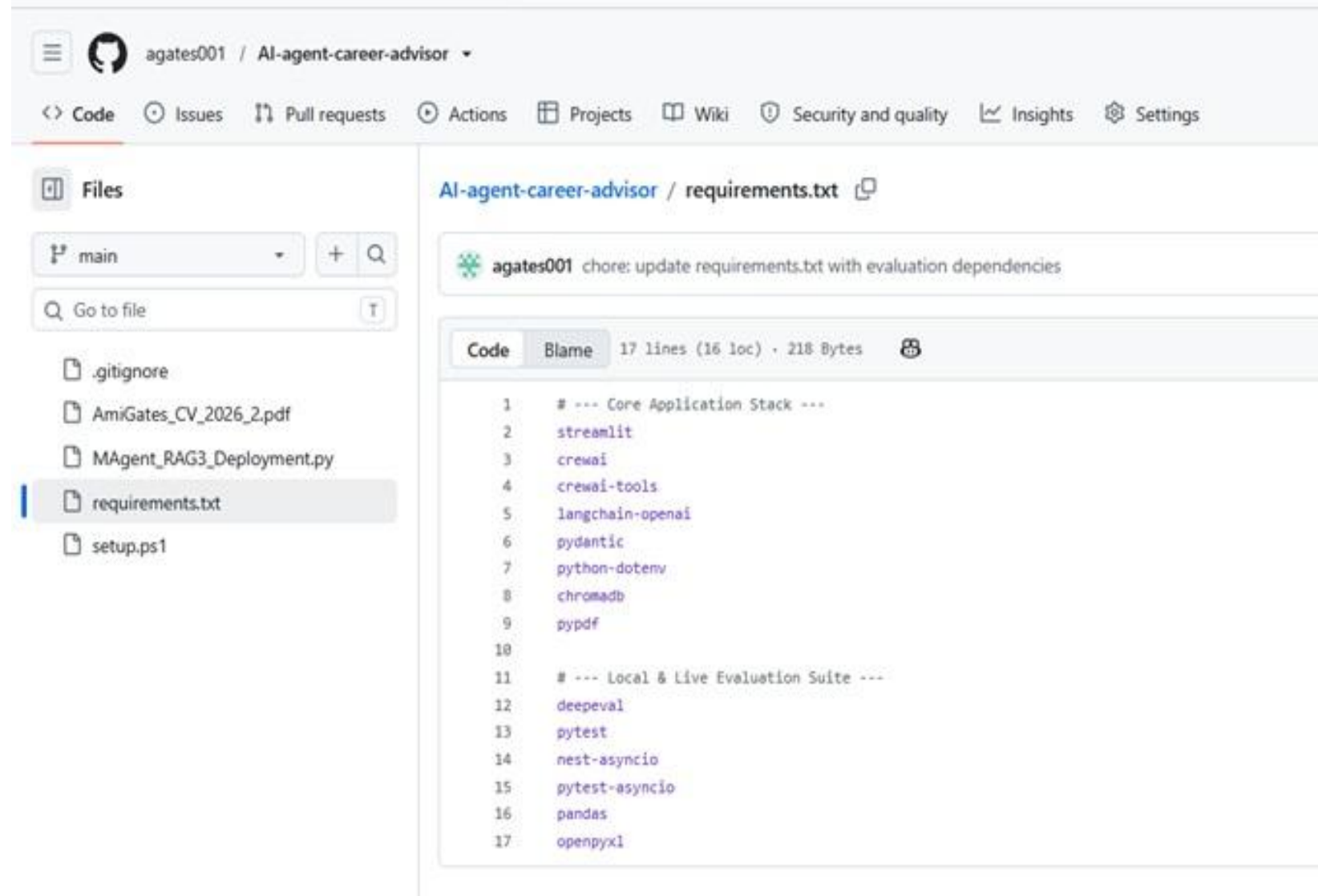
git add requirements.txt

git commit -m "chore: update requirements.txt with evaluation dependencies"

git push origin main

PORTS

We can Confirm the Push in Git



agates001 / Al-agent-career-advisor

Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

Files

main + 🔍

Go to file T

- .gitignore
- AmiGates_CV_2026_2.pdf
- MAgent_RAG3_Deployment.py
- requirements.txt**
- setup.ps1

Al-agent-career-advisor / requirements.txt

agates001 chore: update requirements.txt with evaluation dependencies

Code Blame 17 lines (16 loc) · 218 Bytes

```
1 # --- Core Application Stack ---
2 streamlit
3 crewai
4 crewai-tools
5 langchain-openai
6 pydantic
7 python-dotenv
8 chromadb
9 pypdf
10
11 # --- Local & Live Evaluation Suite ---
12 deepeval
13 pytest
14 nest-asyncio
15 pytest-asyncio
16 pandas
17 openpyxl
```


Next Steps

- 1) We have now updated requirements.txt and pushed the new version of it to Git.
- 2) Next, we will write and push to Git the Eval Program.
- 3) We will write the Eval Project in Python in VSCode.
- 4) Then, we will test/run it “locally” to assure it is working.
- 5) Once working, we will push it to GitHub and Render it live.

Here is the Code for the Eval

Mine is one Git right now.

https://github.com/agates001/Al-agent-career-advisor/blob/main/Indirect_Project_Files/Eval_Code_Audit_MAgent_RAG.py

HOWEVER – yours is not yet on Git. You will write it locally in VSCode and then you will run it using THIS COMMAND in the VSCode terminal:

streamlit run Eval_Code_Audit_MAgent_RAG.py --server.port 8502

Why?

You need to use streamlit to render this locally and to see how it will work. Next, to avoid conflicting with your other App, choose a server port.

What Problems Will We Face

- 1) We want to run our Multi-Agent App live and enable the Eval App (which will be a “choice”) to access the output.
- 2) BUT – we cannot do this with GitHub or with local files.
- 3) Why?
 - 1) When the Multi-Agent App runs, it will create a Markdown file as output (just like it did in our VSCode locally). However, it cannot “push” that file back to our GitHub.
 - 2) GitHub is a **static repo, not a live database**. So, Live Apps do not have write-access and cannot modify the repo during runtime. (Only we can push files to our Git).
 - 3) So, even though the two Apps (the Multi-Agent and the Eval) are ours, they cannot “see” or engage with each other using our current methods.
- 4) **So – what can we do?**

Apps that Chat – Shared Database

- 1) To allow our Apps (the MAgent and the Eval) to interact on Render, we need a lightweight, free, cloud database.
 - 1) Options: Supabase, ElephantXQL/PostgreSQL, etc.
- 2) Instead of writing output to a markdown file, the apps will chat via a cloud database.
- 3) To do this, we will need to make updates to the code. We will change things like:
 - 1) Instead of explicitly writing the output to a .md file, we will use the final output (result.raw) and place it in a free database table.
 - 2) We will also place all the text from the resume we use into a database table.
 - 3) Next, the Eval App will “call” (via secure API call) that same database, pull text, and use the OpenAI judge (in our case via gpt-4o-mini) to run the audit.
- 4) The database is 24/7, synch, and live. It is a non-filessharing bridge between the apps. It does not use or require any form of file sharing and does not violate any container isolation.

Getting the Free Database

- 1) There are always many options for everything we do.
- 2) We will use Supabase which provides a free PostgreSQL database that acts as a secure bridge between our applications (their “containers”).

<https://supabase.com>

Our Next Steps:

- 1) Set up the database
- 2) Update our Multi-Agent RAG App so that it sends its output to the database rather than to the Markdown. My suggestion here is to create a copy of the current App (Python Code/Workflow) that is in VSCode, and make all updates to the copy. Rename it something smart like MAgent_RAG_LiveEvals.py.
- 3) Update the Eval App (in VSCode) to grab the information (the output) from the database. Here again, I suggest you make a copy, rename, and make updates in the copy.
- 4) We will need to do this, test it in VSCode, and then push it to GitHub.

Why are we using Supabase instead of MongoDB?

1. Simple, Native Pydantic Integration

Supabase is built directly on PostgreSQL, a relational database. It stores data in clean rows and columns. Because both our MAgent App and our Eval App rely heavily on **Pydantic** models (which expect structured data rows), mapping text “payloads” into a **Postgres table** takes just two lines of clean Python code. MongoDB uses a nested JSON document structure, which requires an extra object-document mapping step (like using Motor or Beanie) to clean up and validate your data before Python can read it. [\[1, 2, 3, 4, 5\]](#) (**AI Resources**)

2. Built-in, Fast PostgREST API

When you create a project in Supabase, it instantly auto-generates a secure web-accessible API endpoint for your data tables. This means your Python Apps (MAgent and Evals for us) can stream text back and forth globally using a lightweight, native HTTPS connection driver without needing a persistent, heavy database connection pool. This is a significant advantage on Render’s \$7 budget container tier, where managing persistent background connections can easily trigger memory spikes and crash our deployment.

3. Generous, Zero-Risk Free Tier

Supabase gives you a 100% free production tier containing up to 500MB of storage space, which allows you to store thousands of text-heavy career reports completely free of charge. While MongoDB Atlas also has a free tier, it enforces much stricter connection limits and network throttling rules that can cause your live multi-agent applications to hang or time out while trying to authenticate during heavy traffic

What is PostgreSQL?

- PostgreSQL is a highly stable, open-source **Relational Database Management System (RDBMS)**. [[1](#), [2](#), [3](#)] (AI Resources)
- Unlike a flat Excel sheet, a relational database organizes data into rows and columns across **multiple tables** and links them together using shared identifiers (relationships).
- It has been continuously developed by a global community for over 35 years. It is widely considered the most advanced, powerful, and reliable open-source database in the world.

PostgreSQL and Vector Databases

PostgreSQL is a dominant force in modern technology, particularly for AI applications, due to several key factors:

1) **The Powerhouse Behind Vector Search (pgvector).**

2) Postgres is not just for text and numbers anymore. Through an extension called pgvector, it can natively store and search AI **vector embeddings**. This means you can run your entire **RAG pipeline** (storing document chunks and performing semantic searches) directly inside your main database, completely replacing specialized tools like ChromaDB or Pinecone if you want to streamline your stack.

3) **Enterprise-Grade Reliability:** It features complete **ACID compliance** (Atomicity, Consistency, Isolation, Durability). This guarantees that even if your RAG application crashes mid-request, your database will never corrupt, lose data, or leave your records in a broken, half-written state.

Important Resources

Always read and learn more!

What is PostgreSQL?

<https://www.heroku.com/what-is-postgresql/>

Intro to the PostgreSQL Database

<https://www.bluecrystal.com.au/news/tech/what-is-postgres/>

The benefits of PostgreSQL

<https://www.prisma.io/dataguide/postgresql/benefits-of-postgresql>

Why Choose PostgreSQL

<https://www.navicat.com/en/company/aboutus/blog/2403-why-choose-postgresql-for-your-next-it-project>

How PostgreSQL is Replacing all other databases

<https://medium.com/@reliabledataengineering/the-postgresql-renaissance-how-one-database-is-replacing-them-all-10b92ad766b9>

STEPS – Back to Building our Evals/Agentic System

Recall, we are using Supabase (<https://supabase.com/>) which gives us free PostgreSQL.

Step 1: Initialize Your Free Hosted PostgreSQL Database

- Open your web browser and go to **supabase.com**.
- Click the green **New Project** button (unless you already have an account – and are connected to GitHub).
- Click **Start your project** or **Sign In** and authenticate **using your existing GitHub account** (this links your code environment seamlessly). (So sign in or start with GitHub – not your email).
- Next, you will see a “Create a new organization” part. See the next slide for what I typed in. Feel free to determine what you want to type in. **Then click Create organization.**

Create a new organization

Organizations are a way to group your projects. Each organization can be configured with different team members and billing settings.

Name

agates001's GitHub Project Org

What's the name of your company or team? You can change this later.

Type

Personal

What best describes your organization?

Plan

Free - \$0/month

Which plan fits your organization's needs best? [Learn more.](#)

Cancel

Create organization

GitHub (optional)

 Connect GitHub

Ideal for agent-first workflows: update your schema in code, push it to GitHub, and Supabase deploys the changes automatically. [Learn more](#)

Project name

agates001's Project

Database password

.....

 Copy

This password is strong. [Generate a password.](#)

Region

 Americas

▼

Select the region closest to your users for the best performance.

Security

☒ Enable Data API

Autogenerate a RESTful API for your public schema.
Recommended if using a client library like [supabase-js](#).

☒ Automatically expose new tables

Grants privileges to Data API roles by default, exposing new tables.
We recommend disabling this to control access manually.

☒ Enable automatic RLS

Create an event trigger that automatically enables Row Level Security on all new tables in the public schema.

ADVANCED CONFIGURATION >

Cancel

Create new project

NOTICE: Do not click the “Connect GitHub”. We are already connected and will manage this ourselves.

Use RSL – VERY IMPORTANT

What it does: RLS stands for **Row Level Security**.

It is a built-in security shield that locks down your tables so that nobody on the internet can read or inject fake data into your database, even if they somehow steal your public API key.

Why you need it: It keeps your candidate data completely secure and protected from malicious actors.

The Critical Step: Because RLS locks everything by default, **your Python scripts will be blocked from writing data** until we add a simple security rule (policy) allowing authenticated access.


```
-- 1. Turn on Row Level Security for your table
ALTER TABLE app_reports ENABLE ROW LEVEL SECURITY;

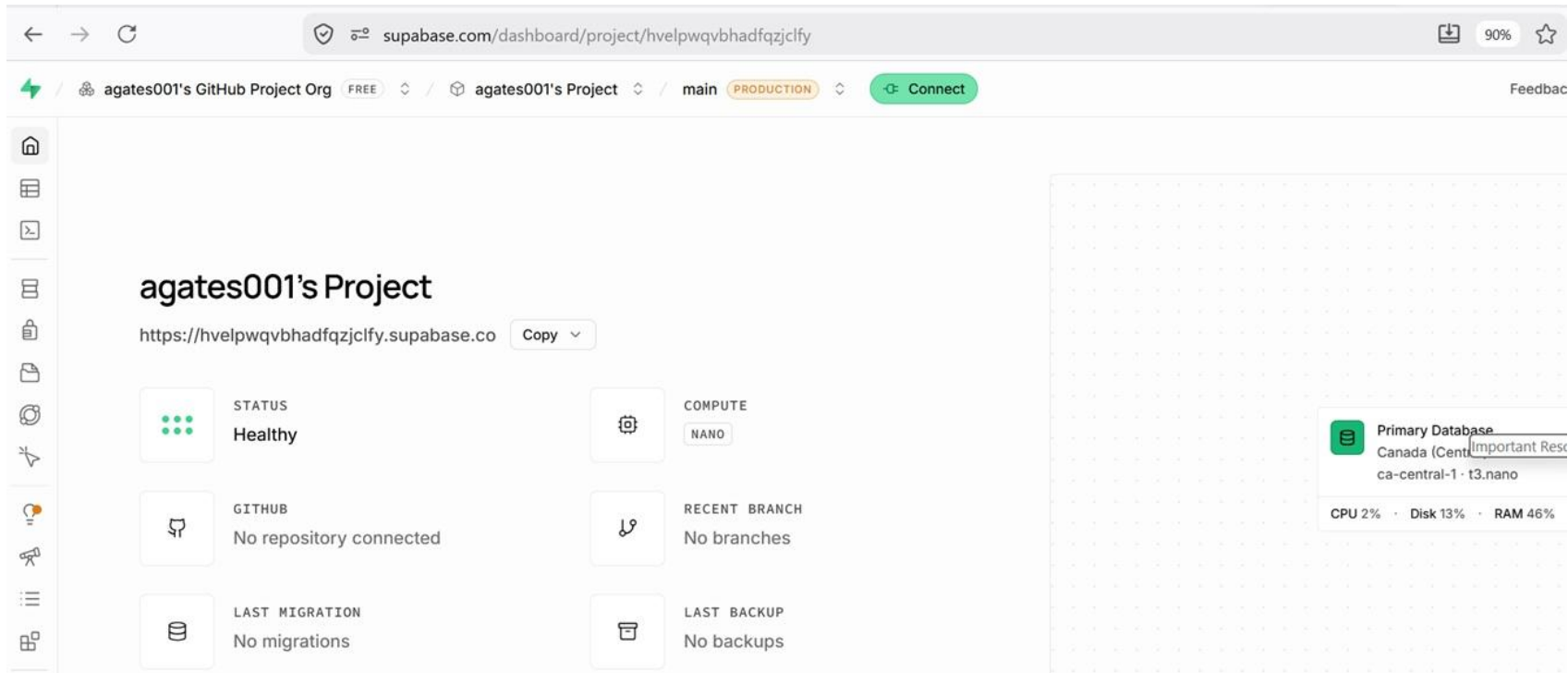
-- 2. Create a security policy that allows your apps to write
CREATE POLICY "Allow anonymous inserts"
ON app_reports
FOR INSERT
TO anon
WITH CHECK (true);

-- 3. Create a security policy that allows your apps to read
CREATE POLICY "Allow anonymous selects"
ON app_reports
FOR SELECT
TO anon
USING (true);
```

Add Python Code For RLS

How to unblock your Python code after checking the boxes

- Once you choose RLS and finish creating your project, etc. and run the table setup script in the **SQL Editor (we will get to this)**, you will need to paste and run an extra script right underneath it.
- This tells PostgreSQL to allow your Python scripts to safely write and read data using your secure API key.
- DO NOT WORRY – we will come back to this shortly. This is FYI so that you know its coming.



We are Here Now

Make sure you have this view (ish).

Review Our Next Steps

Let's review what we have so far and where we are headed.

What we have:

- 1) We have a Multi-Agent with RAG App that we wrote using Python in VSCode. Recall that we use OpenAI as our LLM and we used CrewAI as our architecture. We also have a requirements.txt file with our libraries.
- 2) Next, we pushed this MAgent App, the requirements.txt, and a resume (used by an Agent/RAG) to GitHub.
- 3) We use Streamlit and Render to run the App "live" on the web.

What we want now:

- 1) As a next step, we want to build another Agent App that I am calling the Eval App for short. This will also be coded in Python in VSCode (which we already did).
- 2) The goal here is for our MAgent and our Eval App to be able to communicate so that the Eval App can "judge" (evaluate/measure) how well our Agent App (I am calling this MAgent for short) is doing.
- 3) To get the App to communicate live on the web, we need a database and wrapper that offer an API option so that MAgent can "write" to the database and the Eval App can read it and use it to judge.
- 4) To achieve this, we used (are using) Supabase which contains PostgreSQL.

Make sure you have access to supabase (review the previous slides) and let's keep going.

Next Steps Checklist

- **Step 1:** Retrieve your project **API keys** from your **Supabase** dashboard.
- **Step 2:** Update your Main App (I call this MAgent) code to stream multi-agent markdown reports. In other words, we want to write the results to the database not to the markdown.
- **Step 3:** Update your Eval App code to pull production data live from the database.

Getting the API Keys From Supabase

- Open your **Supabase Dashboard**. You may already be there but if not, you can go back a few slides to see it.
- Click on **API Keys**.
- Copy the following values:
 - **Project URL** . This is actually a specific URL generated by Supabase to point directly to your database API.
 - Project URLs look SOMETHING like: <https://hvelpetcqzjclfy.supabase.co> where the hvelpetcqzjclfy is called Project Reference ID, and we each have our own. What you see here on this slide is a made up value so that you cannot see mine directly 😊
 - To determine yours, check on the API Keys page and look up at your own URL. Within that URL you will see this “reference ID” which is a 20 char string.
 - See the next slide for an image...

← → ↻ supabase.com/dashboard/project hvelpwqvbha: [REDACTED] settings/api-keys

agates001's GitHub Project Org FREE agates001's Project main PRODUCTION Connect

Settings

- CONFIGURATION
 - General
 - Compute and Disk
 - Infrastructure
 - Integrations
 - API Keys**
 - JWT Keys
 - Log Drains
 - Add-ons
- INTEGRATIONS
 - Data API ↗
 - Vault BETA ↗
- BILLING
 - Subscription ↗
 - Usage ↗

API Keys

Configure API keys to securely control access to your project

Publishable and secret API keys Legacy anon, service_role API keys

Your new API keys are here
We've updated our API keys to better support your application needs. [Join the discussion on GitHub](#)

Having trouble with the new API keys? [Contact support](#)

Publishable key

This key is safe to use in a browser if you have enabled Row Level Security (RLS) for your tables and configured policies.

NAME	API KEY
default No description	sb_publishable_[REDACTED]

Publishable keys can be safely shared publicly

Here you can see the 20 digits - mine starts with hvel....

Reference ID

Your Project URL

Now that you have the Project Reference ID,
you exact Project URL is:

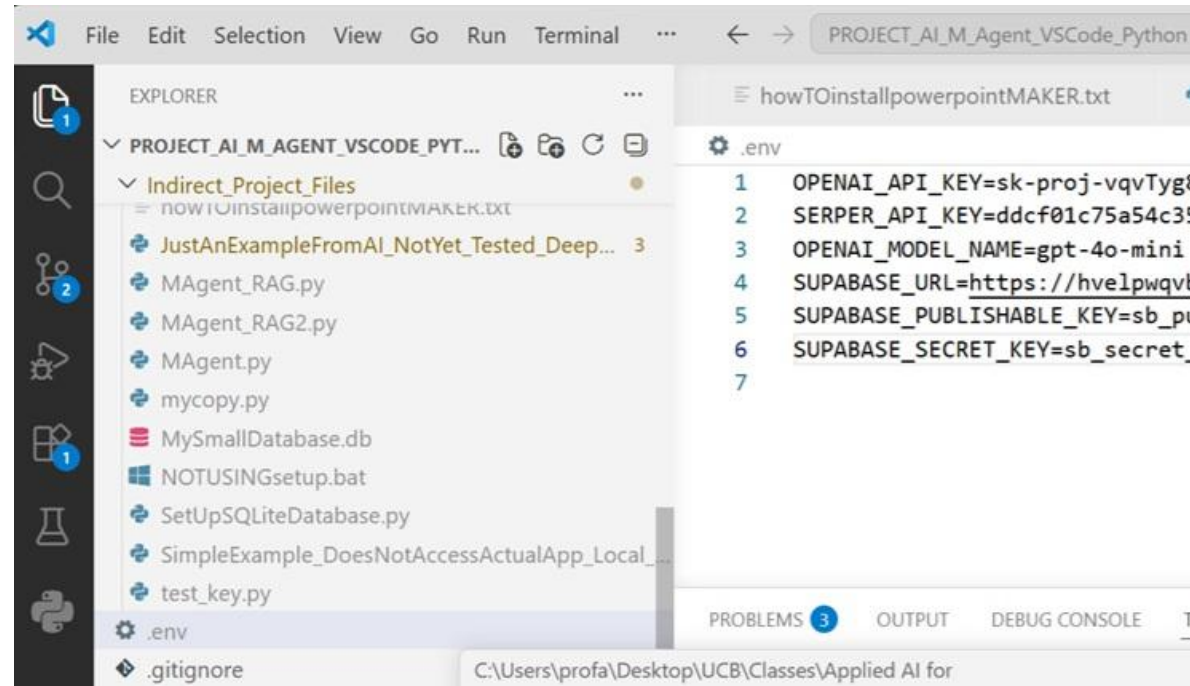
<https://yourProjectReference.supabase.co>



Next, get API Keys and Update Environment Areas in VSCode and in Render

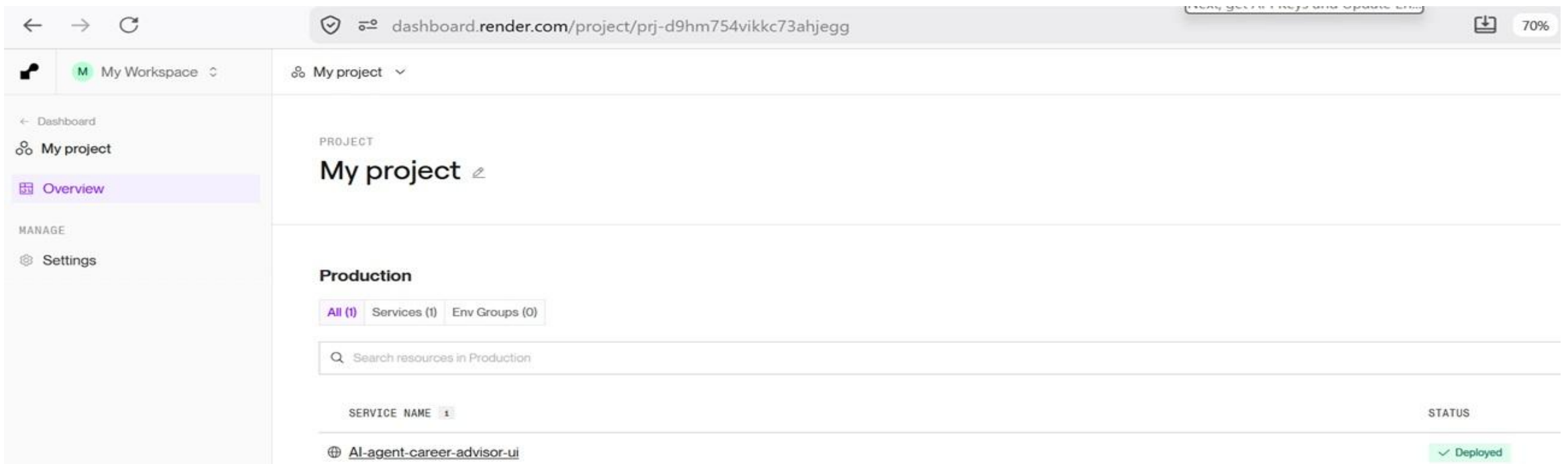
- 1) From Supabase, in the API Keys area, you will see Publishable key and Secret keys (just one).
- 2) Get them both and save them.
- 3) Next, we need to do two things. We need to add them correctly to .env in our VSCode Project area. Then, we need to update Render's Environment variables to reflect these.
- 4) Let's do this one at a time. Make sure you have both keys.

Add Supabase Keys to .env in VSCode



You must add these three lines with YOUR information:

```
SUPABASE_URL=https://yoursupabaseproject20digitref.supabase.co  
SUPABASE_PUBLISHABLE_KEY=your_sb_publishable_key_here  
SUPABASE_SECRET_KEY=your_sb_secret_key_here
```



- 1) Go to www.render.com and click “Dashboard” (upper right) or just go to <https://dashboard.render.com>
- 2) Then, log in with GitHub unless you are already logged in.
- 3) Next, click “My Project”. There you will see your Project that we placed on Render to run live on the web. Mine is called AI-agent-career-advisor-ui.

Next, update Render’s Environment Variables

Next, to Add New Env Variables to Render

- 1) Click the Project name.
- 2) Then, on the left you will see “Environment” – click this
- 3) Then you will see Environment Variables (and the ones you already put in).
- 4) From there, click Edit (near to the upperish right).
- 5) Then you will be able to see + Add variable. You can use this to add the three new env variables:

SUPABASE_URL=https://**yoursupabaseproject20digitref**.supabase.co
SUPABASE_PUBLISHABLE_KEY=your_sb_publishable_key_here
SUPABASE_SECRET_KEY=your_sb_secret_key_here

6) When you have added all three items, click the ***save, rebuild, deploy button*** to save your changes. You will see Render doing a lot of work as it rebuilds and redeploys your App. When its does you will see “**Your Service is live**”. Mine is live at : <https://ai-agent-career-advisor-ui.onrender.com> (at the time of this tutorial). **PLEASE DO NOT click the Run AI Agent on my site!** Just have a look to see what the live App looks like.

What Did We Do So Far?

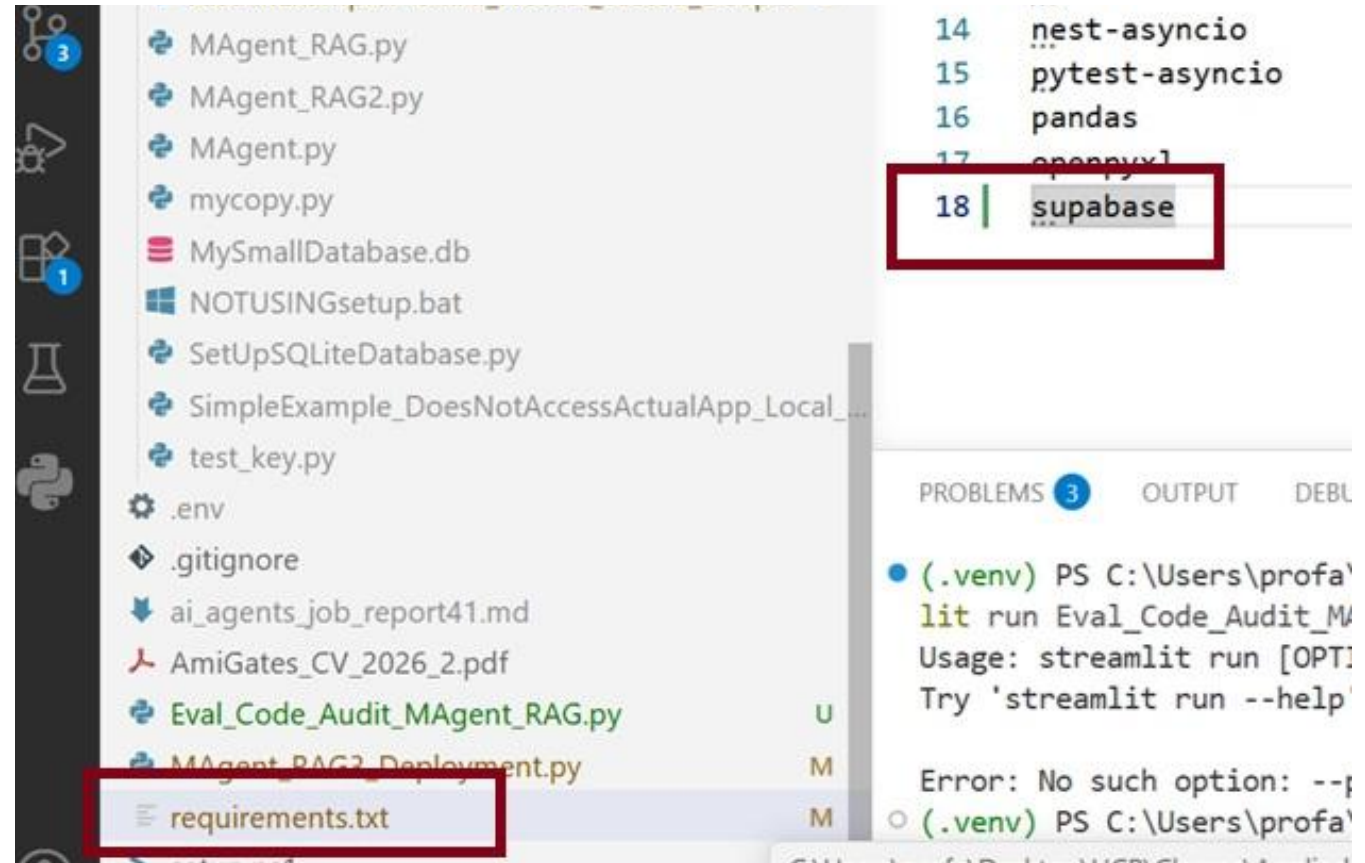
OK – let's review. Its important to do this often or you get bogged down in the details and forget the big picture 😊

- 1) Right now, we have our old App (I often call it MAgent for short) which is our multi-Agent RAG workflow that we coded in Python on VSCode. We have pushed this and a few other elements to Git, and then used streamlist (via Python libraries) and Render to live deploy it.
- 2) Next, we wrote in Python an Eval App that is another Agent that will judge/measure/test our MAgent App to see how it does with its goals. We coded and tested this in VSCode.
- 3) We now want BOTH Apps, the MAgent and the Eval App to run live *AND* to be able to communicate with each other.
- 4) Important: If we just wanted to run both Apps live, that easier. We already have the steps for that. But we want MORE! We want them to communicate live (over the web), share information (via a shared API accessible database), etc.
- 5) So, to do this, we needed the database and the wrapper, etc. We are using Supabase which uses PostgreSQL if you recall. To get this to work, we needed to log into supabase, set it up, get the new Environment variables, and place them where they below. This creates the ability (via API) for our Apps (our MAgent and our Eval) to communicate and share.
- 6) WOW!! We are almost done setting up Supabase and next we will UPDATE both App – MAgent and Eval so that they can communicate via Supabase and PostgreSQL.
- 7) Let's do these next steps.
- 8) MAKE SURE you understand where we are, where you are, what we are doing, and why.

Next, we will update code our multi-agent RAG App (I call this MAgent) in VS Code so our multi-agent system can automatically stream its markdown reports straight into that cloud table (database) to share with Eval (or any other Apps we write).

- 1) Install the Supabase library into VSCode.
 - 1) Open **requirements.txt**
 - 2) Into that area, place: **supabase**
 - 3) pip install -r requirements.txt

Install the Supabase Library



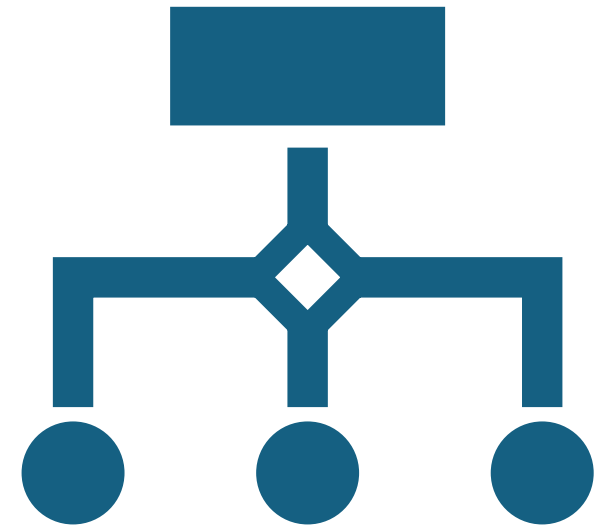
Push the new requirements.txt to github

Inside VSCode:

```
git add requirements.txt
```

```
git commit -m "chore: add supabase  
package to dependencies"
```

```
git push origin main
```



Update the MAgent App with the new Code

- Recall that we have a multi-agent workflow with RAG that we developed and deployed live. (I generally call this MAgent for short)
- Next, we need to update this code so that it can use the supabase and can communicate live with our Eval App.
- Let's make a copy of our multi AGent RAG Python code and then save that as something like ***MAgent_RAG_DBLiveShare.py*** (or whatever you like that makes sense).
- Now, let's make the following updates to the MAgentShare (which is my short nickname for this new Python file that is our multiagent with RAG that will share its output with the supabase.)

We will Need a Table Name

To update our new MAgent App so it can communicate with other Apps live Eval, we need a shared database table.

So, we need a **table name** that we can place in our code.

Let's do this first inside of the Supabase Dashboard.

- 1) Go to the Supabase dashboard and look for (on the left menu) the Table Editor.
- 2) Since this Project is new, we do not have any tables. Let's **create one**.
- 3) Click on Create a Table. Check Enable Row Level Security (RLS). DO NOT enable realtime. Then Save. See next slide for image.

Create a new table under `public`

Name

agent_reports

Description

Location where Apps/Agents Communicate and Share



Enable Row Level Security (RLS) RECOMMENDED

Restrict access to your table by enabling RLS and writing Postgres policies.



Policies are required to query data

You need to create an access policy before you can query data from this table. Without a policy, querying this table will return an empty array of results. You can create policies after saving this table.



Documentation



Enable Realtime

Broadcast changes on this table to authorized subscribers.

Cancel

Save Ctrl S

Your New Table and its Name

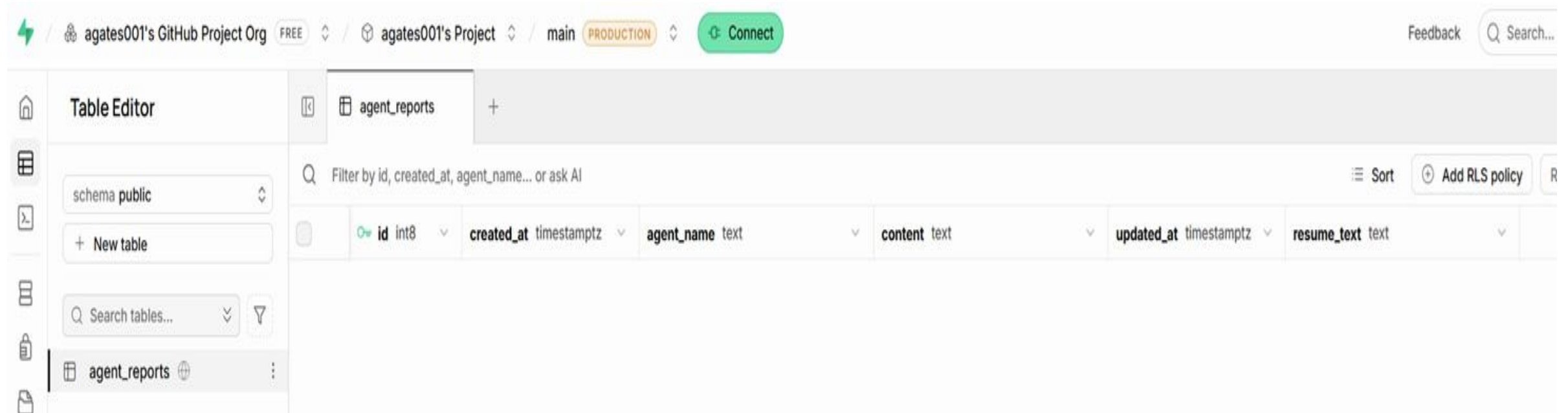
Now the Supabase will show you that you have a new empty table.

The name of the table is what you named it. I called mine: “**agent_reports**” (see the previous slide)

Make sure you know your table name. You will use it in the update code.

Making Some Additions to the Table to Have the right Columns and to Accept our Resume

- 1) Go to the Supabase Dashboard and click the “Table Editor”. There you can see your agent_reports table.
- 2) Right now, the table is empty and has only two columns, the first is called “id”. We need to build up this table.
- 3) Click the + sign and create the following new field/column called “agent_name”. The type is “text”. Then save. (You can search for text as a type if you do not want to scroll to look for it. Do not worry – its there.)
- 4) Leave the default value as NULL. Also, the only button to select below is “Allow Nullable”. This is not a primary key, etc.
- 5) Now, repeat this process and add the following from the next slide.....



Adding More To Our Database

agent_name: Type text | Allow NULL ← We did this one already

Do these:

content: Type text | Allow NULL

updated_at: Type timestampz | Allow NULL

resume_text: Type text | Allow NULL

Where Are We

1) Our new Supabase Table
“agent_reports” is now structured with six columns (id, created_at, agent_name, content, updated_at, resume_text).

2) Next, we need to further update our MAgent code in VSCode.

In the MAgent App Code, we need to get the MAgent to send the resume to the database (and not just use it locally). This way, the Eval App will also be able to “see and use” it.

Updating MAgent (the most recent) in VSCode.

```
git add MAgent_RAG_DBLiveShare.py
git commit -m "feat: extract and stream resume text to supabase cloud"
git push origin main
```

Remember that we have our most recent version of our multi-agent RAG code that will write its information to the our new supabase table.

We update the MAgent code to do this.

But, we have not yet updated that same MAgent Code to also “stream” the resume into the resume_text column of our new supabase database table.

Let’s do that now:

- 1) Open the code in VSCode.
- 2) Use my code (here on GitHub) to update your code. (Or you can use AI or any other method if you want to figure it out on your own 😊)

https://github.com/agates001/AI-agent-career-advisor/blob/main/MAgent_RAG_DBLiveShare.py

Then push to GitHub – see next slide.

Next, Update the Eval App and Push it

- 1) Just like with MAgent, make a copy and rename the Eval App in VSCode.
- 2) Notice that your Eval App (your Eval Python program in VSCode) currently reads the markdown file and your local resume.

Example:

```
target_report_path = "ai_agents_job_report41.md"  
target_resume_path = "AmiGates_CV_2026_2.pdf"
```

This will not work live on the web as these files are local. In addition, we already updated the MAgent to write its results and the resume to our new supabases database table called agent_reports.

So, we need to update our Evals Program in VSCode to use the supabase database table for its information (rather than local files). The steps follow.

The Final Code for the Eval App

You can find a completed version of the Eval App (that now uses the Supabase database table for the results and the resume) here:

https://github.com/agates001/AI-agent-career-advisor/blob/main/Eval_Code_DBLiveShare.py

Recall that this was written in VSCode and then pushed to GitHub via the following. Here, be sure to use YOUR code names 😊

```
git add Eval_Code_DBLiveShare.py
```

```
git commit -m "feat: complete live cloud data bridge for  
evaluation app" git push origin main
```

What We Now Have

Here is a Review of what we did and what we now have:

- 1) First, we wrote in Python and VSCode a multi-agent with RAG workflow (application) using OpenAI as the LLM. We tested it locally to assure that it worked. It wrote results to a Markdown file we were able to review.
- 2) Next, we used Streamlit Python libraries to deploy this **locally** as a webpage so we could see it in that view.
- 3) Then, we used Render to deploy (render) it live on the web for anyone to use.
- 4) To do this, we also used GitHub so that Render was able to grab our finished App and our local files – like the resume.
- 5) After the above, we decided to create another Agent (another App) in Python in VSCode to evaluate (judge) the results generated by our multi-Agent with RAG workflow (App).
- 6) We first did this locally.
- 7) Next, we wanted to be able to run the Eval App on the web as well. In addition, we wanted to enable both Apps (our MAgent – the multi agent workflow with RAG as well as the Eval App) to be able to communicate and share data and information live over the web – rather than just locally.
- 8) To enable live sharing, we employed Supabase which internally uses PostgreSQL. We created a table in Supabase, populated the table with columns, and then updated both our Apps – the MAgent and the Eval – so that each would write and read from the database table rather than locally.
- 9) We pushed the new and updated MAgent and Eval Python code (Apps) to GitHub.
- 10) As a LAST STEP!! We will Render them both so that they can interact and do their jobs fully deployed and live on the web.
- 11) !!! DO NOT forget that every time your run this full App – you will incur some small cost. So do not over-run.

Update and Add to Render – Start with the Latest Version of your MAgent App

- 1) Render automatically pulls from GitHub each time you push. This means that Render has all the files – the old ones and the new ones.
- 2) So, we need to get Render to run the newest versions.
- 3) As a first step, be sure that the newest versions and the newest requirements.txt are in your GitHub. They should be already – but this will be important.
- 4) Next, enter your Render Dashboard and click on your Project in there. Mine is called “**AI-agent-career-advisor-ui**” and you have to click it.
- 5) Then, click on Settings in the left nav. and scroll down to the “Start Command” field. You will need to change this to point to your NEW copy (the latest version) of the MAgent that can use the supabase, etc.

EXAMPLE: Here is what I put in my Start Command field:

streamlit run MAgent_RAG_DBLiveShare.py

Then click Save Changes which will auto-deploy this App online.

Let it finish and check it before going to the next slide and task.

This Shows
that Render
successfully
put your Final
MAgent Online

The screenshot displays the Render dashboard interface. The top navigation bar shows 'My Workspace' with a dropdown arrow, followed by 'My project', 'Production', and 'AI-agent-career-advisor-ui'. Below this, the URL 'https://ai-agent-career-advisor-ui.onrender.com' is visible. The left sidebar contains a menu with 'Environment', 'AI-agent-career-advisor-ui', 'Events' (highlighted), 'Settings', 'MONITOR' (with sub-items 'Logs' and 'Metrics'), 'MANAGE' (with sub-items 'Environment', 'Shell', 'Scaling', 'Previews', and 'Disk'), and 'Changelog'. The main content area shows the deployment details for 'AI-agent-career-advisor-ui' on the 'Production' environment. It indicates the deployment was successful on July 27, 2026, at 7:15 PM, with a 'Live' status. The commit hash 'e9b8a66' is shown, along with the description 'feat. complete live cloud data bridge for evaluation app'. Below this, there is a search bar for logs and a list of log entries for July 27. The logs show the Uvicorn server starting on port 8501, followed by instructions to view the Streamlit app in a browser. The local URL is 'http://localhost:8501', the network URL is 'http://10.30.55.49:8501', and the external URL is 'http://74.220.50.48:8501'. The logs conclude with the message 'Your service is live' and the primary URL 'https://ai-agent-career-ad'.

My Workspace ▾ My project ▾ / Production ▾ / AI-agent-career-advisor-ui ▾
<https://ai-agent-career-advisor-ui.onrender.com> □

← Environment
AI-agent-career-advisor-ui
Events
Settings

MONITOR
Logs
Metrics

MANAGE
Environment
Shell
Scaling
Previews
Disk

Changelog ▾

July 27, 2026 at 7:15 PM ✓ Live
e9b8a66 feat. complete live cloud data bridge for evaluation app

All logs ▾ 🔍 Search logs

Jul 27

07:17:23 PM [j8n99] 2026-07-28 01:17:23.977 Uvicorn server started on :::8501
07:17:24 PM [j8n99]
07:17:24 PM [j8n99] You can now view your Streamlit app in your browser.
07:17:24 PM [j8n99]
07:17:24 PM [j8n99] Local URL: <http://localhost:8501>
07:17:24 PM [j8n99] Network URL: <http://10.30.55.49:8501>
07:17:24 PM [j8n99] External URL: <http://74.220.50.48:8501>
07:17:24 PM [j8n99]
07:17:32 PM --> Your service is live 🎉
07:17:32 PM -->
07:17:32 PM --> //////////////////////////////////////
07:17:32 PM -->
07:17:32 PM --> Available at your primary URL <https://ai-agent-career-ad>
07:17:32 PM -->

Notice

Render can be a bit of a pain to work with.

For example, notice that we placed our multi-agent RAG App (the one I call MAgent for short) in “My project” in Render.

So, when you add the Eval App to Render, be sure to put it in the same location or you will have trouble.

Errors

Errors and Bugs Happen – to Everyone – for many reasons.

YOU must figure out why and then correct them.

You are welcome to get AI to help you – as this is an AI class!

You are NOT welcome to get me or the TAs to help you – as we will not be with you on the job 😊

Other notes:

1) As soon as you push something to GitHub, it automatically is also pushed to Render. So, if you get an error, fix it in VSCode, push it to Git, it will then get into Render and you need only refresh your Live Webpage.

Next – let's get the Eval App to Render as well.

Since your files are already inside your GitHub repository – we can move quickly to update Render...

- Open your **Render Dashboard** and assure you can see your other App – the MAgent App.
- Click the **New service** or +New.
- From there, you will see Git Provider as a Tab (which is good) and you will see your github username/your project name. Click this. After your click your Project, many options/fields will come up live Name, Project, Language, Branch, etc.

IMPORTANT: You will need to fill out these fields exactly right and add Environment variables. The next two slides will show images of mine to help.

Also, you will see the \$7/mo checked. Change this to 0\$

Leave the Source Code as-is.

Create a name that matches your Eval App to your liking. I used “AI-agent-career-advisor-evaluator”

See next slide for more detail and for what to place under Build Command.

Language

Choose the **runtime environment** for this service.

Python 3

Branch

The Git branch to build and deploy.

main

Region

Your services in the same **region** can communicate over a **private network**. You currently have services running in **Ohio**.

Ohio (US East)

1 existing service

Deploy in a new region +

Root Directory Optional

If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a **monorepo**.

e.g. src

Build Command

Render runs this command to build your app before each deploy.

```
$ pip install -r requirements.txt
```

Start Command

Render runs this command to start your app with each deploy.

```
$ streamlit run Eval_Code_DBLiveShare.py
```

Make sure the name of the file matches what you have in GitHub EXACTLY

So Far You Have something like the above – do not deploy yet – you still need to Add the Environment Variables

Environment Variables

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)

OPENAI_API_KEY	
SERPER_API_KEY	
OPENAI_MODEL_NAME	
SUPABASE_URL	
SUPABASE_PUBLISHABLE_KEY	
PYTHON_VERSION	

+ Add Environment Variable

 Add from .env

Add Environment Variables.

© 2026 Dr. Ami M. Gates|
All Rights Reserved | Free
Distribution Only

Update the Python
Version on Render
for the Eval App to
the Same Version
as your VSCode
Which is 3.12.x.
Mine is 3.12.

This is easy to miss and will break your Eval Application – and you will not know why and AI will likely not know why either and will send you on a wild chase.

So – you are welcome!

Remember – CrewAI and many other AI Libraries we are using are not yet compatible with > Python 3.12.x. I am sure they will be eventually – but not today!

Also even though we already adjusted this in Render for our MAgent App, Render treats each App separately. This is why we also needed to add the Environment variables again for our Eval App.

So, on the previous slide notice that we had to add the env var that notes the python version.

If you forget – you can update it later....

© 2026 Dr. Ami M. Gates |
All Rights Reserved | Free
Distribution Only

The screenshot shows the Render dashboard interface. On the left is a sidebar with navigation options: Dashboard, AI-agent-career-advisor-evalua..., Events, Settings, MONITOR (Logs, Metrics), and MANAGE (Environment, Shell, Scaling, Previews, Disk, One-Off Jobs). The 'Environment' option is highlighted. The main content area shows the service 'AI-agent-career-advisor-evaluator' with its Service ID, repository, and URL. Below this is the 'Environment' section, which includes 'Environment Variables'. A description states: 'Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)'

KEY	VALUE
OPENAI_API_KEY	
SUPABASE_PUBLISHABLE_KEY	
SUPABASE_URL	
PYTHON_VERSION	3.12.10

At the bottom of the table is a '+ Add variable' button with a dropdown arrow.

Next, click
Edit and then
Add Variable.

The type into the new variable:

PYTHON_VERSION

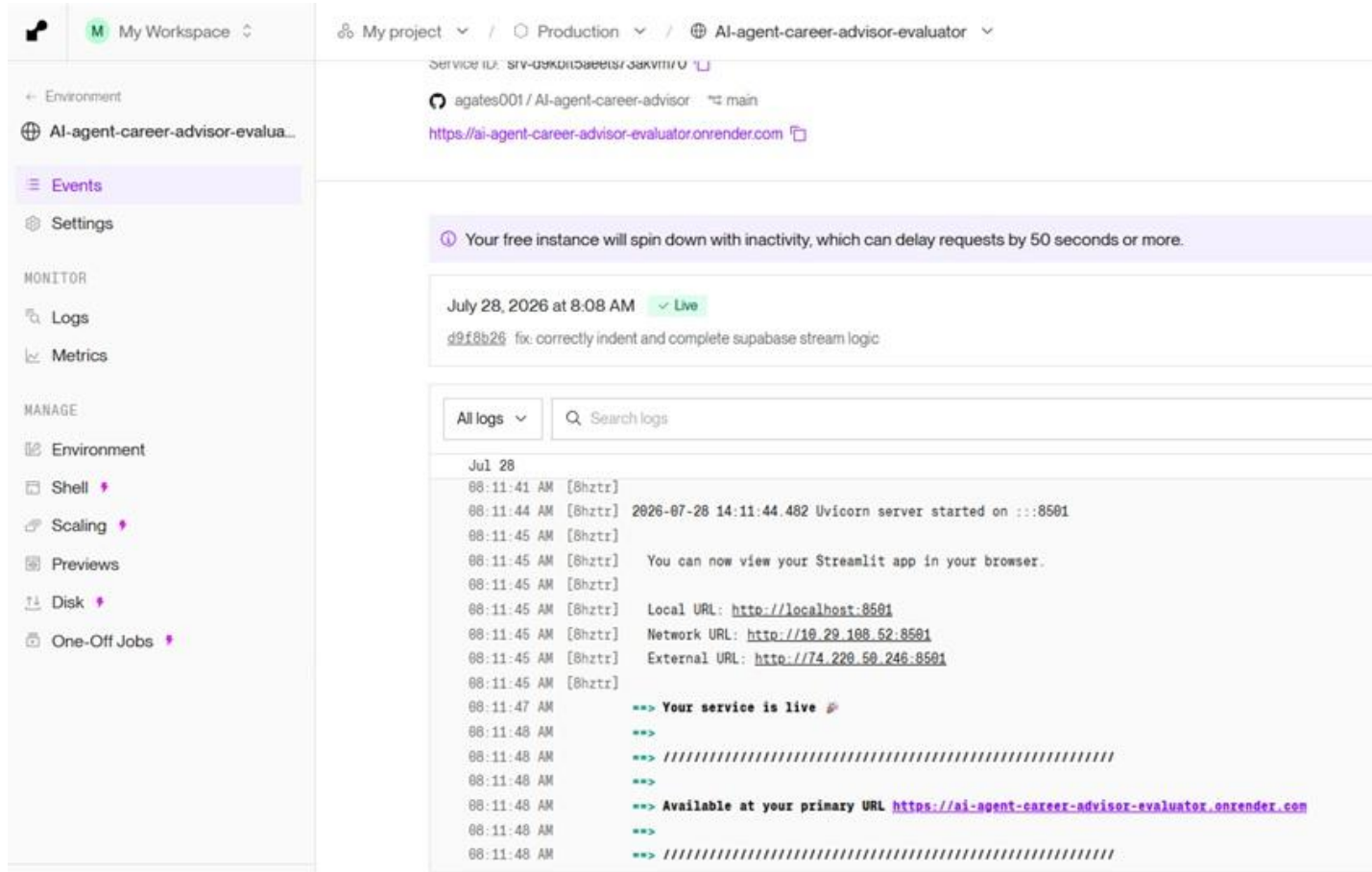
3.12.10

I used ***python --version*** to get the Python version I am using in VSCode. You can do the same.

Then click Save, rebuild, and deploy. This takes sometimes a minute or two. You can always get errors – it happens. Do not panic. You will need to track them down and fix them.

Now we can Launch – Click Deploy Web Service

Render will do many things – be patient – at the end you will see the following. I shared my URL here – DO NOT use it or click it! Remember – we pay for these services – I do as well!



The screenshot shows the Render dashboard for a project named "AI-agent-career-advisor-evaluator". The left sidebar contains navigation options: Environment, Events, Settings, MONITOR (Logs, Metrics), and MANAGE (Environment, Shell, Scaling, Previews, Disk, One-Off Jobs). The main panel displays the service status as "Live" with a timestamp of "July 28, 2026 at 8:08 AM". A warning message states: "Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more." Below this, a log entry shows the deployment process, including the start of the Uvicorn server and the generation of local, network, and external URLs. The final status message is "Your service is live" followed by the primary URL: <https://ai-agent-career-advisor-evaluator.onrender.com>.

This can take a few minutes .
Once its done, if its successful, you will see your URL which you can copy/paste to a web browser to view the App live.

This can take a few minutes as well.

The Webpage with the Eval App

DO NOT click to run my Apps – each time costs money. This is an example of what you will see. You can run your own Apps 😊



ai-agent-career-advisor-evaluator.onrender.com



Independent App Evaluation Dashboard

This standalone program audits the output files generated by your main Multi-Agent/RAG application.



Run Quality Audit on Latest App Output

If Everything Worked – You will see Both Apps Deployed in “My project” in Render

The screenshot shows the Render dashboard interface. The browser address bar displays the URL `dashboard.render.com/project/prj-d9hm754vikkc73ahjegg`. The left sidebar contains navigation links: 'Dashboard', 'My project', 'Overview' (highlighted), and 'Settings' under the 'MANAGE' section. The main content area shows the 'My project' overview. Under the 'Production' environment, there are two tabs: 'All (2)' and 'Services (2)'. A search bar is present with the text 'Search resources in Production'. Below this, a table lists the deployed services:

SERVICE NAME	STATUS
AI-agent-career-advisor-evaluator	✓ Deployed
AI-agent-career-advisor-ui	✓ Deployed

At the bottom of the table, there is a '+ New service' button.

The RLS (Row Level Security) Step

Recall that we selected RLS Options to avoid others from doing naughty things.

- Now, at the end and before we run the two Apps, we need to take one more step. Right now, because we are using RLS, Supabase locks the database gate and blocks incoming data unless a specific authorization rule allows that data through.
- Now, in “real life” there can be many more complicated methods connected to security that we cannot cover in this class. We will create a nice work-around that is secure for our purposes.
- We will add a standard **Public Select Policy** right inside your Supabase settings dashboard.
 - 1) Log in to your Supabase Dashboard.
 - 2) Enter the Project and choose Authentication from the left nag

agates001's Github Project Org

agates001's Project

main

Production

Connect

Project Overview

Table Editor

SQL Editor

Database

Authentication

Storage

Edge Functions

Realtime

Advisors

Observability

Logs

agates001's Project

Go to Authentication

G then A

Copy

STATUS

Healthy

COMPUTE
NANO

RECENT BRANCH
No branches

LAST MIGRATION

LAST BACKUP

© 2026 Dr. Ami M. Gates| All Rights Reserved | Free Distribution Only

Next, Do the Following

- Select “Policies” from the inner submenu
- Then, find your **agent_reports** table in the main list and click the **Create Policy** button on the far right side of its row.
- Clicking Create Policy will open up the configuration wizard panel where we can tell the database to allow your apps to read the data rows safely.

Complete the fields as follows:

Policy Name: Type: Allow public read access

Policy Behavior: Click the dropdown menu and select **SELECT the Permissive option.**

Policy Command for clause: Choose “SELECT”

Roles: Leave it as public (or anon).

Now – check the next slide for an image showing where to type “true” and then click Save policy.

Notice the image on the right shows all the steps we did.

The final step here is to change line 7 to the word true.

Then you can click the Save policy button.

×

Create a new Row Level Security policy

Policy Name

Table on clause

Allow public read access

public.agent_reports

Policy Behavior as clause

Permissive

Policy Command for clause

☒ SELECT

☐ INSERT

☐ UPDATE

☐ DELETE

☐ ALL

Target Roles to clause

Defaults to all (public) roles if none selected

USE OPTIONS ABOVE TO EDIT

1 create policy "Allow public read access"

2 on "public"."agent_reports"

3 as PERMISSIVE

4 for SELECT

5 to public

6 using (

7 true

8);

Cancel

Save policy

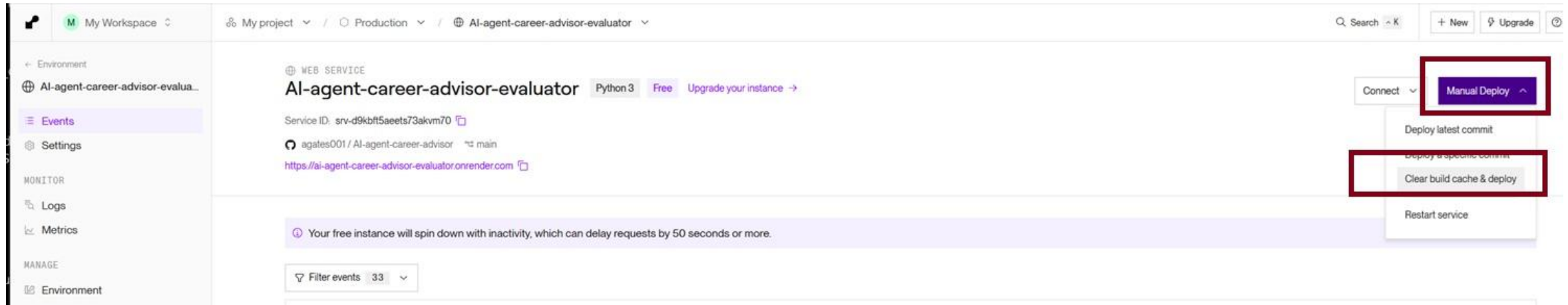
Refresh and Rebuild your Apps on Render

Even though we updated the RLS, Render has cached versions.

We must first clean this up and force a rebuild.

Rebuild the Main MAgent App (AI-agent-career-advisor-ui in Render for me)

- Close any Apps Online.
- Open your **Render Dashboard** (://render.com) and open **My Project**.
- Click on your main **AI-agent-career-advisor-ui** web service slot.
- Look up at the top right corner and click the right arrow next on the **Manual Deploy** button.
- Select **Clear build cache and deploy** from the dropdown menu.
- When its done you will see something like – Your Service is live and you will also see your APP URL. Mine is: <https://ai-agent-career-advisor-ui.onrender.com> (**DO NOT run my App – I am just showing you what the URL looks likeish**)
- Go back the My project and do the same for the Eval App (called AI-agent-career-advisor-evaluator on mine.)



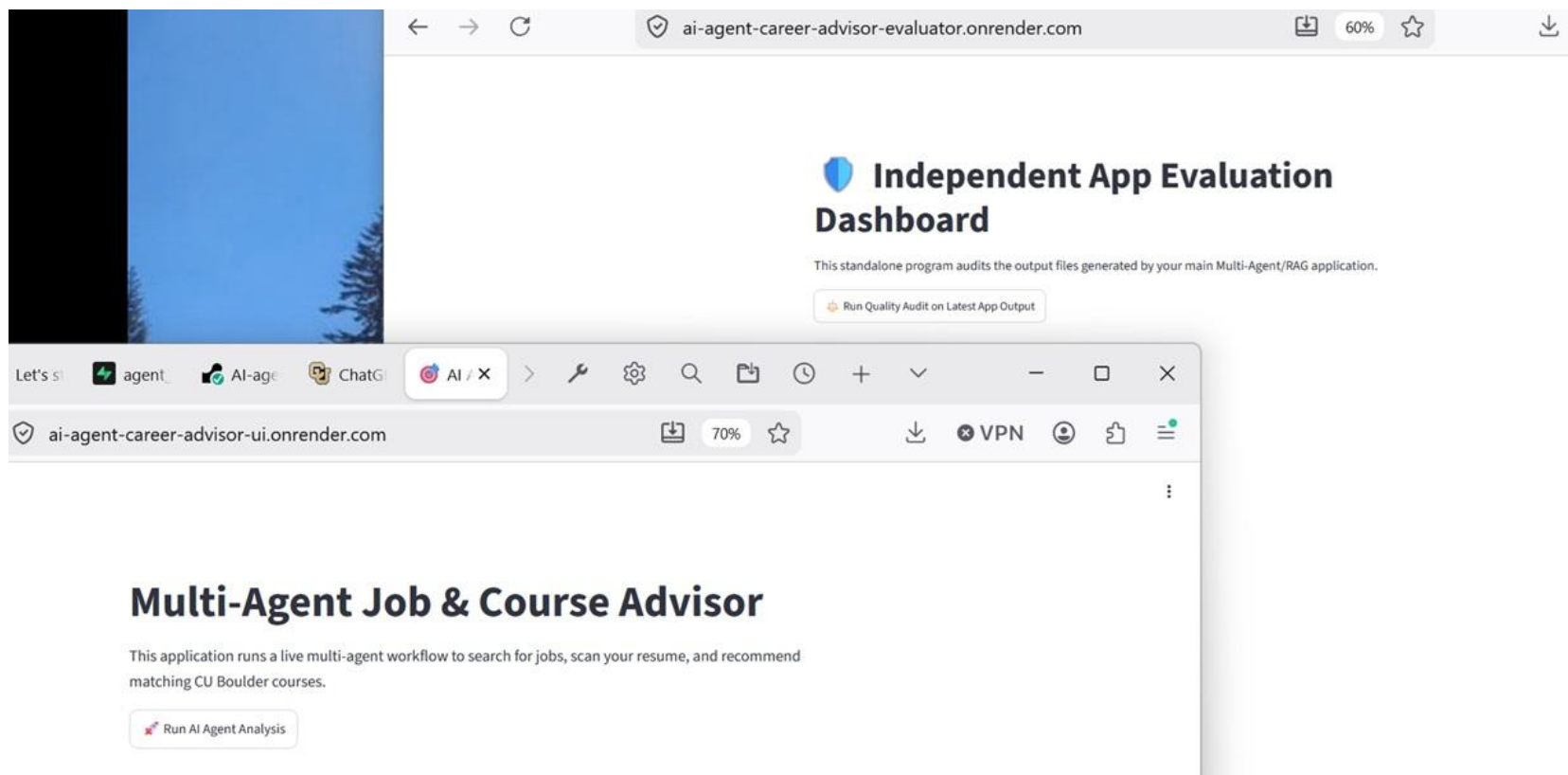
Rebuilding the Eval App in My projects in Render

After Both Are Re-Built

Now – both your Apps are rebuilt and should respond to the changes make to the RLS permissions, etc.

This should enable the MAgent App to write to your Supabase database table and should allow the Eval App to read it for its evaluation.

To check this, run both Apps. You can do this by doing to the App URLs on any Browser. Open both in different tabs.

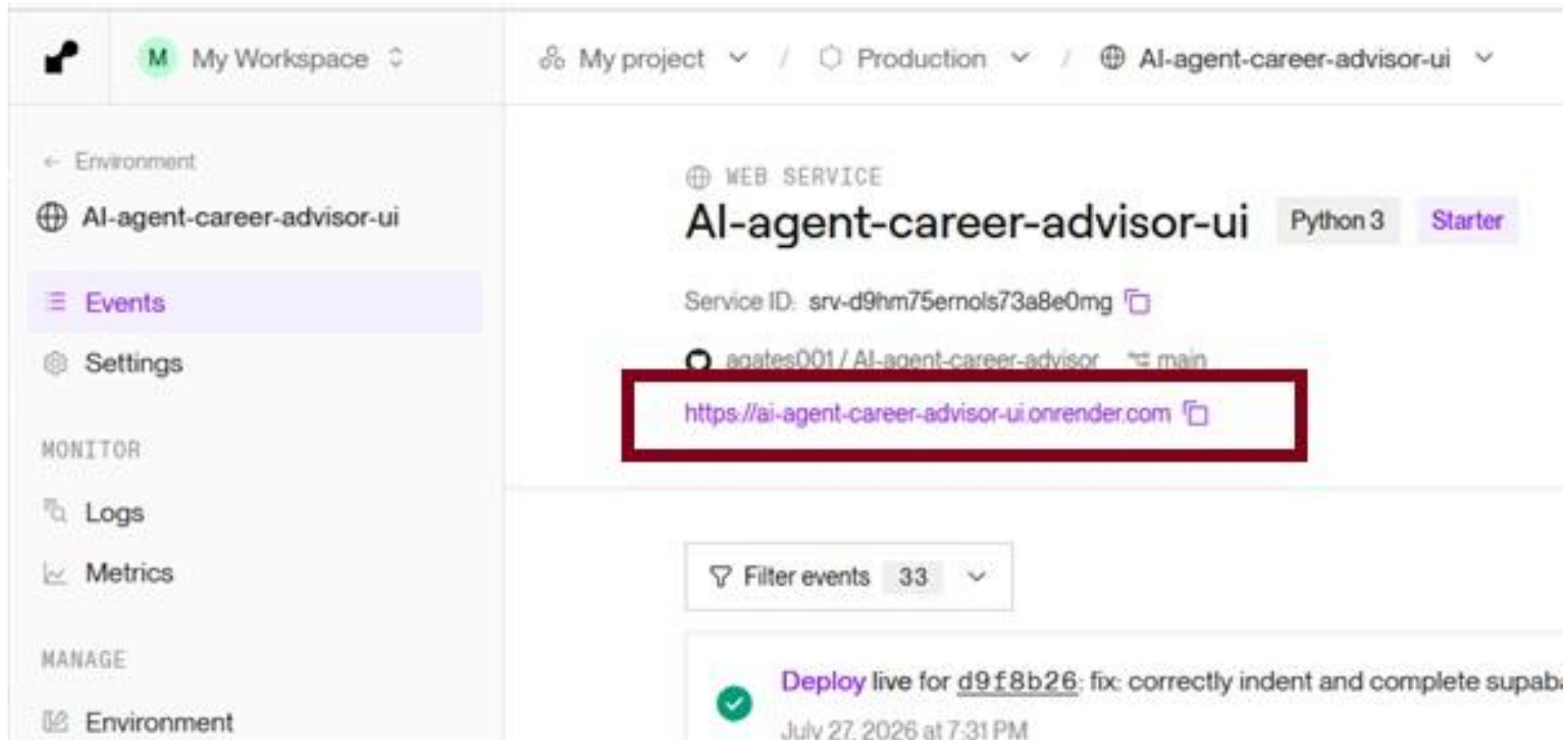


First – WOW!! We made it. Think about all you learned!! You coded a multi agent work flow – live deployed – then coded an eval agent – live deployed – and used a database for their communication. That’s HUGE.

You should each of your Apps – the MAgent and the Eval open as show on the right. In Render, you will have two URLs. Open them both. **Mine are above.**

HOW TO Run the Final Complete MAgent and Eval Apps

Reminder: Finding Web Links to Your Apps in Render





Multi-Agent Job & Course Advisor

This application runs a live multi-agent workflow to search for jobs, scan your resume, and recommend matching CU Boulder courses.

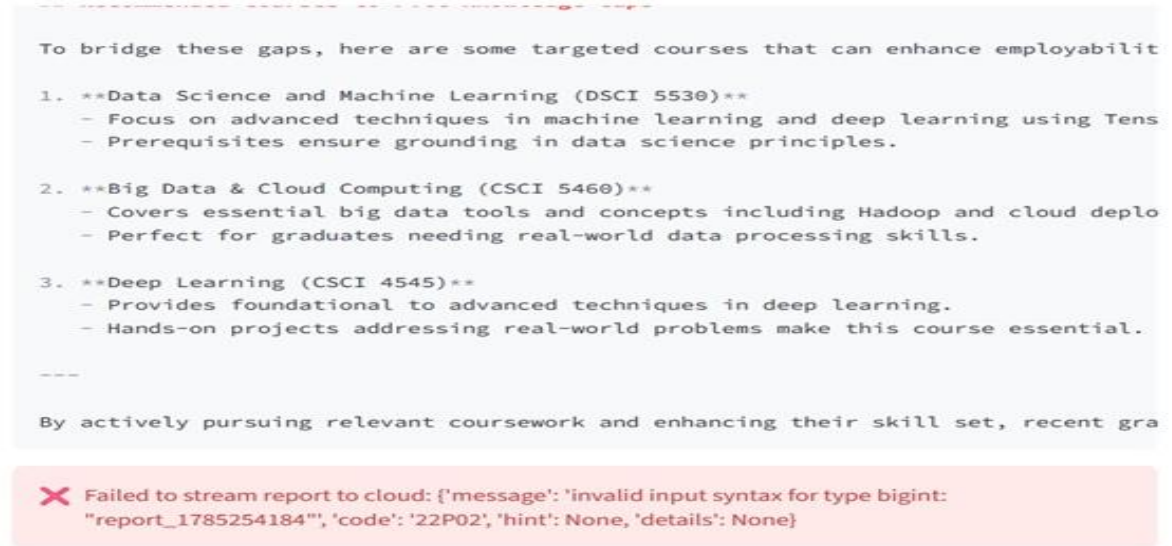


Click to Run the
MAgent

Next

- 1) **Run your MAgent first.** You have to because without its results, the Eval cannot evaluate the results 😊
- 2) Once it runs, you can and should save the Page as a pdf.
- 3) You should also check your Supabase database columns. It should have populated.

What If It Fails?



Errors and failures are not only the best learning experience ever – but they are common. Do not let them upset you. If you had to do this entire Project from scratch without these slides and images, imagine how many errors and backups you might have to have done

Even so, you will still run into problems.

Here is an example of one. This means that my MAgent did not write to the Supabase (our cloud database). This also means that our Eval cannot run.

Let's investigate this.

Learning About Errors with Google and AI

When you do/create something new, you often have two choices. First, you can try to figure out errors yourself. This can be quite a challenge. Next, if you are lucky, you can Google it or ask AI or both. Either way, you will need to learn more about, track down, and try to correct the error.

In this case, AI is telling the following after I gave it a long explanation of what I am doing and what happened. The more you tell AI, the better it can help.

AI Response

Why This Is Happening

- In Postgres (the engine behind Supabase), changing a primary key column's data type from a number (bigint) to a string (text) via a visual dashboard interface often fails silently or errors out under the hood if there are internal indexes, sequence generators, or foreign key dependencies attached to it. The UI may look updated, but the raw database system remains locked to a numeric validator

AI Suggested Fix

Instead of wrestling with the visual grid settings, we can run a single direct SQL query to force Postgres to rebuild the table structure correctly.

- Open your **Supabase Dashboard** browser tab.
- Look at the left sidebar menu and click on the **SQL Editor** icon
- Click the **New Query** button at the top of the SQL panel.

NOTE from Ami – there is no New Query button. AI ****VERY**** often gives incomplete or incorrect information – all the time! YOU must resolve this. As it turns out – we are where we need to be anyway.

- Clear out anything in the code field box, paste this exact command, and click the blue **Run** button on the far right. Here again, the Run button is now green. Applications change all the time and so details do as well. That really does not matter either! We can still place the SQL into that SQL area and click Run. Let's do that....
- sql to place in there....

-- Force the id column to drop its integer constraints and convert directly to a text type

```
ALTER TABLE agent_reports
```

```
ALTER COLUMN id TYPE text;
```

This created a new error:

Error: Failed to run sql query: ERROR: 22023: identity column type must be smallint, integer, or bigint

Sharing this with AI, we get yet another suggestion. That one failed too! I noted the error to AI and it gave a third option for the SQL. (see right)

sql

-- 1. Strip away auto-increment numeric identity restriction

```
ALTER TABLE agent_reports
```

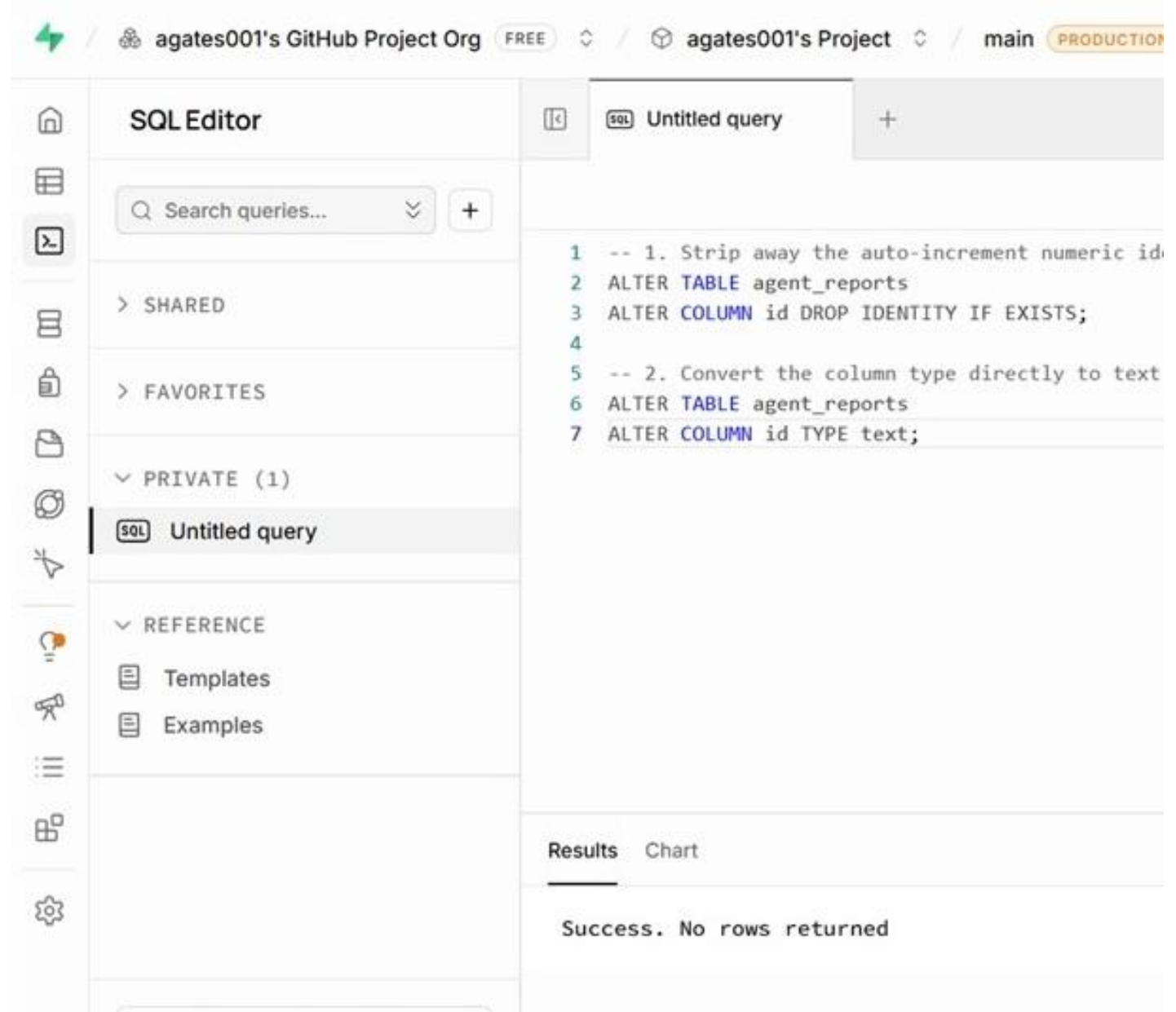
```
ALTER COLUMN id DROP IDENTITY IF EXISTS; -
```

-- 2. Convert the column type directly to text

```
ALTER TABLE agent_reports
```

```
ALTER COLUMN id TYPE text;
```

Here is What This Looks Like in Supabase



Re-Run the MAgent App Online

Let's see if it works this time!

Refresh the page.

Then Run the App there.

Same as before, we hope it runs with no errors and we double check the Supabase table ourselves to assure it populated.

Yes!! It ran and is saying it accessed the cloud database with success. Let's check that....

In Supabase, go tp the dashboard, choose Table Editor, click our table (called agent_reports). It worked 😊

The screenshot shows the Supabase Table Editor interface. The browser address bar displays the URL: `supabase.com/dashboard/project/hvelpwqvbhadfqzjclfy/editor/17480`. The interface includes a sidebar on the left with a 'Table Editor' section, a search bar for tables, and a list of tables including 'agent_reports'. The main area shows the 'public.agent_reports' table with columns: 'id', 'created_at', 'agent_name', 'content', 'updated_at', and 'resume_text'. A single row of data is visible, showing a report ID, timestamp, agent name 'RAG_Multi_Agent', and content about exploring job opportunities at Georgetown University.

	id	created_at	agent_name	content	updated_at	resume_text
	report_178525575	2026-07-28 16:22:37.487824	RAG_Multi_Agent	'''markdown # Exploring Job Opportun	2026-07-28 16:22:34.513062	Georgetown University, Washington DC

Finally – we can run the Eval App

Let's do that....run this online. It can take a few minutes – do not panic.

So the first run created an error: here it is

ai-agent-career-advisor-evaluator.onrender.com

Independent App Evaluation Dashboard

This standalone program audits the output files generated by your main Multi-Agent/RAG application.

Run Quality Audit on Latest App Output

TypeError: list indices must be integers or slices, not str

Traceback:

```
File "/opt/render/project/src/.venv/lib/python3.12/site-packages/streamlit/run
result = func()
      ^^^^^
File "/opt/render/project/src/.venv/lib/python3.12/site-packages/streamlit/run
exec(code, module.__dict__) # noqa: S102
      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/opt/render/project/src/Eval_Code_DBLiveShare.py", line 67, in <module>
generated_report_content = newest_record["content"]
                        ^^^^^^^^^^^^^^^^^^^^^^^^^
```

Again, let's Ask AI

I told AI what I did and gave it the error. It got quite excited and noted:

This is actually an incredibly exciting error [1.1]! It means your **MAgent app successfully wrote the data row to Supabase**, and your **Eval App successfully downloaded it** across the cloud.

The security gates are open, and the data bridge is working. We are just hitting a minor one-word syntax adjustment in how the script unpacks the raw database results.

Why This Happened

When you ask Supabase to download rows via Python, it returns the results as a **List** (an array) of database records.

On line 67 of your script, your code expects a single record dictionary (`newest_record`), but Supabase handed it a list containing that record instead: `[ {"id": "...", "content": "..."} ]`. When Python tries to find `["content"]` directly inside a list container rather than picking the item inside it first, it crashes with `TypeError [1.1]`.

python

```
# Add [0] to extract the record dictionary out of the list package safely
if response.data:
    newest_record = response.data[0]
    generated_report_content = newest_record["content"]
    parsed_resume_content = newest_record["resume_text"]
else:
    st.error("No records found in the database.")
    st.stop()
```

AI Suggested the above. We need to UNDERSTAND by looking at our own code that all I need to do in my case is change:

newest_record = live_reports

to

newest_record = live_reports[0] ## Including[0] so Eval grabs the top row

Let's change, push to Git, and re-Render...refresh the App and run it

Outcome

The Eval App successfully connected to and read the data!
This means the fix worked.

Here is the Eval's run and its output. Notice it graded/judged my MAgent with 1/5 (for pretty crappy).

This is GREAT! Why? Because I created the App not to be very good, but to take us through all the steps to write and deploy these Agents.

This also shows that Eval is doing its job.

Now – YOU will think about how to make the Agents better for your goals and your Project.

Independent App Evaluation Dashboard

This standalone program audits the output files generated by your main Multi-Agent/RAG application.

 Run Quality Audit on Latest App Output

 Connected! Successfully retrieved report [report_1785255754] and resume synced at 2026-07-28T16:22:34.513062+00:00.

[> !\[\]\(d66ff64371a51729ac8c1cdaa685ba6f_img.jpg\) View Extracted Production Report](#)

Final Quality Audit Metrics

Overall System Score

1 / 5

Judge Decision Reasoning: The generated career report contains several critical errors that lead to a failing score. Firstly, the job recommendations are misaligned with the candidate's qualifications. The candidate has extensive experience in data science, machine learning, and statistics, yet the report suggests entry-level positions that do not reflect the candidate's advanced expertise and teaching roles. Secondly, the report fails to recommend any courses that are new and relevant; it does

This Concludes Our Agent Coding, Testing, and Deploying (with Database Communication Live) Book/Tutorial

Thank you all! I hope you enjoyed this Guide and learned a lot about AI, VibeCoding, Agents, LLMs, Deployment, Database Cloud Sharing, Prompt Engineering, Safely Using AI, and Using other tools like CrewAI, Streamlit, Render, GitHub, and Supabase.

All the best,

Dr. Ami Gates

Professor of Computer Science and Data Science